

SYSTEM USER'S GUIDE

PDP-8 FAMILY

SYSTEM USER'S GUIDE
FOR
PDP-8/I
PDP-8/L
AND
PDP-8
PDP-8/S
PDP-5

For additional copies order No. DEC-08-NGCB-D from Program Library, Digital
Equipment Corporation, Maynard, Mass. Price \$3.00

1st Printing December 1966
2nd Printing (Rev) August 1968
3rd Printing February 1969
4th Printing (Rev) March 1969
5th Printing May 1969

Your attention is invited to the last two pages of this manual. The How To Obtain Revisions and Corrections offers you a means of keeping up-to-date with DEC's software. The Reader's Comments Card, when filled in and returned, is beneficial to both you and DEC. Each card received is considered when documenting subsequent manuals, and where the comments imply or ask for assistance, a knowledgeable DEC representative will contact you.

Copyright © 1966, 1968, 1969 by Digital Equipment Corporation

Documents Referenced:

Read-In Mode Loader	DEC-08-LRAA-D
Binary Loader	DEC-08-LBAA-D
HELP Loader	DEC-08-LHAA-D
Symbolic Tape Editor	DEC-08-ESAB-D
PAL III Symbolic Assembler	DEC-08-ASAC-D
MACRO-8 Assembler	DEC-08-CMAA-D
DDT-8	DEC-08-CDAA-D
ODT-8	DEC-08-COCO-D
FOCAL	DEC-08-AJAC-D
FORTTRAN (4K)	DEC-08-AFC0-D
FORTTRAN (8K)	DEC-08-A2A0-D
TC01 Bootstrap Loader	DEC-08-LUAA-D
DECtape Programming	DEC-08-SUB0-D
TC01/TU55 DECtape Formatter	DEC-08-EUFA-D
Disk Monitor System	DEC-D8-SDAA-D
PDP-8 Users Handbook	F-85
Small Computer Handbook	C-800

The following are registered trademarks of Digital Equipment Corporation, Maynard, Massachusetts:

DEC	PDP
FLIP CHIP	FOCAL
DIGITAL	COMPUTER LAB

CONTENTS

SECTION 1 INTRODUCTION

Console Switch Positioning

Introduction

How To Use The Guide

System and Utility Programs

Computer Console Switches and Indicators

Switches

Indicators

ASR33 Teletype

Power Controls

Printer

Keyboard

Paper Tape Reader

Paper Tape Punch

High-Speed Tape Reader and Punch Units

Reader Unit

Punch Unit

DECtape Control and Transport Units

Control Unit

Transport Unit

Initializing the System

Paper Tape Formats

Abbreviations

SECTION 2 SYSTEM PROGRAMS

Read-In Mode (RIM) Loader

Purpose

Storage Requirements

Loading

Binary (BIN) Loader

Purpose

Storage Requirements

CONTENTS (Cont)

Loading	
HELP Loader	
Purpose	
Storage Requirements	
Loading	
Symbolic Tape Editor	
Purpose	
Storage Requirements	
Loading	
Operating Modes	
Restart Procedures	
Special Key Functions	
Search Feature In Command Mode	
Switch Register Options	
Commands	
Buffer Overflow	
Diagnostics	
PAL III Symbolic Assembler	
Purpose	
Storage Requirements	
Loading	
Output Control	
Diagnostics	
MACRO-8 Symbolic Assembler	
Purpose	
Storage Requirements	
Loading	
Symbol Table Modifications	
Switch Register Options	
Diagnostics	
DDT-8	
Purpose	
Storage Requirements	
Loading	

CONTENTS (Cont)

Restart Procedure

Editing Notes

Commands

Diagnostics

ODT-8

Purpose

Storage Requirements

Loading

Commands

Relocating the Breakpoint

Restrictions

Diagnostics

FOCAL

Purpose

Storage Requirements

Optional Equipment

Loading

Initial Dialogue

Commands

The Trace Feature

Control Characters

Summary of Functions

Arithmetic Operations

Overload Recovery

Loading Program Tapes

Estimating Length of User's Program

Calculating Trigonometric Functions

Diagnostics

FORTAN (4K)

Purpose

Storage Requirements

Loading

Symbolprint

CONTENTS (Cont)

- Input/Output Control
- DECtape I/O Statements Option
- Dynamic Error Correction
- Diagnostics

FORTRAN (8K)

- Purpose
- Storage Requirements
- Equipment Requirements
- Loading
- Diagnostics

SECTION 3 DECTAPE

TC01 Bootstrap Loader

- Purpose
- Storage Requirements
- Required Equipment
- Loading

DECTape Library System

- Purpose
- Storage Requirements
- Equipment Requirement
- Loading
- Library System

SECTION 4 DISK MONITOR SYSTEM

Disk System Builder

- Purpose
- Storage Requirements
- Equipment Requirements
- Loading
- Building a Monitor
- System Modes

CONTENTS (Cont)

Disk System Bootstrap Loader

Purpose

Storage Requirements

Required Equipment

Loading

Disk System Program Library

Purpose

Storage Requirements

Loading

Disk Library

SECTION 5 SYSTEM DEMONSTRATIONS

System Demonstrations

A FORTRAN Program Calling a PAL III Subprogram

A FOCAL Program Calling a PAL III Subprogram

A FOCAL Program

APPENDIX A ASCII CHARACTER SET

APPENDIX B GLOSSARY OF TERMS

APPENDIX C OFF-LINE TAPE PREPARATION AND EDITING

APPENDIX D SUMMARY OF PDP-8/I SUBROUTINES

ILLUSTRATIONS

INTRO-1	PDP-8/I Computer Console
INTRO-2	ASR33 Teletype Console
INTRO-3	ASR33 Teletype Keyboard
INTRO-4	High-Speed Paper Tape Reader and Punch Units
INTRO-5	DECtape Transport Unit
RIM-1	Loading the RIM Loader
RIM-2	Checking the RIM Loader
BIN-1	Loading the BIN Loader

ILLUSTRATIONS (Cont)

BIN-2	Loading a Binary Coded Object Tape Using BIN
HELP-1	Loading the HELP Loader
HELP-2	Checking the HELP Loader
HELP-3	Loading the HELP Bootstrap Tape Into Core
EDIT-1	Generating a Symbolic Program On-Line Using Editor
EDIT-2	Generating A Symbolic Tape Using Editor
EDIT-3	Loading a Symbolic Tape Using Editor
PAL-1	Assembling with PAL III Using Low-Speed Reader/Punch
PAL-2	Assembling with PAL III Using High-Speed Reader/Punch
MACRO-1	Assembling a MACRO-8 Source Program Using the Low-Speed Reader/Punch
MACRO-2	Assembling a MACRO-8 Source Program Using the High-Speed Reader/Punch
DDT-1	Loading and Executing DDT-8
DDT-2	Loading DDT-8 and External Symbol Table Tapes (LSR Only)
DDT-3	Appending New Symbols to External Symbol Table
DDT-4	Generating New External Symbol Tape Off-Line (TTY and LSP Only)
ODT-1	Loading and Executing ODT-8
ODT-2	Generating Binary Tape Using High-Speed Punch
FOCAL-1	Activating FOCAL For Use
FORTRAN(4)-1	Compiling a FORTRAN Program
FORTRAN(4)-2	Generating a Memory Map Using Symbolprint
FORTRAN(4)-3	Loading a Compiled FORTRAN Program
FORTRAN(4)-4	Executing a Compiled FORTRAN Program
FORTRAN(8K)-1	Loading BIN into Data Field 1 Using RIM with HSR
FORTRAN(8K)-2	Loading FORTRAN Compiler (Pass 1 or 2) Into Field 1 Using BIN with HSR
FORTRAN(8K)-3	Executing Pass 1 of the FORTRAN Compiler
FORTRAN(8K)-4	Executing Pass 2 of the FORTRAN Compiler
FORTRAN(8K)-5	Loading Relocating Linking Loader using BIN with HSR
FORTRAN(8K)-6	Loading Relocatable Binary Tapes Using the Relocating Linking Loader
FORTRAN(8K)-7	Executing Stored FORTRAN(8K) Program
DECTAPE-1	Toggling in TC01 Bootstrap Loader
DECTAPE-2	Loading the TC01 Bootstrap Loader Using RIM
DECTAPE-3	Checking TC01 Bootstrap Loader
DECTAPE-4	Loading DECTape Library System Using TC01 Bootstrap Loader
DISK-1	Loading the Disk System Builder Using the BIN Loader

PREFACE

The purpose of this guide is twofold: (1) To familiarize the new user with the PDP-8 family of computers and its input/output devices, and to serve as a useful reference for the experienced user. (2) To furnish precise instructions on how to load, execute, and operate DEC's system and utility programs with and without extended memory, DECdisk, and DECTape facilities.

The first section explains the use of switches and indicators on the computer, Teletype, high-speed paper tape reader/punch, and DECTape transport consoles, and other general information. Precise operating procedures for using DEC's software is found in the second section. Subsequent sections cover the use of the DECTape transport unit, the Disk Monitor System, and demonstrations of the system in use. The appendices include a glossary of terms, and other useful information.

Unless specified, flowcharts apply equally to the PDP-8/1, 8, 8/S, and 8/L computers, as does the text even though reference is usually made only to the PDP-8/1.

Details on peripherals not covered in this guide can be found in the PDP-8 Users Handbook (F-85), PDP-8/1 Small Computer Handbook (C-800), and other DEC publications.

SECTION 1
INTRODUCTION

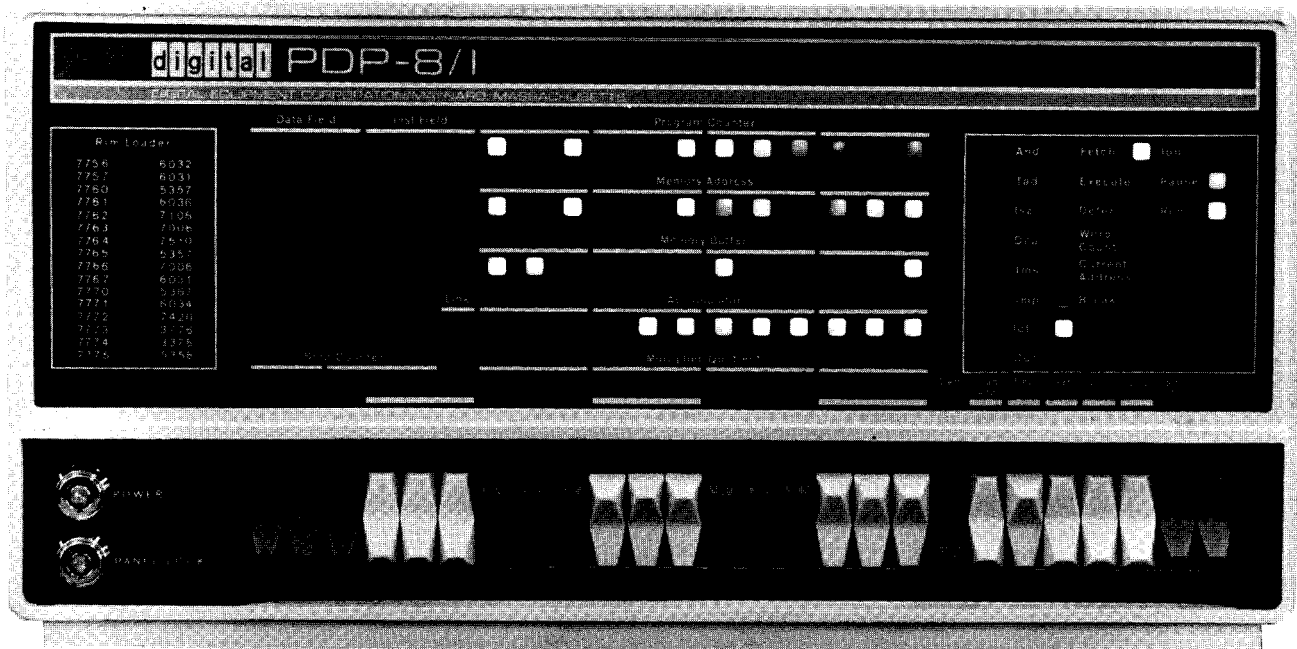


Figure INTRO-1 PDP-8/I Computer Console

CONSOLE SWITCH POSITIONING

PDP-8/I

When the top of a switch is out (push the bottom of the switch in) it represents a binary 1 or is considered set, conversely, when the bottom of the switch is out it represents a binary 0 or is not set.

PDP-8, 8/S,
and 8/L

When the switch is up it represents a binary 1 or is considered set, conversely, when the switch is down it represents a binary 0 or is not set.

Note: PDP and Programmed Data Processor are registered trademarks of Digital Equipment Corporation.

This System User's Guide is intended for use at the PDP-8/I computer¹ console when operating with DEC's software. The purpose of the guide is to furnish the user precise instructions on how to load, execute, and operate DEC's system and utility programs with and without extended memory, DECTape, and DECdisk².

This section briefly defines each program (see tabs) in Section 2, and will familiarize the new user with the switches and indicators on the computer console, as well as the controls, keys, and switches on the Teletype, high-speed paper tape reader/punch, and DECTape transport consoles. Subsequent subsections describe the various paper tape formats, the procedures for initializing the computer and input/output (I/O) devices, a list of the abbreviations used herein, and other general information. For the experienced user, this section serves as a handy source of frequently needed information.

Section 2 is tab indexed by system and utility program. The purpose of each program is given, followed by specific core memory requirements and program origin, possible optional hardware, step-by-step instructions (detailed in flowcharts) for loading, executing, and operating the program, summaries of on-line commands and error diagnostics if any, and other useful information.

Section 3 covers the use of DECTape. The TC01/TU55 Bootstrap Loader and Library System are briefly described, and their operation is thoroughly detailed. The five permanent Library System subprograms are defined, and where applicable, system questions are shown, explained, and answered.

Section 4 explains the use of the Disk System Builder and briefly describes the library of system and utility programs presently available with the Disk Monitor System.

Section 5 contains three runnable programs to demonstrate the ease with which DEC's system, utility, and service programs can be used. The new user can familiarize himself with his PDP-8/I and DEC-supplied software by duplicating the demonstration programs.

¹ Unless specified, reference to the PDP-8/I computer also applies to the PDP-8, PDP-8/S and PDP-8/L computers.

² DEC's disk system is thoroughly documented in the PDP-8/I Disk Monitor System (DEC-D8-SDAA-D).

The appendices include a list of the ASCII (USA Standard Code for Information Interchange) Character Set, a short glossary of terms, and a summary of the presently available PDP-8/I subroutines.

HOW TO USE THE GUIDE

The user should be acquainted with the material in Section 1 before attempting to operate the system, and then the guide should be made available to the user during system operation.

The programs herein are arranged generally in their order of use, i.e., the loaders precede the assemblers and the assemblers precede the debugging programs. Therefore, as the user progresses from one phase of operation to the next he will also progress from one section of the guide to the next. By scanning the tabs, the user has fast access to the essential operating information for any system or utility program.

SYSTEM AND UTILITY PROGRAMS

All system and utility programs require at least a 4K PDP-8/I computer with an ASR33 Teletype, and can utilize a high-speed paper tape reader/punch, with the following exceptions.

- a. The HELP Bootstrap tape of the HELP Loader is read into core using the low-speed (Teletype) reader only.
- b. FORTRAN (8K) requires at least an 8K PDP-8/I computer with a high-speed paper tape reader/punch.

Each tabbed program in Section 2 is briefly defined below.

Read-In Mode (RIM) Loader, used to load into core memory programs punched on paper tape in RIM format (see PAPER TAPE FORMATS), however, it is primarily used to load the BIN Loader. (See DEC-08-LRAA-D.)

Binary (BIN) Loader, used to load into core memory programs punched on paper tape in BIN format (see PAPER TAPE FORMATS), which includes the user's object programs and all programs tabbed in Section 2, excluding the RIM, BIN, and HELP Loaders. DEC-supplied paper tapes in BIN format are identified by a blue Digital label on the leader portion of the tape (red Digital labels denote ASCII format). (See DEC-08-LBAA-D.)

HELP Loader, used to load into core memory programs punched on paper tape in BIN format. HELP is in two parts: the first part consists of 11_8 instructions which must be toggled into core using the computer console switches; the second part is the HELP Bootstrap tape which

is read into core using the low-speed (Teletype) reader--the HELP Bootstrap tape contains the RIM and BIN Loaders. (See DEC-08-LHAA-D.)

Symbolic Tape Editor, used to prepare, edit, and generate symbolic (source) program tapes on-line from the Teletype keyboard. (See DEC-08-ESAB-D.)

PAL III Symbolic Assembler, used to translate source programs written in the PAL III language into object programs in two passes through the Assembler. The optional third-pass produces an octal/symbolic listing of the assembled program. (See DEC-08-ASAB-D.)

MACRO-8 Assembler, used to translate source programs written in the MACRO-8 language, containing macros and literals, into object programs in two passes through the Assembler. This Assembler also generates indirect linkages to off-page references. The optional third-pass produces an octal/symbolic assembly listing. (See DEC-08-CMAA-D.)

Dynamic Debugging Technique (DDT-8), used to aid the user in finding mistakes in his program by allowing him to execute small sections at a time, to stop execution where he wishes, and to change portions of his program, all from the keyboard using the symbolic language of the source program. (See DEC-08-CDDA-D.)

Octal Debugging Technique (ODT-8), used for the same purpose as DDT-8 (above) except that the user communicates in the octal representation of the program. ODT-8 requires less core area than DDT-8, and it can be loaded in either the upper or lower portion of core, depending on where the user's program is loaded. (See DEC-08-COCO-D.)

FOCAL (FOrmula CALculator), an on-line, conversational, service program used to solve numerical problems of any complexity; used as a programming tool by students, scientists, and engineers. (See DEC-08-AJAC-D.)

FORTRAN (4K), used to compile and operate a user program written in the 4K PDP-8 version of the FORTRAN language; compilation requires only one pass through the compiler. (See DEC-08-AFCO-D.)

FORTRAN (8K), used to compile, debug, and operate a user program written in Basic FORTRAN (as described in USA Standards); consisting of a two-pass compiler, Linking Loader, Run-Time Monitor, Operating System, and a library of subprograms. (See DEC-08-A2AO-D.)

TC01 Bootstrap Loader, used to load and start the DECtape Library System. (See DEC-08-LUAA-D.)

DECtape Library System, is a collection of five programs (INDEX, ESCAPE, UPDATE, DELETE, and GETSYS), used to load named files into core memory, define new named files, delete named files, and to create a new skeleton library system. (See DEC-08-SUB0-D.)

Disk System Builder, used to build a customized Disk Monitor System suited to the user's particular machine configuration. (See DEC-D8-SDAA-D.)

Disk/DECtape Bootstrap Loader, is used only when the resident Monitor area has been cleared or its contents otherwise destroyed. (See DEC-D8-SDAA-D.)

Disk System Program Library, is a collection of system and utility programs. The standard package includes an Editor, Assembler, transparent DDT, FORTRAN system, Peripheral Interchange Program (PIP), and a transparent Loader. (See DEC-D8-SDAA-D.)

Certain programs can operate using extended memory, DECtape, and/or Disk facilities.

They are:

- a. Extended Memory - FORTRAN (8K); Disk Monitor System programs.
- b. DECtape -- DECtape Bootstrap Loader and Library System; Disk Monitor System programs; FORTRAN (4K) with DECtape option (see DEC-08-AFCO-D).
- c. Disk - Disk Monitor System programs.

COMPUTER CONSOLE SWITCHES AND INDICATORS

Manual control of the PDP-8/I computer is by means of switches on the computer console. Indicator lamps on the console light to denote the presence of a binary one in specific bits of the various registers and to indicate the status of the computer or of the program being executed. See PDP-8 Users Handbook (F-85) for details.

The locations of the switches and indicators are shown in Figure INTRO-1. The purpose of each switch and indicator on the computer console is explained below.

SWITCHES

POWER	This key-operated switch applies and removes the computer's primary power supply.
-------	---

PANEL LOCK	This key-operated switch when turned clockwise disables all console switches except the SR; turned counterclockwise, all console switches function normally.
START	Executes the stored computer program, starting at the address specified in the PC.
LOAD ADDRESS	Sets the contents of the SR into the PC, and INST FIELD into the IF, and the DATA FIELD into the DF.
DEPOSIT	Deposits the contents of the SR into the location specified by the PC, and increments the PC by 1.
	PDP-8/I -- The DEP switch is activated when the top of the switch is depressed.
	PDP-8, 8/S, -- The DEP switch is activated when the switch is raised. and 8/L
EXAMINE	Displays the contents of the location in the PC in the MB, and increments the PC by 1.
CONTINUE	Continues program execution at the location specified by the PC.
STOP	Stops program execution.
SINGLE STEP	When set, the computer executes instructions one cycle at a time for each depression of CONT.
SINGLE INSTRUCTION	When set, the computer executes one instruction at a time for each depression of CONT.
SWITCH REGISTER (SR)	When LOAD ADD is depressed, the contents of the SR is loaded into the PC. When DEP is depressed, the contents of the SR is loaded into the MB and memory. The 12 positions represent a 12-bit binary word, usually read in octal.
DATA FIELD (DF)	Denotes the core memory field of data storage and retrieval when LOAD ADD is depressed.
INSTRUCTION FIELD (IF)	Denotes the core memory field from which instructions are taken when LOAD ADD is depressed.

INDICATORS

PROGRAM COUNTER (PC)	Contents represent the address of the next instruction to be executed.
MEMORY ADDRESS (MA)	Contents represent the address of the word currently being read or written. After depressing DEP or EXAM, the contents represent the address of the word previously read or written.
MEMORY BUFFER (MB)	Contents represent the word being read or written.
ACCUMULATOR (AC)	Indicates the contents of the AC.
LINK (L)	Indicates the contents of the Link.

MULTIPLIER
QUOTIENT
(MQ)

Activated only with the EAE option. At the start of a multiplication the contents represent the multiplier; at the end the least significant half of the product. At the start of a division the contents represent the least significant half of the dividend; at the end the quotient.

Instruction
and Status
Indicators

Located to the right of the above indicators. Indicates the type of instruction being executed, and the status of the program being executed.

ASR33 TELETYPE

The ASR33 Teletype is the basic input/output device for PDP-8/1 computers. It consists of a printer, keyboard, paper tape reader, and paper tape punch, all of which can be used either on-line under program control or off-line. The Teletype controls (Figure INTRO-2) are described as they apply to the operation of the computer. For off-line operations, see Appendix C. See PDP-8 Users Handbook (F-85) for details.

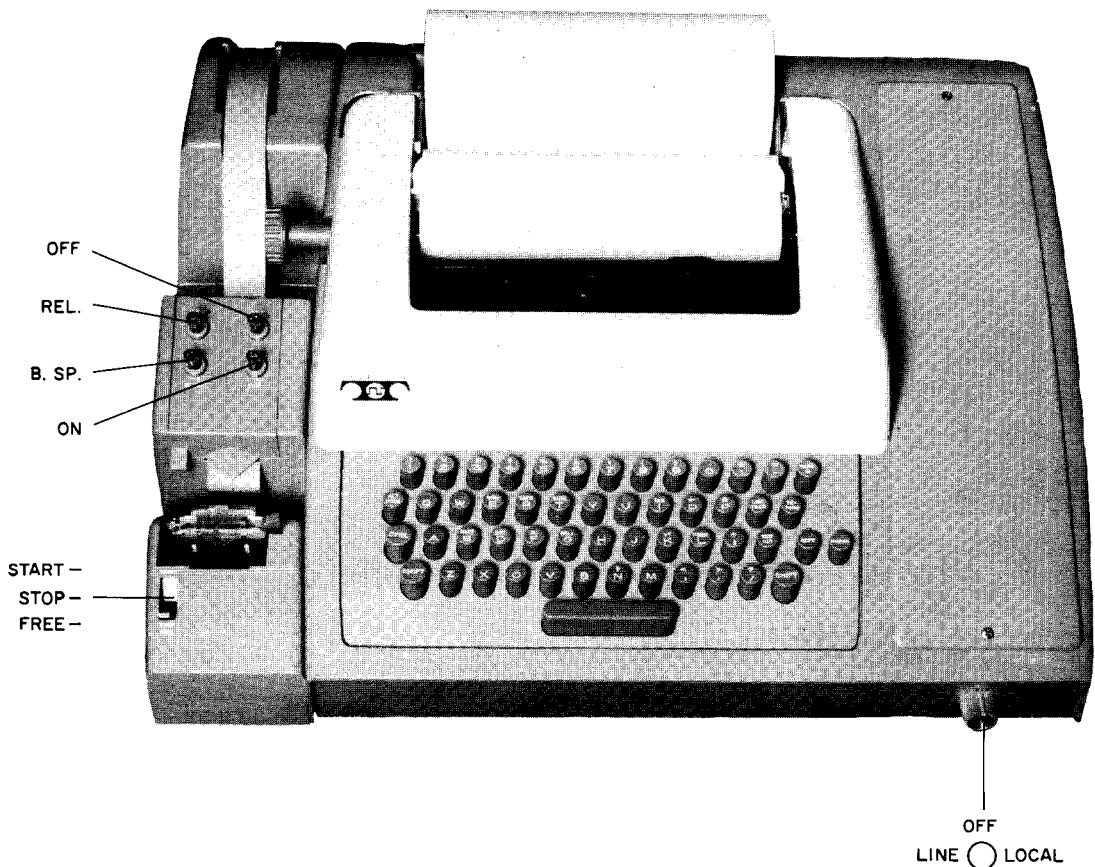


Figure INTRO-2 ASR33 Teletype Console

POWER CONTROLS

LINE	The Teletype is energized and connected to the computer as an input/output device, under computer control.
OFF	The Teletype is de-energized.
LOCAL	The Teletype is energized for off-line operation.

PRINTER

The printer provides a typed copy of input and output at 10 characters per second maximum.

KEYBOARD

The Teletype keyboard is similar to a typewriter keyboard. However, certain operational functions are shown on the upper part of some of the keytops. These functions are activated by holding down the CTRL key while depressing the desired key. For example, when using the Symbolic Editor, CTRL/FORM causes Editor to enter command mode.

Although the left and right square brackets are not visible on the keyboard keytops, they are shown in Figure INTRO-3 and are generated by typing SHIFT/K and SHIFT/M, respectively. Also the ALT MODE key is identified as ESC (ESCAPE) on some keyboards.

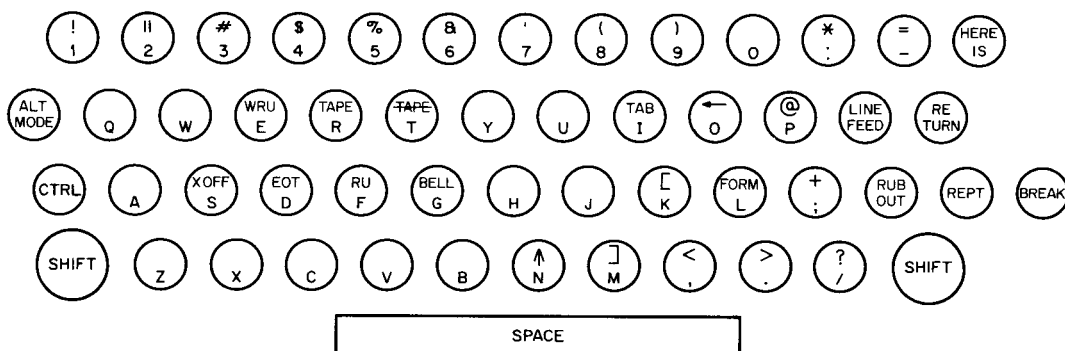


Figure INTRO-3 ASR33 Teletype Keyboard

PAPER TAPE READER

The paper tape reader is used to input into core memory data punched on eight-channel perforated paper tape at a rate of 10 characters per second maximum. The reader controls are shown in Figure INTRO-2 and described below.

START	Activates the reader; reader sprocket wheel is engaged and operative.
STOP	Deactivates the reader; reader sprocket wheel is engaged but not operative.
FREE	Deactivates the reader; reader sprocket wheel is disengaged.

Positioning Tape in Tape Reader

The following procedure describes how to properly position paper tape in the low-speed reader.

- a. Raise the tape retainer cover.
- b. Set reader control to FREE.
- c. Position the leader portion of the tape over the read pens with the sprocket (feed) holes over the sprocket (feed) wheel and with the arrow (printed or cut) pointing outward.
- d. Close the tape retainer cover.

PAPER TAPE PUNCH

The paper tape punch is used to perforate eight-channel rolled oiled paper tape at a rate of 10 characters per second. The punch controls are shown in Figure INTRO-2 and described below.

RELease	Disengages the tape to allow tape removal or loading.
Back SPace	Backspaces the tape one space for each firm depression of the B.SP. button.
ON	Activates the punch.
OFF	Deactivates the punch.

HIGH-SPEED TAPE READER AND PUNCH UNITS

A high-speed paper tape reader and punch unit is pictured in Figure INTRO-4 and descriptions of the reader and punch units follow. (See PDP-8 User's Handbook (F-85) for details.)

READER UNIT

The high-speed paper tape reader is used to input data into core memory from eight-channel fan-folded (non-oiled) perforated paper tape photoelectrically at 300 characters per second. Primary power is applied to the reader when the computer POWER switch is turned on. The reader is under user control from the keyboard through the computer or under program control. However, tape can be advanced past the photoelectric sensors without causing input by pressing the tape feed button (the white rectangular button located in the center of Figure INTRO-4).

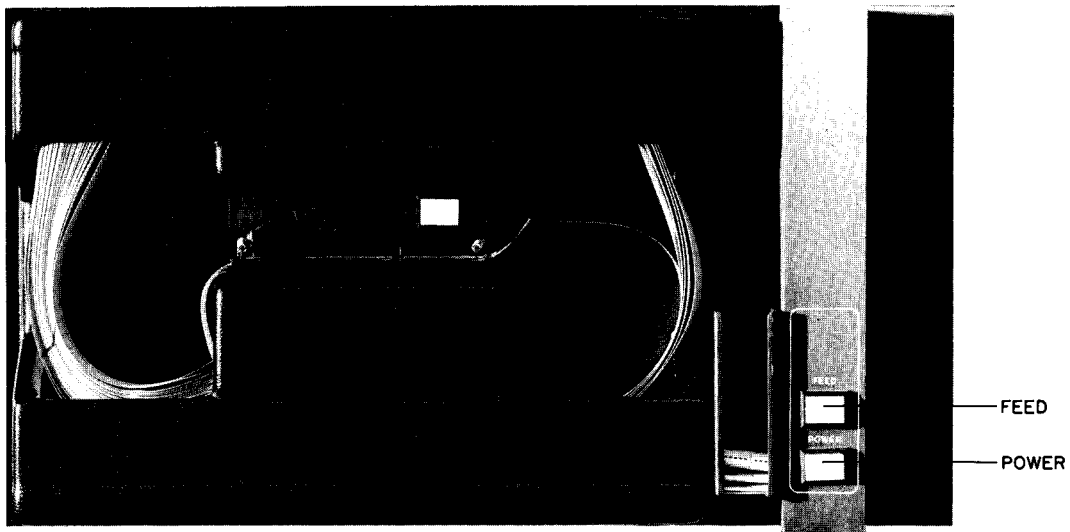


Figure INTRO-4 High-Speed Paper Tape Reader
and Punch Units

Loading Instructions

- a. Raise tape retainer cover (located beneath the tape feed button).
- b. Put tape into right-hand bin with channel one (see PAPER TAPE FORMATS) of the tape toward the rear of the bin.
- c. Place several folds of leader tape through the reader and into the left-hand bin.
- d. Place the tape over the reader head with feed holes engaged in the teeth of the sprocket wheel.
- e. Close the tape retainer cover.
- f. Depress the tape feed button (white rectangular button above the reader head) until leader tape is over the reader head.

CAUTION

Do not use oiled paper tape in the high-speed reader—oil collects dust and dirt which can cause reader errors.

PUNCH UNIT

The high-speed paper tape punch is used to record computer output on eight-channel fan-folded perforated paper tape at 50 characters per second. All characters are punched under program control from the computer. Blank tape (feed holes only, no data) may be produced

by pressing the FEED button (see Figure INTRO-4). Primary power is available to the punch when the computer POWER switch is turned on. Power is applied to the punch when the POWER button is depressed to on (the punch motor can be heard). The two labeled buttons on the punch enclosure are described below.

- | | |
|-------|---|
| POWER | This microswitch is depressed to turn the punch ON and OFF. |
| FEED | While this button is depressed, the punch produces feed-hole-only punched tape for leader/trailer purposes. |

CAUTION

Do not use oiled paper tape in the high-speed punch -- oil collects dust and dirt which can cause reader errors.

DECTAPE CONTROL AND TRANSPORT UNITS

DECTape is a fast, convenient, reliable input/output data storage facility and updating device. The standard DECTape transport unit is pictured in Figure INTRO-5 and descriptions of the control and transport units follow. (See PDP-8 User's Handbook (F-85) for details.)

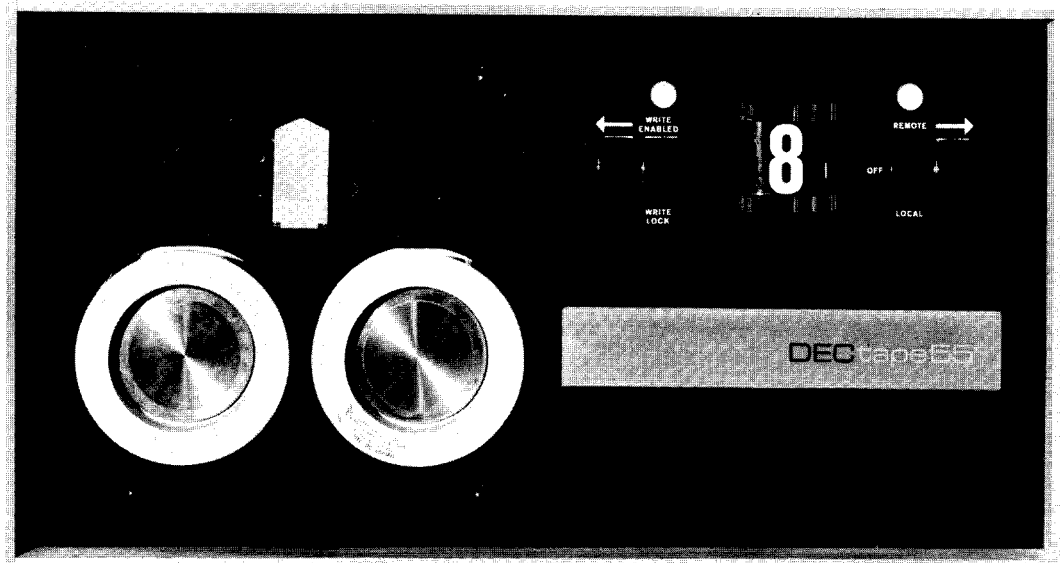


Figure INTRO-5 DECTape Transport Unit

CONTROL UNIT

The DECTape control unit interprets and controls the transfer of data between the computer and the transport unit. The DECTape control unit is usually located inside the rack containing the DECTape transport and can control up to eight separate DECTape transports.

TRANSPORT UNIT

The DECTape transport unit is a bidirectional magnetic tape transport utilizing a 10-track recording head to read and write five duplexed channels. Tape movement can be controlled by commands from the computer program or by commands from the manual operation of switches located on the front panel of the transport; however, manual operation does not transfer data to the computer.

Only Certified DECTapes (prerecorded with timing and marking tracks) should be used. Otherwise, the blank tape must be certified using the DECTOG program (DEC-08-EUFA-D).

Transport Controls

→	When depressed (must be in LOCAL mode), tape feeds onto right spool.
REMOTE	Transport is energized and under program control.
OFF	Transport is de-energized.
LOCAL	Transport is energized and under user control from external transport switches.
Unit Selector	Identifies the transport to the control unit.
WRITE ENABLED	DECTape is available for search, read, and write activities.
WRITE LOCK	DECTape is available for search and read activities only.
←	When depressed (must be in LOCAL mode), tape feeds onto left spool.

The REMOTE and WRITE ENABLED lamps light to indicate the status of the transport.

Operating Procedure

- a. Set switch to OFF.
- b. Place DECTape on left spindle with DECTape label out.
- c. Wind four turns of tape on right spool.
- d. Set switch to LOCAL.
- e. Wind a few turns on right spindle with -- switch to make sure tape is properly mounted.
- f. Dial correct unit number on unit selector (number 8 is equivalent to 0).

- g. Depress REMOTE switch.
- h. Depress WRITE ENABLED or WRITE LOCK switch.
- i. DECtape transport is now under program control.

INITIALIZING THE SYSTEM

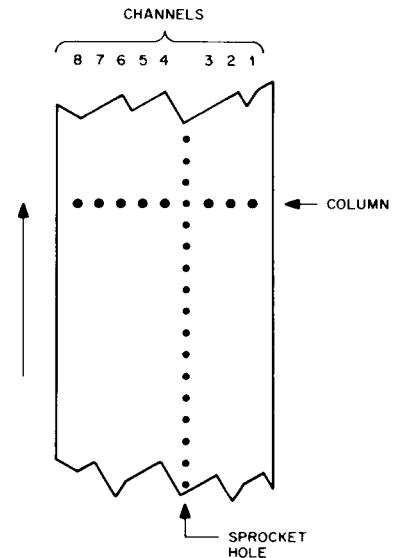
Before using the computer system, it is good practice to initialize all units. To initialize the system, ensure that all switches and controls are as specified below.

- a. Main power cord is properly plugged in.
- b. Teletype is turned OFF.
- c. Low-speed punch is OFF.
- d. Low-speed reader is set to FREE.
- e. Computer POWER key is ON.
- f. PANEL LOCK is unlocked.
- g. Console switches are set to
 DF=000 IF=000 SR=0000
 SING STEP and SING INST are not set
- h. High-speed punch is OFF.
- i. DECtape REMOTE lamps OFF.

The system is now initialized and ready for your use.

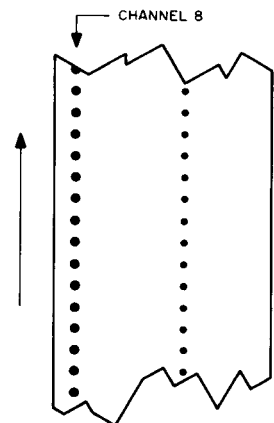
PAPER TAPE FORMATS

Data are recorded (punched) on paper tape by groups of holes arranged in a definite format along the length of the tape. The tape is divided into channels which run the length of the tape, and into columns which extend across the width of the tape as shown in the adjacent diagram. The paper tape readers and punches used with the PDP-8/I computers accept 8-channel paper tape. The various formats are briefly explained and identified below.



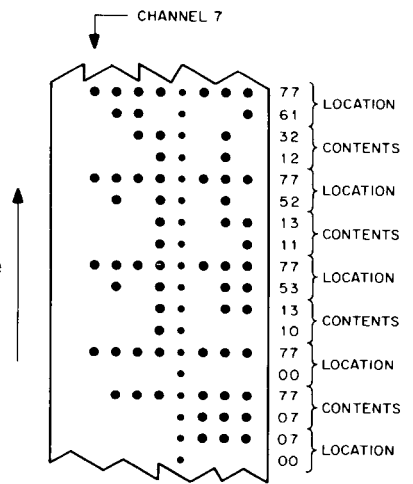
Leader/Trailer Format

Leader/trailer tape is used to introduce and conclude the object program when punched on paper tape. Leader/trailer tape can be recognized by a consistent channel 8 punch only as shown in the adjacent diagram.



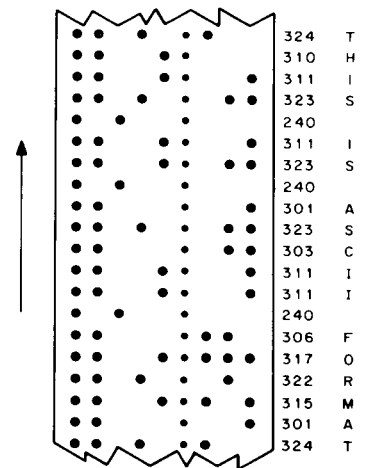
RIM Format

Paper tape punched in RIM format can be identified by the absence of a channel 8 punch, and by a channel 7 punch in every fourth column. The channel 7 punch indicates the start of a line of coding, and that (the first) column and the second column contain the location and the third and fourth columns contain the contents of the location.



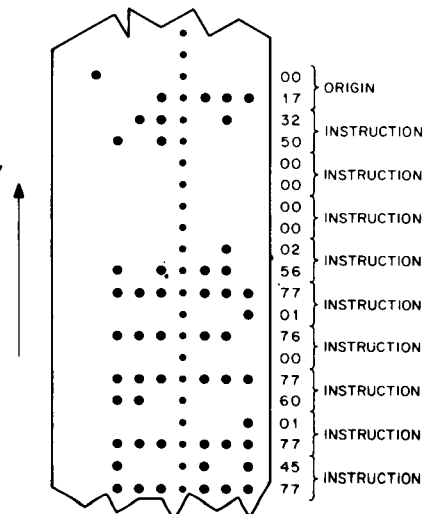
ASCII Format

ASCII (USA Standard Code for Information Interchange) format uses all eight channels to represent a single character (letter, number, or symbol) as shown in the adjacent diagram.



Binary Format

Binary format can be recognized by the absence of a channel 8 punch, an occasional channel 7 punch, and frequent sections of blank tape. The channel 7 punch denotes an origin of a program or subprogram or a change in origin, and subsequent columns contain the instructions (two columns per instruction) or data of succeeding locations.



ABBREVIATIONS

The abbreviations listed below are used throughout the guide.

<u>Abbreviations</u>	<u>Meaning</u>
AC	Accumulator
ADDR	Address
B. SP.	Back Space
BIN	Binary
CLC	Current Location Counter
CONT	Continue
CR	Carriage Return
CR/LF	Carriage Return-Line Feed
CTRL/L	Control/L (represents holding down the CTRL key while depressing the L key or the key following the slash)
DEC	Digital Equipment Corporation
DEP	Deposit
DF	Data Field
EAE	Extended Arithmetic Element
EXAM	Examine
IF	Instruction Field
INST	Instruction
L	Link
LF	Line Feed
LOAD ADD	Load Address
LOC	Location
LSP	Low-Speed Punch
LSR	Low-Speed Reader
HSP	High-Speed Punch
HSR	High-Speed Reader
KBRD	Keyboard
PC	Program Counter
PROG	Program
MA	Memory Address
MB	Memory Buffer
MQ	Multiplier Quotient
REL	Release
RIM	Read-In Mode
SA	Starting Address
SHIFT/P	Shift/P (similar to CTRL/L)
SING INST	Single Instruction
SING STEP	Single Step
SR	Switch Register
SW	Console Switches
TTY	Teletype

SECTION 2
SYSTEM PROGRAMS

PURPOSE

The RIM Loader is used to load into core memory programs punched on paper tape in RIM format, e.g., the Binary Loader. (See DEC-08-LRAA-D for details.)

STORAGE REQUIREMENTS

RIM requires locations 7756-7776 (21_8 locations). Starting Address=7756.

LOADING

RIM is loaded (toggled) into core memory using the console switches. RIM can use either the low- or high-speed readers when loading RIM coded program tapes into core. The locations and corresponding instructions for both input devices are listed below.

Location	Instruction	
	Low-Speed Reader	High-Speed Reader
7756	6032	6014
7757	6031	6011
7760	5357	5357
7761	6036	6016
7762	7106	7106
7763	7006	7006
7764	7510	7510
7765	5357	5374
7766	7006	7006
7767	6031	6011
7770	5367	5367
7771	6034	6016
7772	7420	7420
7773	3776	3776
7774	3376	3376
7775	5356	5357
7776	0000	0000

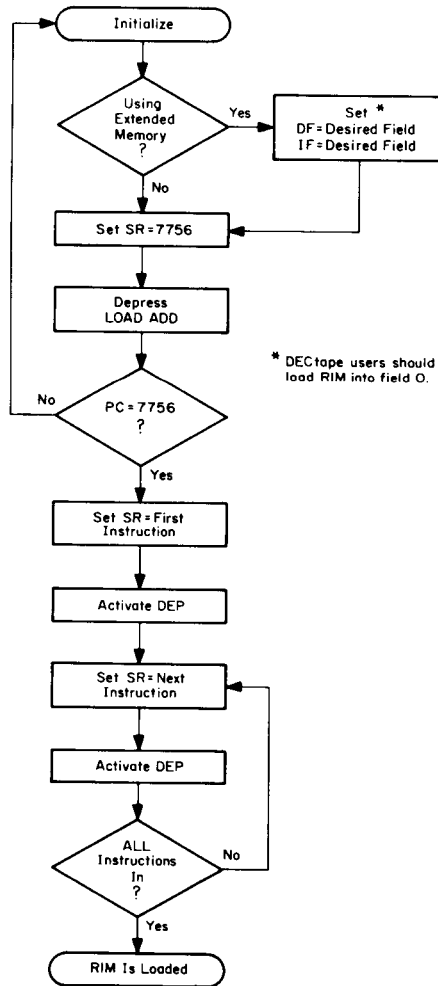


Figure RIM-1 Loading the RIM Loader

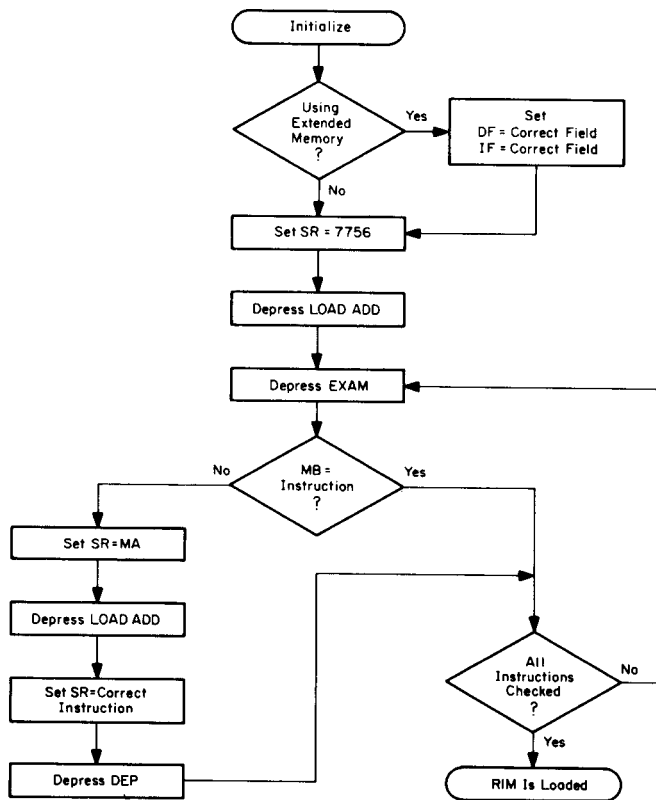


Figure RIM-2 Checking the RIM Loader

PURPOSE

The BIN Loader is used to load into core memory binary coded programs punched on paper tape. When in core, BIN can be destroyed only by the user's program because DEC's programs (excluding Disk/DECtape Monitor) do not use the last page of core (location 7600-7777). (See DEC-08-LBAA-D for details.)

STORAGE REQUIREMENTS

BIN occupies locations 7625-7752 and 7777 (123_8 locations). Starting Address=7777

LOADING

RIM is used to load BIN into core. BIN must be loaded into the same field as RIM, and the input device (low- or high-speed reader) must be that which was selected when loading RIM.

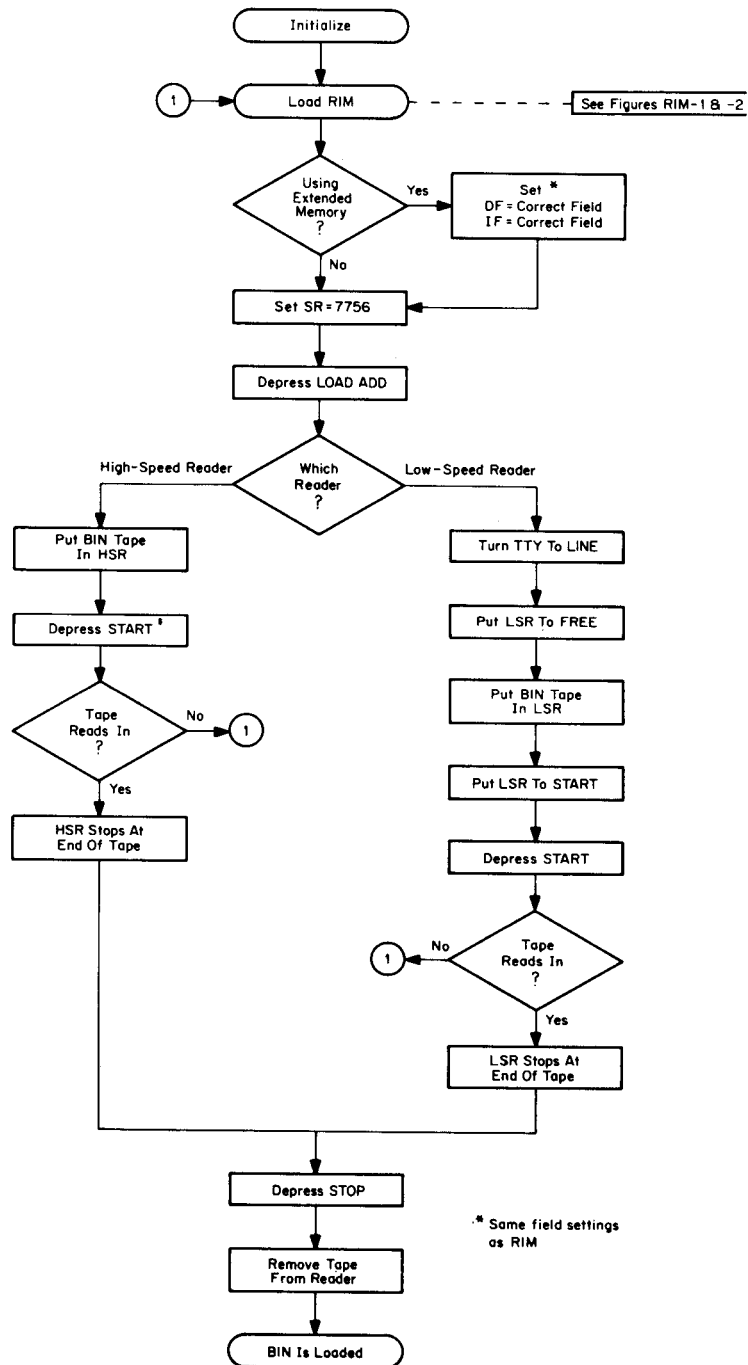


Figure BIN-1 Loading the BIN Loader

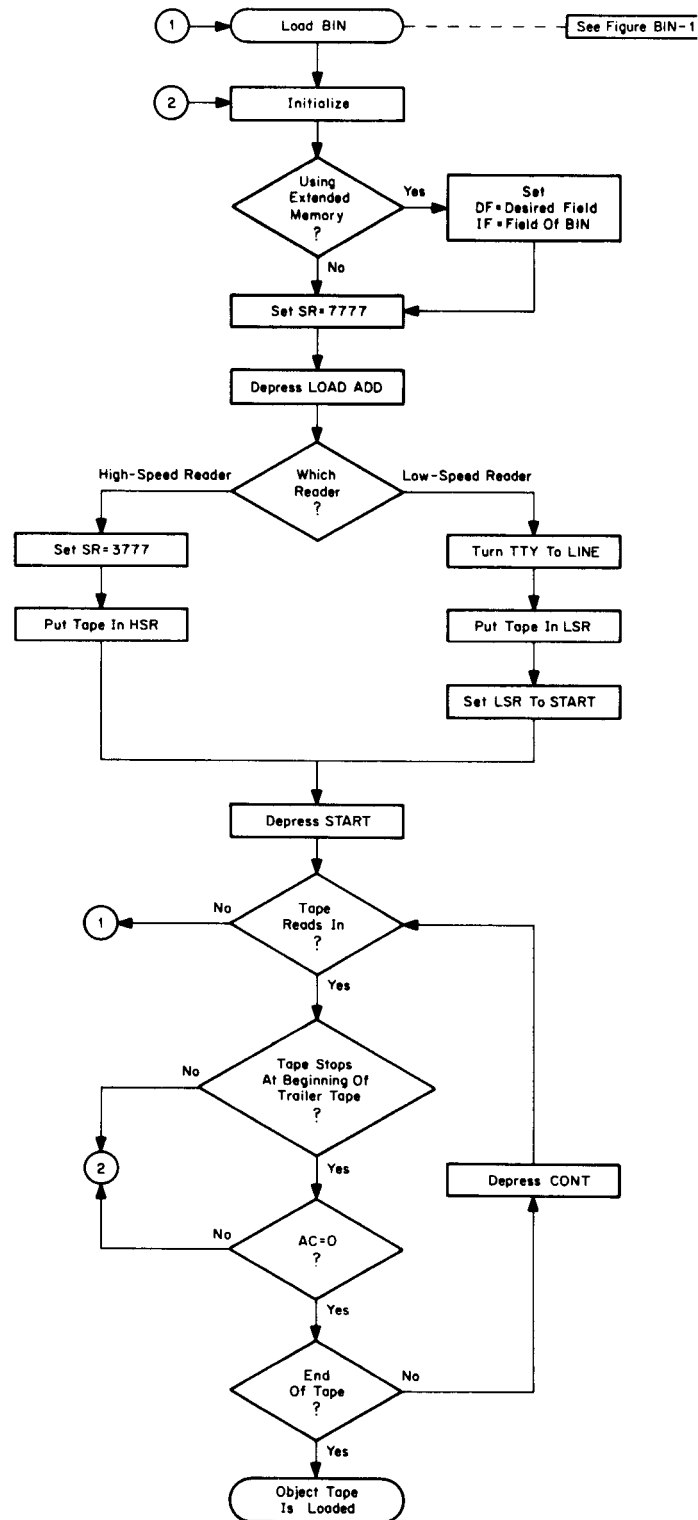


Figure BIN-2 Loading A Binary Coded Object Tape Using BIN

PURPOSE

The HELP Loader is used to quickly load into core memory the RIM and BIN Loader programs. (See DEC-08-LHAA-D for details.)

STORAGE
REQUIREMENTS

HELP uses locations 0005-0036 (32_8 locations) to load the HELP tape into core. The HELP tape contains the RIM and BIN Loaders.

LOADING

HELP is in two parts: The first part consists of the 11_8 instructions shown below, which are toggled into core using the console switches. The second part is the HELP Bootstrap Loader punched on paper tape, which is loaded into core using the low-speed reader.

<u>Location</u>	<u>Instruction</u>
0027	6031
0030	5027
0031	6036
0032	7450
0033	5027
0034	7012
0035	7010
0036	3007
0037	2036
0040	5027

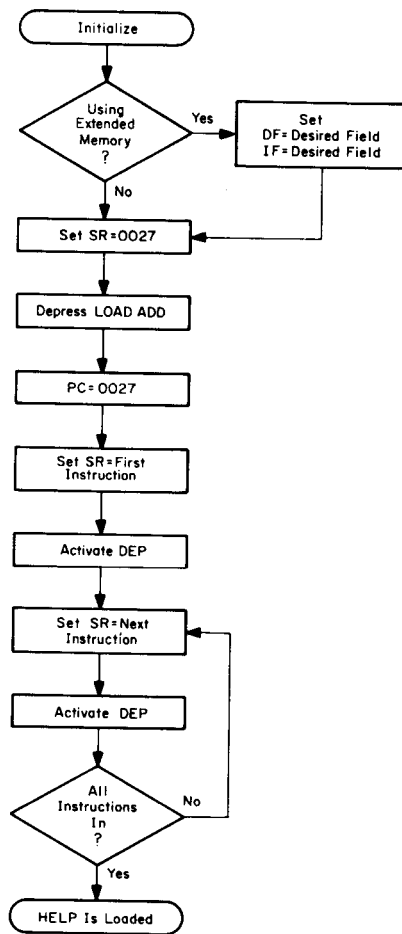


Figure HELP-1 Loading the HELP Loader

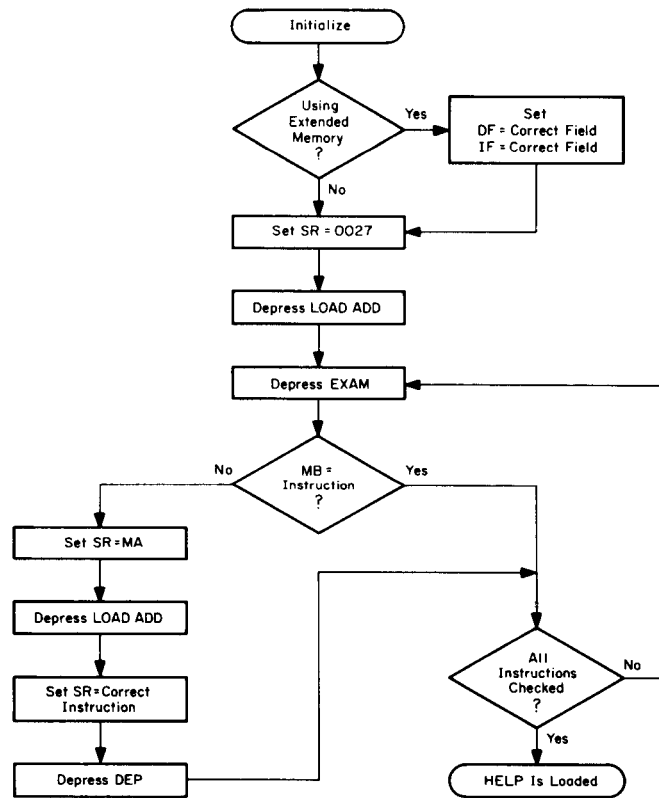


Figure HELP-2 Checking the HELP Loader

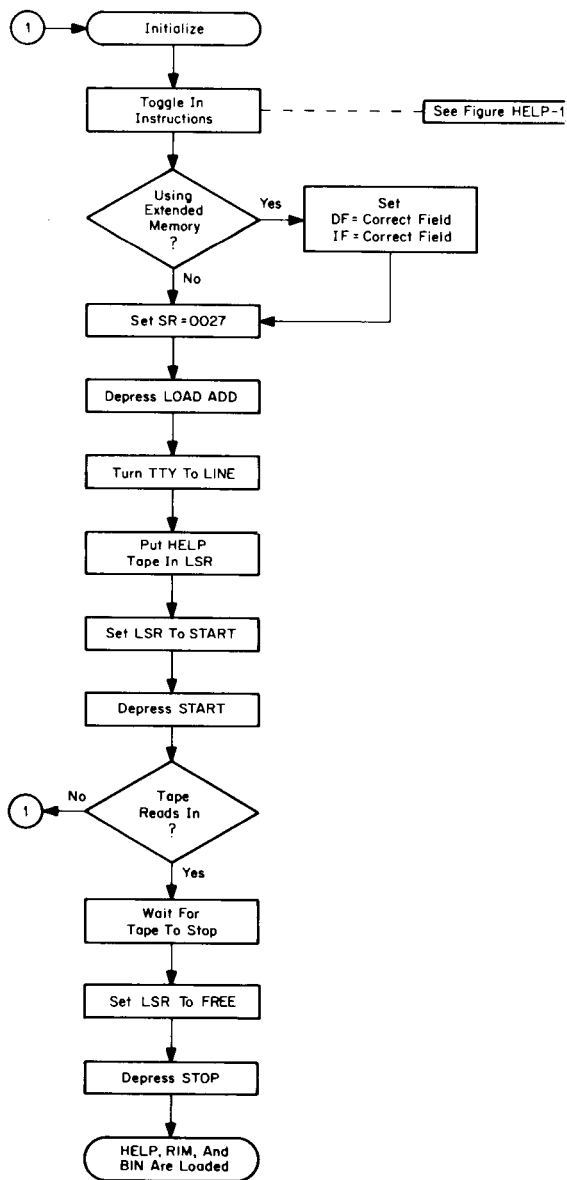


Figure HELP-3 Loading the HELP Bootstrap Tape Into Core

PURPOSE

The Symbolic Tape Editor is used to prepare, edit, and generate symbolic program tapes on line. (See DEC-08-ESAB-D for details.)

STORAGE
REQUIREMENTS

Editor requires locations 0-1577 (1600₈ locations). Starting Address=0200.

LOADING

BIN is used to load Editor into core memory. The loading of the user's symbolic tapes is performed by Editor itself under keyboard control.

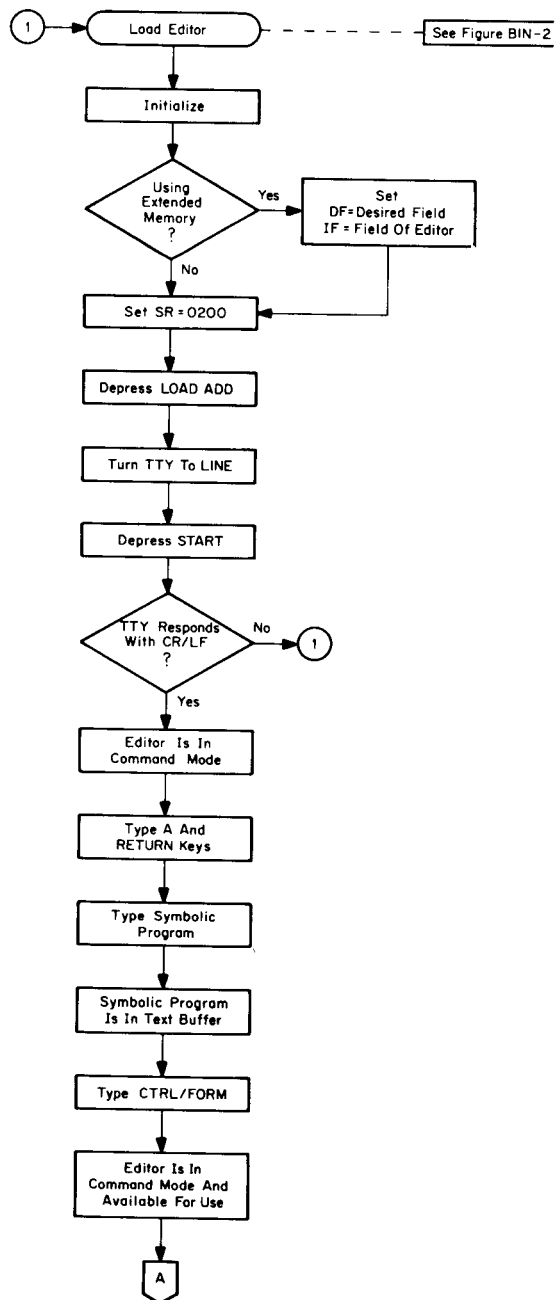


Figure EDIT-1 Generating a Symbolic Program On-Line Using Editor

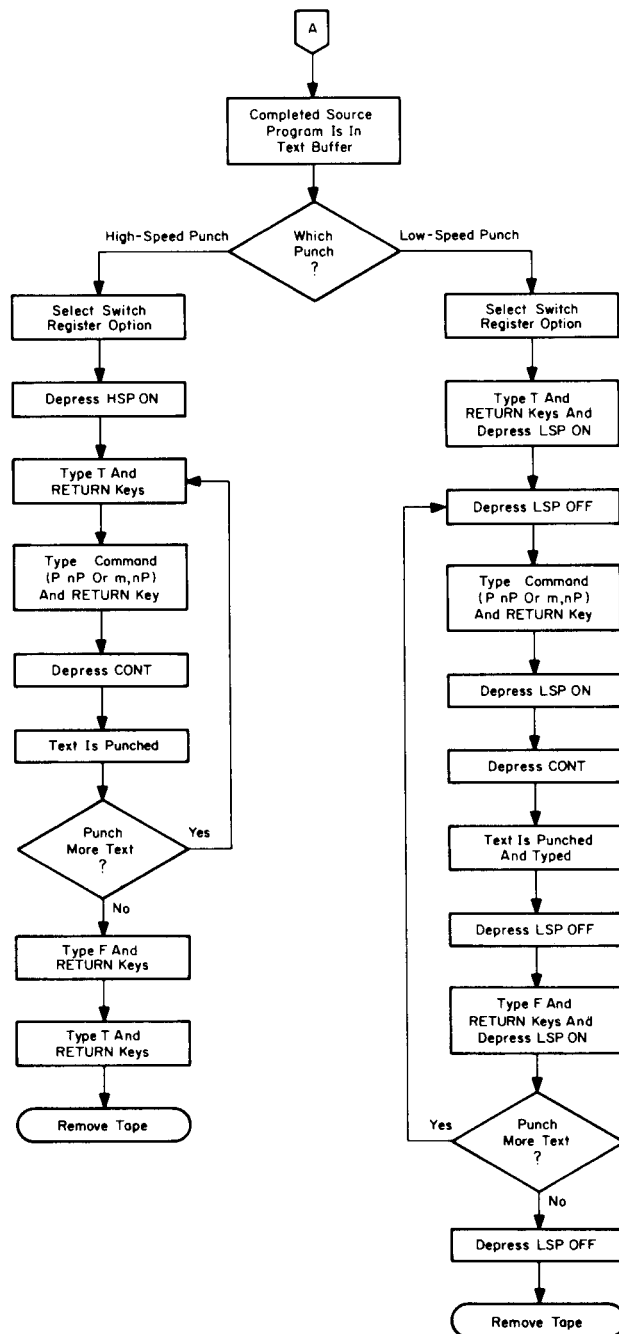


Figure EDIT-2 Generating a Symbolic Tape Using Editor

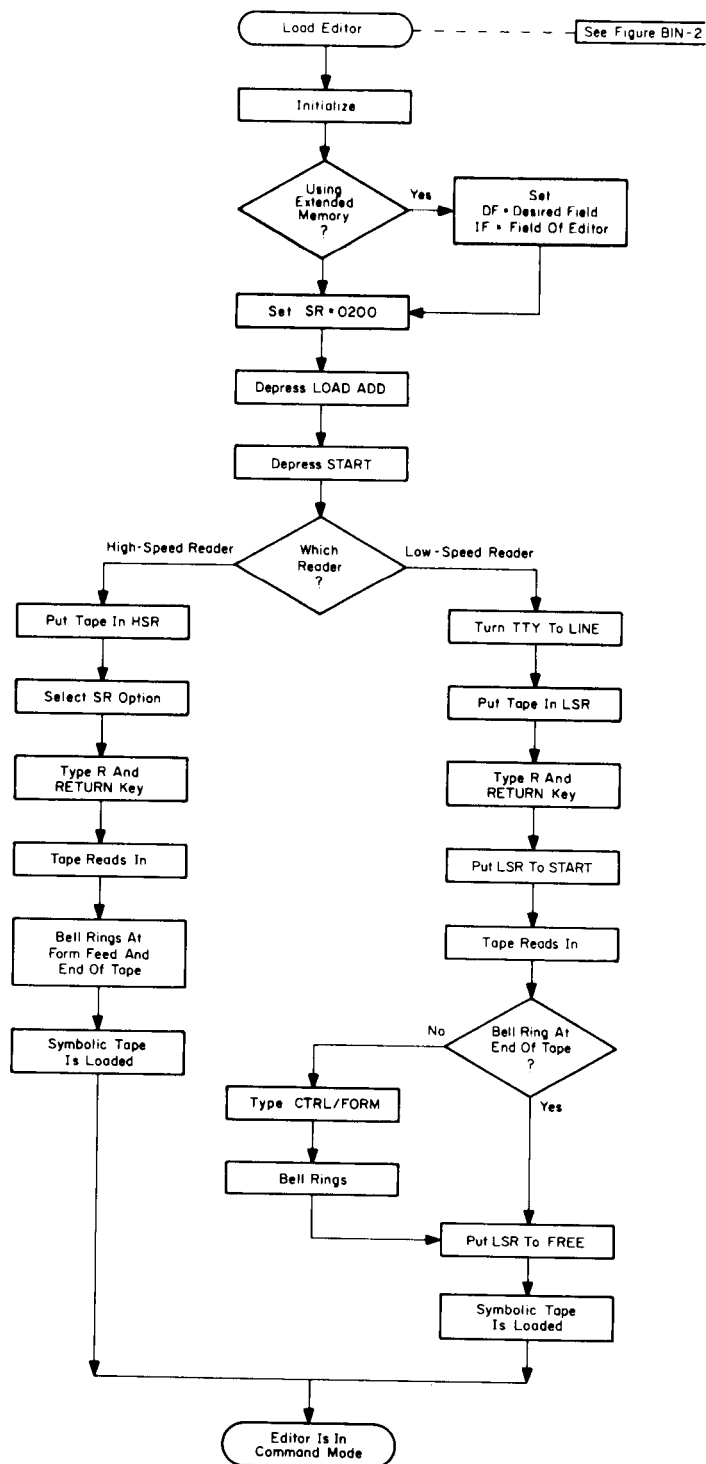


Figure EDIT-3 Loading a Symbolic Tape Using Editor

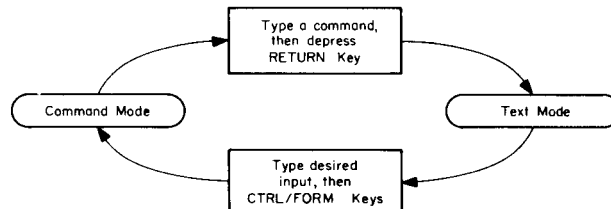
OPERATING MODES

Editor is always in one of the following modes.

Command Mode: All characters typed on the teleprinter are interpreted as commands to the Editor.

Text Mode: All characters typed or tapes being read in are interpreted as text to be put into the text buffer in the manner specified by the previous command and the SR options.

Transition between modes:



RESTART PROCEDURES

If the user stops the computer for any reason, he may restart it at location 0177 without disturbing the text in the buffer. A CR/LF will follow.

If no CR/LF is desired, restart at location 0200.

CAUTION

If Editor is restarted at location 0176, all text currently in the buffer is wiped out, and the text buffer is re-initialized for a new program.

SPECIAL KEY FUNCTIONS

<u>Key</u>	<u>Command Mode</u>	<u>Text Mode</u>
RETURN	Execute preceding command	Enter line in text buffer
←	Cancel preceding command (Editor responds with ? CR/LF)	Cancel line to the left margin
RUBOUT	Same as ←	Delete to the left one character for each depression; a \ (back-slash) is echoed (not used in READ command)
CTRL/FORM		Return to command mode (bell rings)

<u>Key</u>	<u>Command Mode</u>	<u>Text Mode</u>
	Used as argument alone or with + or - and a number (.=, .+5L, .-2\$) (a space is equivalent to a +)	Legal text character.
/	Value equal to number of last line in buffer; used as argument (/ -5G, /L)	Legal text character
LINE FEED	List next line	Used in SEARCH command to insert CR/LF into line
ALT MODE (ESC)	List next line	
>	List next line	
<	List previous line	
=	Used with . and / to obtain their value (.=27)	
:	Same as = (gives value of any legitimate argument)	
CTRL/TAB		Produces a tab which on output is interpreted as ten spaces or a tab/rubout, depending on SR option.

SEARCH FEATURE IN COMMAND MODE

Following a nS command, Editor waits for the user to specify the search character which when typed is not echoed. When Editor locates and types the search character, typing stops and all or any combination of the following operations may be carried out.

- a. Type new text and terminate line with the RETURN key
- b. ← delete entire line to the left
- c. RETURN delete entire line to the right
- d. RUBOUT delete from right to left one character for RUBOUT typed (a \ is echoed for each RUBOUT typed)
- e. LINE FEED insert a CR/LF, thus dividing line into two
- f. CTRL/FORM search for next occurrence of search character
- g. CTRL/BELL change search character to next character typed by the user

SWITCH REGISTER OPTIONS

Switch Register options are used with input and output commands to control the reading and punching of paper tape.

<u>SR Bit</u>	<u>Position</u>	<u>Function</u>
0	0	Input text as is
	1	Convert all occurrences of 2 or more spaces to a tab
1	0	Output each tab as 10 spaces
	1	Tab is punched as tab/rubout
2	0	Output as specified
	1	Suppress output*
10	0	Low-speed punch
	1	High-speed punch
11	0	Low-speed reader
	1	High-speed reader

COMMANDS

Input

R	Read incoming text from tape reader into core
A	Append incoming text from keyboard into core

→ for odd
to not -

Editing

L	List entire text buffer
nL	List line n
m,nL	List lines m through n inclusively
nC	Change line n
m,nC	Change lines m through n inclusively
I	Insert before first line
nI	Insert before line n
K	Delete entire text buffer
nD	Delete line n
m,nD	Delete lines m through n inclusively
m,n\$ kM	Move lines m through n to before line k
G	Print next tagged line (if none, Editor types ?)
nG	Print next tagged line after line n (if none, ?)
S	Search buffer for character specified after RETURN key and allow modification (search character is not echoed on printer)
nS	Search line n, as above
m,nS	Search lines m through n inclusively, as above

Output

P	Punch entire text buffer
nP	Punch line n
m,nP	Punch lines m through n inclusively

*Bit 2 allows the user to interrupt any output command and return immediately to command mode.

F	Punch leader tape, a Form Feed, and trailer tape
T	Punch about 6 inches of leader/trailer tape
nN	Do P, F, K, and R commands n times

where m and n are decimal integers, and m is smaller than n. P and N halt to allow user to select SR option. Press CONT to execute command.

Commands are executed upon depressing the RETURN key.

BUFFER OVERFLOW

Editor has storage for about 5000_{10} characters (approximately 60 heavily commented lines or 340 uncommented lines). When the text buffer is exceeded, operation continues, but a bell rings for every location used beyond the buffer limit. The user may expand the text buffer by changing location 0001 to contain the address of the last location (should not be greater than location 7570) used prior to buffer overflow. Very large programs should be divided into sections.

DIAGNOSTICS

- a. Editor checks commands for nonexistent information and incorrect formatting, and when an error is detected Editor types a ? and ignores the command. However, if an argument is provided for a command that doesn't require one, the argument is ignored and the command is executed properly.
- b. Corrections and additions to the user's program may be either typed in from the teleprinter keyboard or read in from the paper tape reader.
- c. Since Editor does not recognize extraneous and illegal control characters, a tape containing these characters can be corrected by merely reading the tape into Editor and punching out a new tape.

PURPOSE

The PAL III Symbolic Assembler is used to translate symbolic (source) programs into binary (object) programs. PAL III is a two-pass assembler with an optional third pass which produces a program assembly listing.

Pass 1: Assembler reads the source tape and defines all symbols used. The user's symbol table and any error diagnostics are typed out.

Pass 2: Assembler reads the source tape and generates the object tape using the symbols defined during Pass 1. Ignore meaningless characters typed when using the low-speed punch, but note any error diagnostic typed.

Pass 3: Assembler reads the source tape and types and/or punches the program assembly listing.

(See DEC-08-ASAC-D for details.)

STORAGE
REQUIREMENTS

PAL III requires locations 0-2735 (2736₈ locations)

Symbol Table Capacity: LSR allows 590 user symbols
HSR allows 495 user symbols

Starting Address=0200

LOADING

BIN is used to load PAL III into core. PAL III is used to read in the symbolic tapes during assembly.

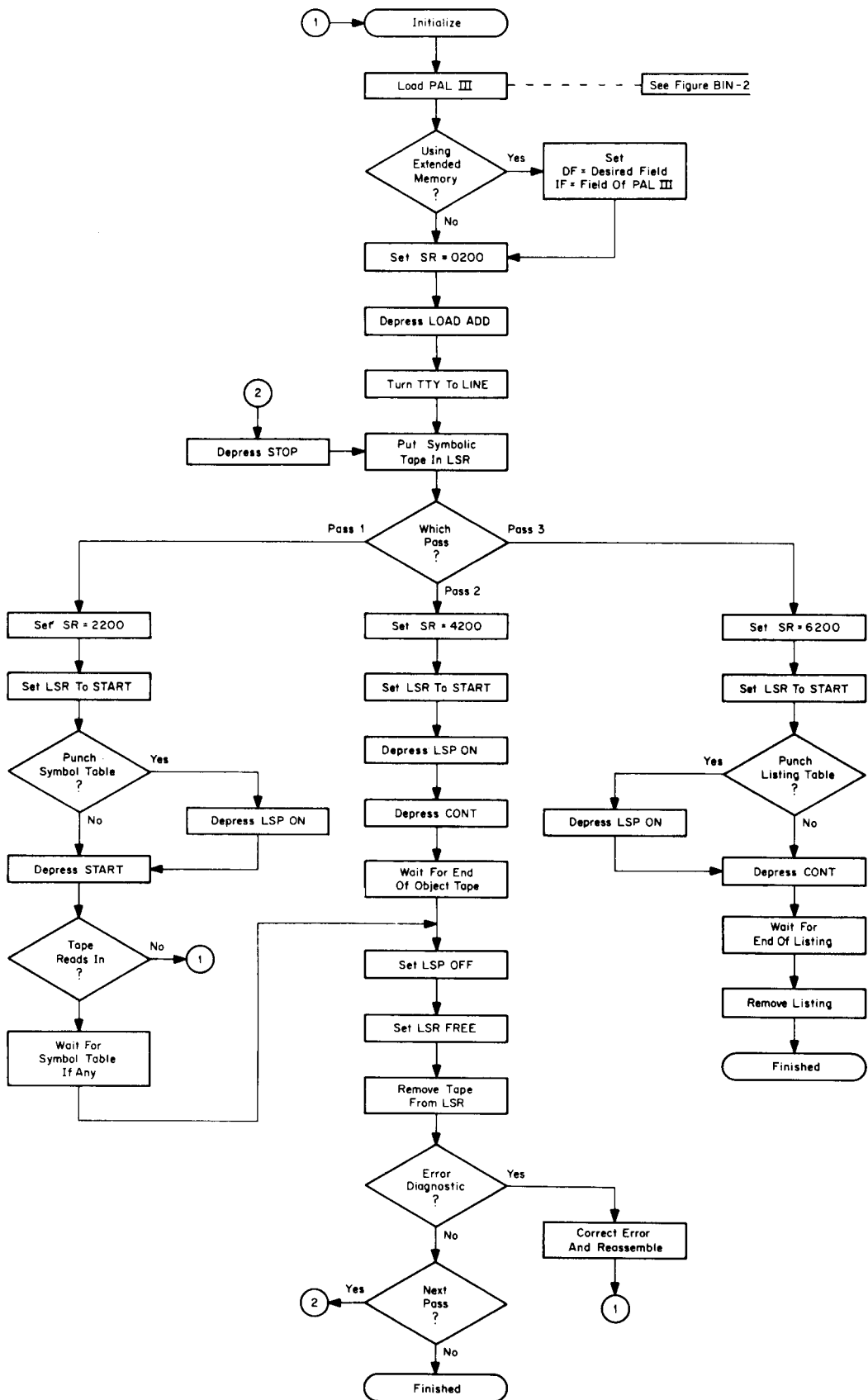


Figure PAL-1 Assembling with PAL III Using Low-Speed Reader/Punch

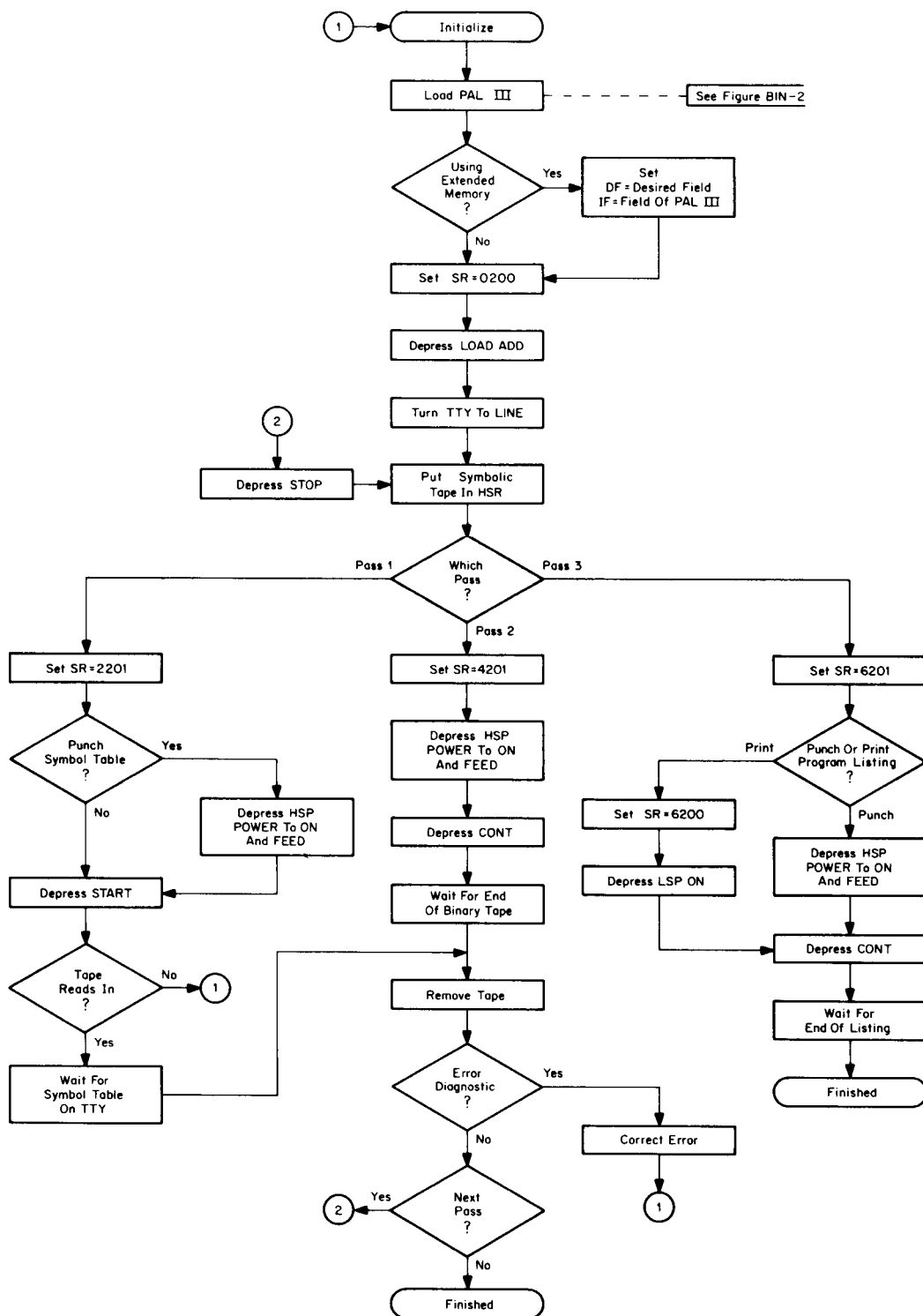


Figure PAL-2 Assembling With PAL III Using High-Speed Reader/Punch

OUTPUT CONTROL

Output is controlled by the setting of switch register bit 11 as shown below.

Pass 1: Bit 11=0 Type and punch symbol table on TTY
 11=1 Punch symbol table on HSP

Pass 2: No effect; binary tape will be punched on the HSP if it
 is turned on.

Pass 3: Bit 11=0 Type and punch program listing on TTY
 11=1 Punch program listing on HSP

DIAGNOSTICS

Format: xx yyyyyy AT nnnn

where xx is the error message (see below), yyyyyy is the symbol or octal
value of the symbol of the error occurring AT location nnnn.

Pass 1:	IC	Illegal Character
	RD	ReDefinition
	DT	Duplicate Tag
	ST	Symbol Table full
	UA	Undefined Address

Pass 2:	IR	Illegal Reference
---------	----	-------------------

PURPOSE

The MACRO-8 Symbolic Assembler is used to translate symbolic (source) programs into binary (object) programs. MACRO-8 is a two-pass assembler with an optional third pass which produces a program assembly listing.

Pass 1: Assembler reads the source tape and defines all symbols and macros used and places them in respective tables.

Pass 2: Assembler reads the source tape and generates the object tape using symbols and macros defined during Pass 1. The Assembler then types and/or punches the user's symbol table, for use with DDT-8, followed by any error diagnostic.

Pass 3: Assembler reads the source tape and types and/or punches the program assembly listing.

See DEC-08-CMAA-D for details.

STORAGE
REQUIREMENTS

MACRO-8 requires locations 0-7577 (7600g locations).

Symbol Table Capacity: 227_g symbols (expandable to 524_g symbols using the switch options)

Starting Address=0200.

LOADING

BIN is used to load MACRO-8 into core. There are two versions of MACRO-8:

Low version: Uses the low-speed reader for all input and the low-speed punch for all output.

High version: Uses the high-speed reader for all input, the high-speed punch for binary output, and the Teletype and low-speed punch for output of error diagnostics, symbol table, and third-pass assembly listing.

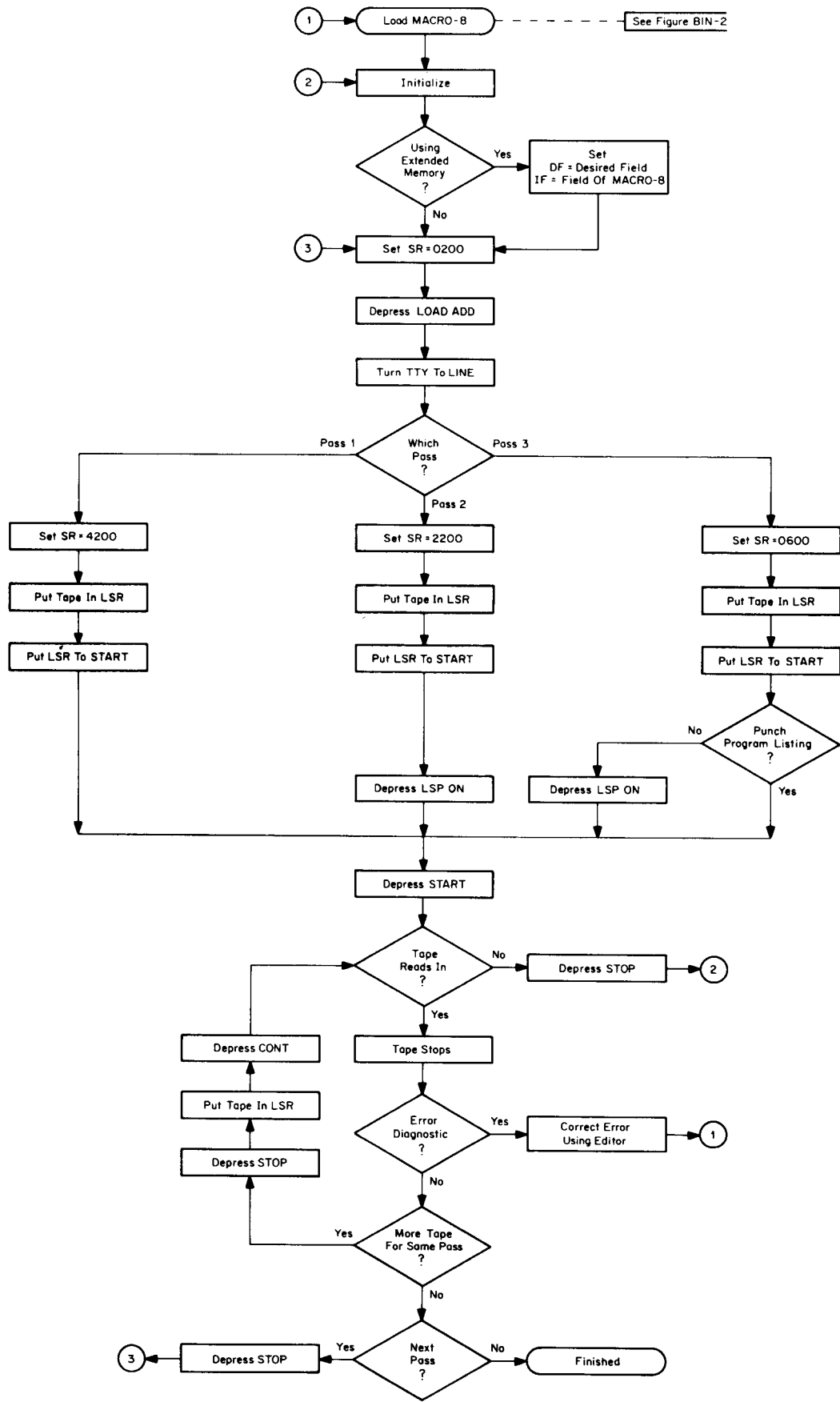


Figure MACRO-1 Assembling a MACRO-8
Source Program Using the Low-Speed Reader/Punch

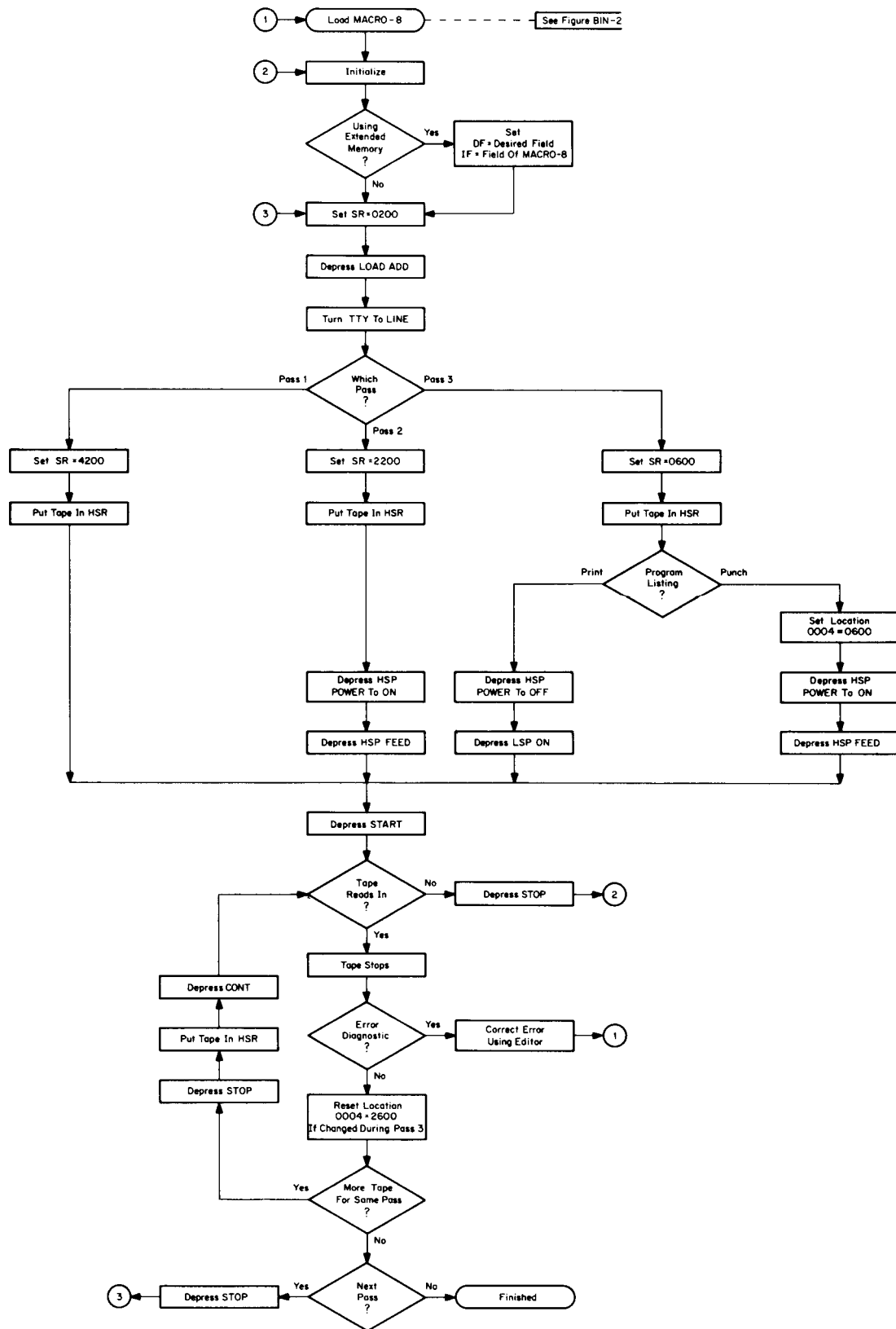


Figure MACRO-2 Assembling a MACRO-8 Source Program
Using the High-Speed Reader/Punch

SYMBOL TABLE MODIFICATION

There are 1134_g locations available for the user's symbols and the macro table. There are three ways to increase the size of this storage area.

- a. Prior to Pass 1, set bit 10 = 1 to add 100_g locations; the double precision integer and floating-point processors are deleted.
- b. Prior to Pass 1, set bit 11 = 1 to add 175_g locations; the macro and number processors are deleted.
- c. Use the pseudo-ops EXPUNGE and FIXTAB to remove unnecessary instruction mnemonics.

SWITCH REGISTER OPTIONS

<u>Bit</u>	<u>Result</u>
0-11 = 0	Enter next pass.
0 = 1	Erase symbol table excluding permanent symbols and enter Pass 1; depress STOP then CONT.
1 = 1	Enter pass 2 to generate another binary tape.
2 = 1	Enter pass 1 without erasing defined symbols.
3 = 1	Enter pass 3.
10 = 1	Delete double precision integer and double precision floating-point processors; this increases the symbol table size by 100 _g symbols.
11 = 1	Delete macro and number processors; this increases the symbol table size by 175 _g symbols.

Bits 10 and 11 are sensed whenever pass 1 is entered. Therefore, MACRO-8 would have to be reloaded to handle subsequent programs that use macros, double precision integers, or floating-point numbers.

In the high version, the high-speed punch may be used as the output device by changing the contents of location 0004 from 2600 to 0600. This is useful for long third pass listings, since the punched output from the high-speed punch can be subsequently listed off line. It is advised that this change not be made until pass 3, so that pass 1 and 2 error diagnostics will be printed.

DIAGNOSTICS

Format: ERROR CODE ADDRESS

where ERROR CODE is a two-letter code listed below, and ADDRESS is either the absolute address of the error or the address of the error relative to the last symbolic tag on that page.

<u>Error Code</u>	<u>Explanation</u>
BE	MACRO-8 internal tables have overlapped
IC	Illegal character
ID	Illegal redefinition of a symbol

<u>Error Code</u>	<u>Explanation</u>
IE	Illegal equal sign
II	Illegal indirect address
IM	Illegal format in a macro definition
LG	Link generated to off-page address*
MP	Missing parameter in macro call
PE	Current, nonzero page exceeded
SE	Symbol table exceeded
US	Undefined symbol
ZE	Page zero exceeded

*This is to inform the user of off-page references which may not be an error. This diagnostic can be suppressed to speed up pass 2 assembly by setting location 1234=7200.

PURPOSE

The Dynamic Debugging Technique for the PDP-8 computers facilitates program debugging by allowing the user to examine core memory locations (registers) and change and correct their contents, place and remove strategic halts and automatically restore and execute the instructions replaced by the halts, and much more. Communication is via the Teletype keyboard using defined commands and the symbolic language of the source program or octal representation, with DDT-8 performing all translation to and from the binary representation. (See DEC-08-CDDA-D for details.)

STORAGE
REQUIREMENTS

DDT-8 requires locations 0004 and 5237-7577 (2341₈ locations)
Permanent Symbol Table requires locations 5237-5000
External Symbol Table is allotted locations 5000-3030 (250 symbol capacity)
Starting Address = 5400

LOADING

BIN is used to load DDT-8 and the object program into core.

The user should have at the console the Pass 3 listing of his object program so that the listing can be updated to reflect any debugging change made to the program.

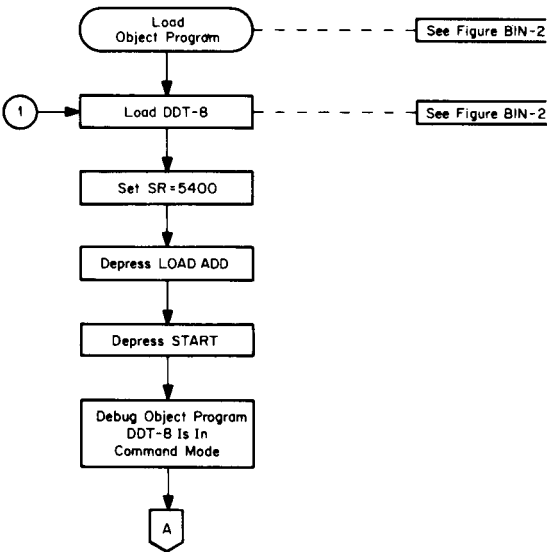


Figure DDT-1 Loading and Executing DDT-8

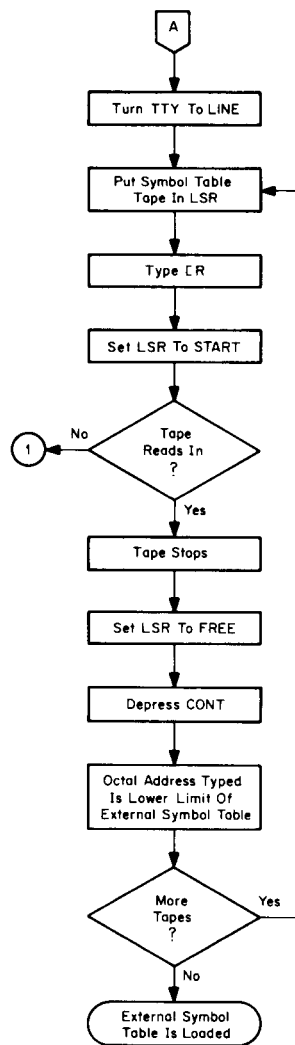


Figure DDT-2 Loading External Symbol Table Tapes
(LSR Only)

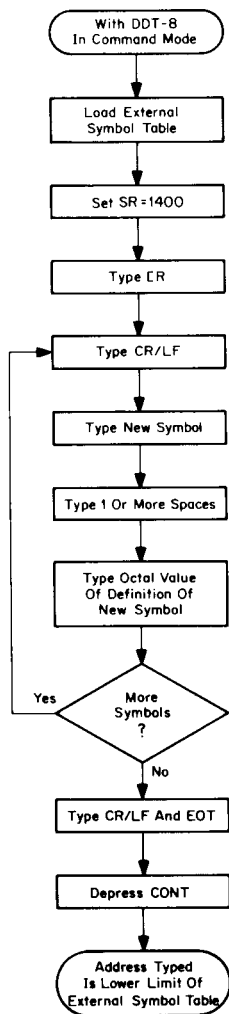


Figure DDT-3 Appending New Symbols to External Symbol Table

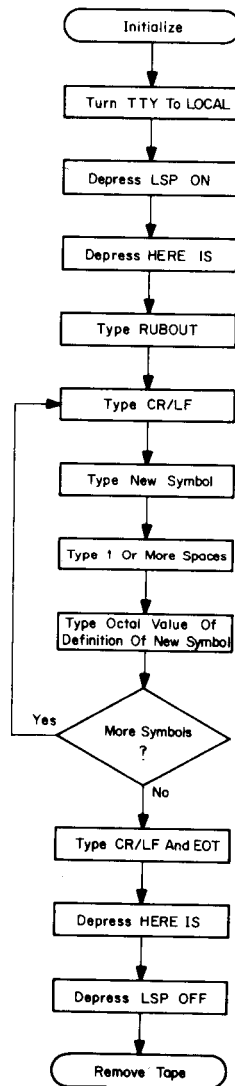


Figure DDT-4 Generating New External Symbol Tape Off-Line
(TTY and LSP Only)

RESTART PROCEDURE

Restart at location 5400 and DDT-8 will be in control.

If the user wishes to restart DDT-8 before he has punched a complete tape with checksum, he must restart at location 5401 to preserve the checksum.

EDITING NOTES

- a. Do not open any symbol table location.
- b. To enter a combined operate class and IOT instruction into an open location, the combination must contain no more than two mnemonics, the second of which must be CLA. Any other combination is ignored.
- c. The symbol table tape is loaded using the LSR only.
- d. Each user symbol occupies four locations in the symbol table area.
- e. Input is interrupted when symbol table storage is full.

COMMANDS

<u>Mode Control</u>	<u>Explanation</u>
[O	Sets DDT-8 to type out in octal mode.
[S	Sets DDT-8 to type out in symbolic mode.
<u>Input</u>	
[R	Read symbol tape into external table from LSR, or define new symbol from keyboard.
<u>Program Examination and Modification</u>	
k/	Open location k (k may be octal or symbolic).
RETURN	Close location currently open; enter modification, if any.
LINE FEED	Close location currently open and open next sequential location; enter modification, if any.
↑ (SHIFT/N)	Close location currently open and open location address therein; enter modification, if any.
<u>Breakpoint Insertion and Control</u>	
[B	Remove current breakpoint.
k[B	Insert a breakpoint at location k.
k[G	Go to location k and start program execution.
n[C	Continue from breakpoint, execute breakpoint n times and return control to user. If n is absent, it is assumed to be 1.
<u>Word Search</u>	
N[W	Begin word search for all occurrences of expression N masked by the contents of [M between the limits imposed by [L and [U. [M, [L, and [U are locations within DDT-8 which may be opened, modified, and closed exactly as any general register k in the user's program.

Output

[T	Punch leader/trailer code.
a;b[P	Punch binary tape from memory bounded by addresses a and b.
[E	Punch end of tape (i.e., checksum and trailer).

NOTE

The character [is generated by depressing the ALT MODE key.

Address Tags

Command

Explanation

[A	Accumulator storage (at breakpoints)
[L	Lower limit of search
[U	Upper limit of search
[M	Mask; used in search
[Y	Link storage (at breakpoints)

Special Characters

Character

Explanation

	(space)	Separation character
+	(plus)	Specifies address arguments relative to symbols
-	(minus)	Same as +
.	(period)	Current location; used in address arguments
=	(equal)	Type last quantity as an octal integer
RETURN		Make modifications, if any, and close register
LINE FEED		Make modifications, if any, close location, and open next sequential location.
/	(slash)	Location examination character; when following the address location, the location is opened and its contents printed
↑	(up-arrow)	When following a location printout, the location addressed therein is opened
←	(back-arrow)	Delete the line currently being typed

DIAGNOSTICS

DDT-8 checks for the errors listed below and types a ? when any is detected. All data between the error point and the previous tab or carriage return is ignored.

- a. Undefined or illegal symbol
- b. Illegal character
- c. Undefined control command
- d. Off-page addressing

PURPOSE

The Octal Debugging Technique for the PDP-8/I is a debugging program which facilitates communication with and alteration of the object program. Communication with the program is from the Teletype keyboard, using octal numbers. (See DEC-08-COCO-D for details.)

STORAGE
REQUIREMENTS

ODT-8 requires 600g consecutive locations, and 1 location on page 0 for breakpoint location.

- Low version: locations 1000-1577
- High version: locations 7000-7577

Breakpoint is initially at location 0004.

Starting Address: is 1000 (low) or 7000 (high)

LOADING

BIN is used to load ODT-8 and the object program into core. The user should have at the console the octal listing of his object program so that the listing can be updated to reflect any debugging change made to the program.

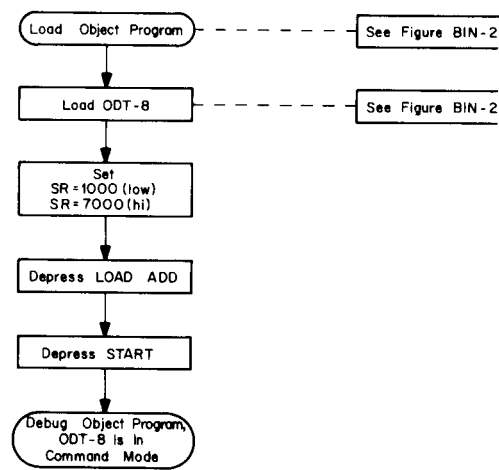


Figure ODT-1 Loading and Executing ODT-8

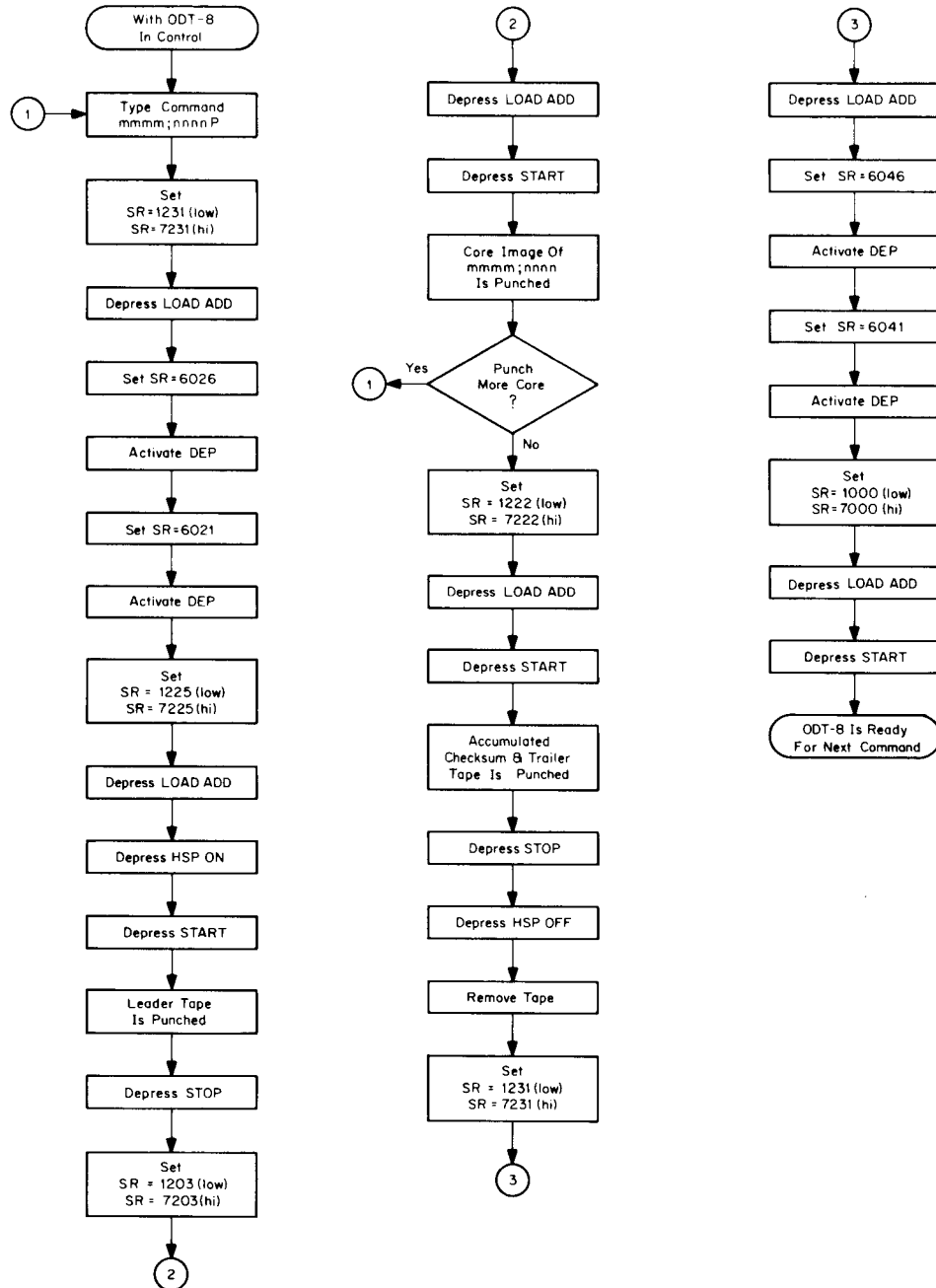


Figure ODT-2 Generating Binary Tape Using High-Speed Punch

COMMANDS

<u>Command</u>	<u>Explanation</u>
/	Reopen latest opened location.
nnnn/	Open location nnnn.
RETURN	Close previously opened location.
LINE FEED	Close location and open next sequential location.
↑ (SHIFT/N)	Close location, take contents of that register as a memory reference and open it.
← (SHIFT/O)	Close location, open indirectly.
nnnnG	Transfer program control to location nnnn.
B	Remove the breakpoint and restore original contents of that location.
nnnnB	Establish the breakpoint at location nnnn.
A	Open location containing AC.
C	Proceed from the breakpoint.
nnnnC	Continue from the breakpoint and iterate past the breakpoint nnnn times; stop at breakpoint
M	Open the search mask. Initially set to 7777
	LINE FEED Open lower search limit
	LINE FEED Open upper search limit
nnnnW	Search defined upper and lower limits of core for nnnn of search mask.
T	Punch leader/trailer tape.
mmmm;nnnnP	Punch binary core image of locations mmmm through nnnn.
E	Punch checksum and trailer tape.

RELOCATING THE BREAKPOINT

ZPAT (the breakpoint symbol) is initially set to location 0004. The breakpoint location can be relocated to any location on page 0 by setting ZPAT equal to the desired location.

RESTRICTIONS

Although ODT-8 is relocatable to any page, it will not operate outside the field in which it is located.

ODT-8 will not turn on the program interrupt. However, it does turn off the interrupt when a breakpoint is encountered. This prevents disrupting interrupts.

The user's program must not use or reference any location occupied or used by ODT-8.

The breakpoint location must not be used by the user program.

DIAGNOSTICS

When ODT-8 detects an error it types a ? followed by a carriage return-line feed. ODT-8 checks for the following conditions.

- a. Only legal control characters and octal digits are acceptable, any other character causes the character or whole line to be ignored.
- b. G typed alone is an error; control will be transferred to location 0000.
- c. Typing a P command with the punch ON is an error; ASCII characters will be punched on the binary tape.
- d. Octal numbers must be from 1 to 4 digits; more than four digits is an error.
- e. An illegal character (neither a valid control character nor a 1- to 4-digit octal number) causes the current line to be ignored.

PURPOSE

FOCAL (FOrmula CALculator) is an on-line, conversational, service program used to solve numerical problems (both simple and complex), to introduce programming to the student or computer novice, and much more. (See DEC-08-AJAC-D for details.)

STORAGE
REQUIREMENTS

FOCAL occupies location 1-3300 and 4600-7576 (6300₈ locations)
Extended functions occupy locations 4600-5377 (1000₈ locations)
Starting Address=0200

OPTIONAL
EQUIPMENT

LAB-8 or LINC-8 computer; Analog-to-Digital Converter (189);
Oscilloscope Display (VC8/I or 34D)

LOADING

BIN is used to load FOCAL into core. FOCAL accepts user programs either from the Teletype keyboard or the low-speed reader (see Figure BIN-2).

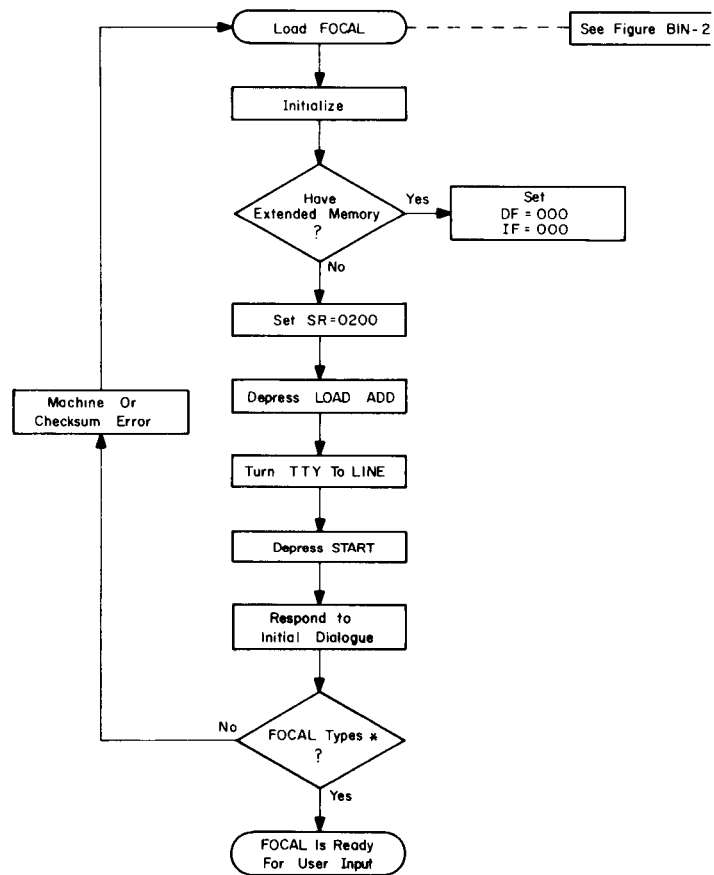


Figure FOCAL-1 Activating FOCAL For Use

INITIAL DIALOGUE

When FOCAL is in core , it types a greeting and identifies itself and the computer being used. FOCAL then asks three questions , to which the user should answer with either YES or NO as shown below.

CONGRATULATIONS!!
YOU HAVE SUCCESSFULLY LOADED 'FOCAL' ON A PDP-8 COMPUTER.

SHALL I RETAIN THE EXTENDED FUNCTIONS? :

The user types YES or NO in answer to this question;
for an explanation of the question , depress RETURN,
and FOCAL will type

'YES' GIVES YOU 'FSIN,FCOS,FATN,FLOG,FEXP' AND 1000
CHARACTERS. 'NO' GIVES YOU 1800 CHARACTERS.
SHALL I RETAIN THE EXTENDED FUNCTIONS? :YES

and repeats the question. The user answered YES.

DO YOU HAVE EXTENDED MEMORY? :YES
NOT YET AVAILABLE: I SHALL USE 4K.
HAVE YOU LOADED THE DISC SYSTEM? :YES
LIBRARY COMMAND IS NOT YET AVAILABLE.

The user answered YES, FOCAL replies

The user answered YES, FOCAL replies

*

The asterisk indicates that FOCAL is now ready
for user input.

COMMANDS

<u>Command</u>	<u>Abbr.</u>	<u>Example of Form</u>	<u>Explanation</u>
ASK	A	ASK ALPHA(I+2*J)	FOCAL types a : for each variable; the user types a value to define each variable.
COMMENT	C	COMMENT	Lines beginning with the letter C are ignored.
CONTINUE	C	C	
DO	D	DO 4.14	Execute line 4.14; return to command following the DO.
		DO 4	Execute all group 4 lines; return to command following DO or when a RETURN is encountered.
ERASE	E	ERASE	Erase the symbol table.
		ERASE 2	Erase all group 2 lines.
		ERASE 2.1	Delete line 2.1.
		ERASE ALL	Delete all user text.

FOR	F	FOR I=x,y,z; TYPE I	Command string following the ; is executed for each value; x,y,z are constants, variables, or expressions. x = initial value of I y = value added to I until I is greater than z; y is assumed = 1 if omitted.
GO	G	GO	Start indirect program at lowest numbered line number.
GOTO	G	GOTO 3.4	Start indirect program at line 3.4.
GO?		GO?	Starts at lowest numbered line number and traces indirect program until another ? is encountered, until an error is encountered, or until program completion.
IF	I	IF (x) 1.2,1.3,1.4;	Where x is identifier or expression. Control is transferred to the first, second, or third line number if (x) is less than, equal to, or greater than zero, respectively. If ; is encountered prematurely, the remainder of the line is executed.
MODIFY	M	MODIFY 1.15	Enables editing of characters on line 1.15. The next character typed becomes the search character. FOCAL positions itself after the search character; then the user may: type new text type CTRL/FORM for next occurrence type CTRL/BELL to change search character type RUBOUT to delete backwards type SHIFT/O to kill backwards type RETURN to end the line type LINE FEED to save rest of line
QUIT	Q	QUIT	Return control to user.
RETURN	R	RETURN	Terminate DO subroutine

SET	S	SET A=5/B*SCALE(3)	Define identifiers in symbol table.
TYPE	T	TYPE FSQT(AL*3+FSQT(B))	Evaluate expression, types out = and result in current output format.
		TYPE "TEXT STRING"!	FOCAL types text within quotation marks. Use ! to generate CR/LF.
WRITE	W	WRITE WRITE ALL WRITE 1 WRITE 1.1	FOCAL types the entire indirect program. FOCAL types all group 1 lines. FOCAL types line 1.1.

THE TRACE FEATURE

<u>Special Character</u>	<u>Example of Form</u>	<u>Explanation</u>
?	?...? or ?...	Those parts of the program enclosed in question marks will be printed out as they are executed. If only one ? is inserted, the trace feature becomes operative, and the program is printed out from that point until another ? is encountered, until an error is encountered, or until program completion.

CONTROL CHARACTERS

%	Format delimiter	Space	Name terminator
"	Text delimiter	'	Expression terminator
!	CR/LF generator	;	Command terminator
#	CR generator	RETURN	Line terminator
\$	Type symbol table		

SUMMARY OF FUNCTIONS

FSQT()	Square Root	FCOS()	Cosine
FABS()	Absolute Value	FATN()	Arc Tangent
FSGN()	Sign Part	FLOG()	Logarithm
FITR()	Integer Part	FDIS()	} Scope Functions
FRAN()	Random Number	FDXS()	
FEXP()	Exponential	FADC()	A-D Input
FSIN()	Sine	FNEW()	User Function

ARITHMETIC OPERATIONS

↑	Exponentiation	Order of precedence is as listed; properly paired enclosures are evaluated first; otherwise evaluation is from left to right.
*	Multiplication	
/	Division	
+	Addition	
-	Subtraction	

OVERLOAD RECOVERY

When the program and symbol table areas become too large the error diagnostic ?03.79 will be typed out. The user should then do one of the following.

- Type ERASE and depress RETURN.
- Restart at location 2216, if ?03.79 follows a legitimate command. This erases all variables.
- As a last resort, restart at 2213. This erases the text.

LOADING PROGRAM TAPES

When loading a long program tape into FOCAL the user can suppress the echo (printing) feature by changing the content of location 2475 to 7000. This will cause only asterisks to be typed as the tape is being read; there will not be a carriage return-line feed at the end of the line.

Entries from the keyboard will not echo unless each entry is preceded by a TYPE command. Output will be typed in the normal manner.

To restore the echo feature, depress the STOP key on the computer console and deposit 4277 into location 2475.

ESTIMATING LENGTH OF USER'S PROGRAM

FOCAL requires five words for each identifier stored in the symbol table, and one word for each two characters of stored program. This may be calculated by

$$5s + \frac{c}{2} \cdot 1.01 = \text{length of user's program}$$

where

s = Number of identifiers defined

c = Number of characters in indirect program

If the total program area or symbol table area becomes too large, FOCAL types the error

?03.79

FOCAL occupies core locations 1-3300 and 4600-7576. This leaves approximately 700_{10} locations for the user's program (indirect program, identifiers, and push-down list). The extended functions occupy locations 4600-5377. If the user decides not to retain the extended functions at load-time, there will be space left for approximately 1100_{10} characters for the user's program.

The following routine allows the user to find out how many core locations are left for his use.

```
*FOR I=1,300; SET A(I)=I
?03.79
*TYPE %4, I*5, " LOCATIONS LEFT "
=+ 705 LOCATIONS LEFT *
```

(disregard error code)

CALCULATING TRIGONOMETRIC FUNCTIONS

<u>Function</u>	<u>FOCAL Representation</u>	<u>Argument Range</u>	<u>Function Range</u>
Sine	FSIN(A)	$0 \leq A < 10^{14}$	$0 \leq F \leq 1$
Cosine	FCOS(A)	$0 \leq A < 10^{14}$	$0 \leq F \leq 1$
Tangent	FSIN(A)/FCOS(A)	$0 \leq A < 10^{14}$ $ A \neq (2N+1)\pi/2$	$0 \leq F < 10^{16}$
Secant	1/FCOS(A)	$0 \leq A < 10^{14}$ $ A \neq (2N+1)\pi/2$	$1 \leq F < 10^{16}$
Cosecant	1/FSIN(A)	$0 \leq A < 10^{14}$ $ A \neq 2N\pi$	$1 \leq F < 10^{16}$
Cotangent	FCOS(A)/FSIN(A)	$0 \leq A < 10^{14}$ $ A \neq 2N\pi$	$0 \leq F < 10^{1440}$
Arc sine	FATN(A/FSQT(1-A ²))	$0 \leq A < 1$	$0 \leq F \leq \pi/2$
Arc cosine	FATN(FSQT(1-A ²)/A)	$0 < A \leq 1$	$0 \leq F \leq \pi/2$
Arc tangent	FATN(A)	$0 \leq A \leq 10^{16}$	$0 \leq F < \pi/2$
Arc secant	FATN(FSQT(A ² -1))	$1 \leq A < 10^{16}$	$0 \leq F < \pi/2$
Arc cosecant	FATN(1/FSQT(A ² -1))	$1 < A < 10^{1300}$	$0 < F < \pi/2$
Arc cotangent	FATN(1/A)	$0 < A < 10^{1615}$	$0 < F < \pi/2$
Hyperbolic sine	(FEXP(A)-FEXP(-A))/2	$0 \leq A < 700$	$0 \leq F \leq 5*10^{1300}$

<u>Function</u>	<u>FOCAL Representation</u>	<u>Argument Range</u>	<u>Function Range</u>
Hyperbolic cosine	$(\text{FEXP}(A)+\text{FEXP}(-A))/2$	$0 \leq A < 700$	$1 \leq F < 5 \times 10^{+300}$
Hyperbolic tangent	$(\text{FEXP}(A)-\text{FEXP}(-A))/(\text{FEXP}(A)+\text{FEXP}(-A))$	$0 \leq A < 700$	$0 \leq F \leq 1$
Hyperbolic secant	$2/(\text{FEXP}(A)+\text{FEXP}(-A))$	$0 \leq A < 700$	$0 < F \leq 1$
Hyperbolic cosecant	$2/(\text{FEXP}(A)-\text{FEXP}(-A))$	$0 < A < 700$	$0 < F < 10^{+7}$
Hyperbolic cotangent	$(\text{FEXP}(A)+\text{FEXP}(-A))/(\text{FEXP}(A)-\text{FEXP}(-A))$	$0 < A < 700$	$1 < F < 10^{+7}$
Arc hyperbolic sine	$\text{FLOG}(A+\text{FSQT}(A^2+1))$	$-10^{+5} < A < 10^{+600}$	$-12 < F < 1300$
Arc hyperbolic cosine	$\text{FLOG}(A+\text{FSQT}(A^2-1))$	$1 \leq A < 10^{+300}$	$0 \leq F < 700$
Arc hyperbolic tangent	$(\text{FLOG}(1+A)-\text{FLOG}(1-A))/2$	$0 \leq A < 1$	$0 \leq F < 8.31777$
Arc hyperbolic secant	$\text{FLOG}((1/A)+\text{FSQT}((1/A)^2-1))$	$0 < A \leq 1$	$0 \leq F < 700$
Arc hyperbolic cosecant	$\text{FLOG}((1/A)+\text{FSQT}((1/A)^2+1))$	$0 < A < 10^{+300}$	$0 \leq F < 1400$
Arc hyperbolic cotangent	$(\text{FLOG}(X+1)-\text{FLOG}(X-1))/2$	$1 < A < 10^{+616}$	$0 \leq F < 8$

DIAGNOSTICS

FORMAT: ?nn.nn @ nn.nn error code @ line number

?00.00	Manual start given from console
?01.00	Interrupt from keyboard via CTRL/C
?01.35	Group zero is an illegal line number
?01.43	Illegal step or line number used
?01.89	GOTO was not used as <u>one</u> word
?01.;2	Line number is too large
?01.:3	Double periods found in a line number
?02.48	Nonexistent line referenced by DO
?02.63	Nonexistent group referenced by DO
?02.81	Storage was filled by push-down list
?03.09	Nonexistent line used or a tight loop
?03.31	Illegal command used
?04.07	No space after IF or illegal format
?04.35	Left of = in error in FOR or SET
?04.48	Excess right parenthesis encountered
?04.56	Illegal terminator in FOR command
?05.63	Bad argument to MODIFY
?06.13	Illegal use of function or number
?06.64	Storage is filled by variables

<u>Code</u>	<u>Meaning</u>
?07.14	Operator missing in an expression or double E
?07.34	No operator used before parenthesis
?07.<0	Double operators used
?07.; 1	No argument given after function call
?07.; 8	Illegal function name given
?08.50	Parentheses do not match
?09.16	Bad argument in ERASE
?09.50	Maximum group number exceeded
?11.20	Input buffer has overflowed
?12.83	Storage was filled by text
?20.41	Logarithm of zero requested
?23.35	Literal number is too large
?26.91	Negative exponent used.
?26.96	Exponent is too large
?28.58	Division by zero requested
?30.48	Imaginary square roots required
?31.<7	Illegal character or unavailable command or unavailable function used.

NOTE

The above diagnostics apply only to the version of FOCAL
1968 issued on tape DEC-08-AJAC-PB.

PURPOSE

FORTRAN (4K), FORMula TRANslator, for the PDP-8 computer is used to compile, debug, and operate a user program written in the PDP-8 version of the FORTRAN language. Compilation requires only one pass. (See DEC-08-AFC0-D for details.)

STORAGE
REQUIREMENTS

Compiler and symbol table requires locations 0003-7577 (7574₈ locations)

Starting Address=0200

Symbolprint requires locations 0600-0777 (200₈ location)

Starting Address=0600

Operating System requires:

locations 0-5177 for paper tape I/O (5200₈ locations)

locations 0-5777 for DECtape I/O (6000₈ locations)

Starting Address=0200

LOADING

BIN is used to load the Compiler, Symbolprint, and Operating System into core. The user's program is loaded by the appropriate FORTRAN system program above.

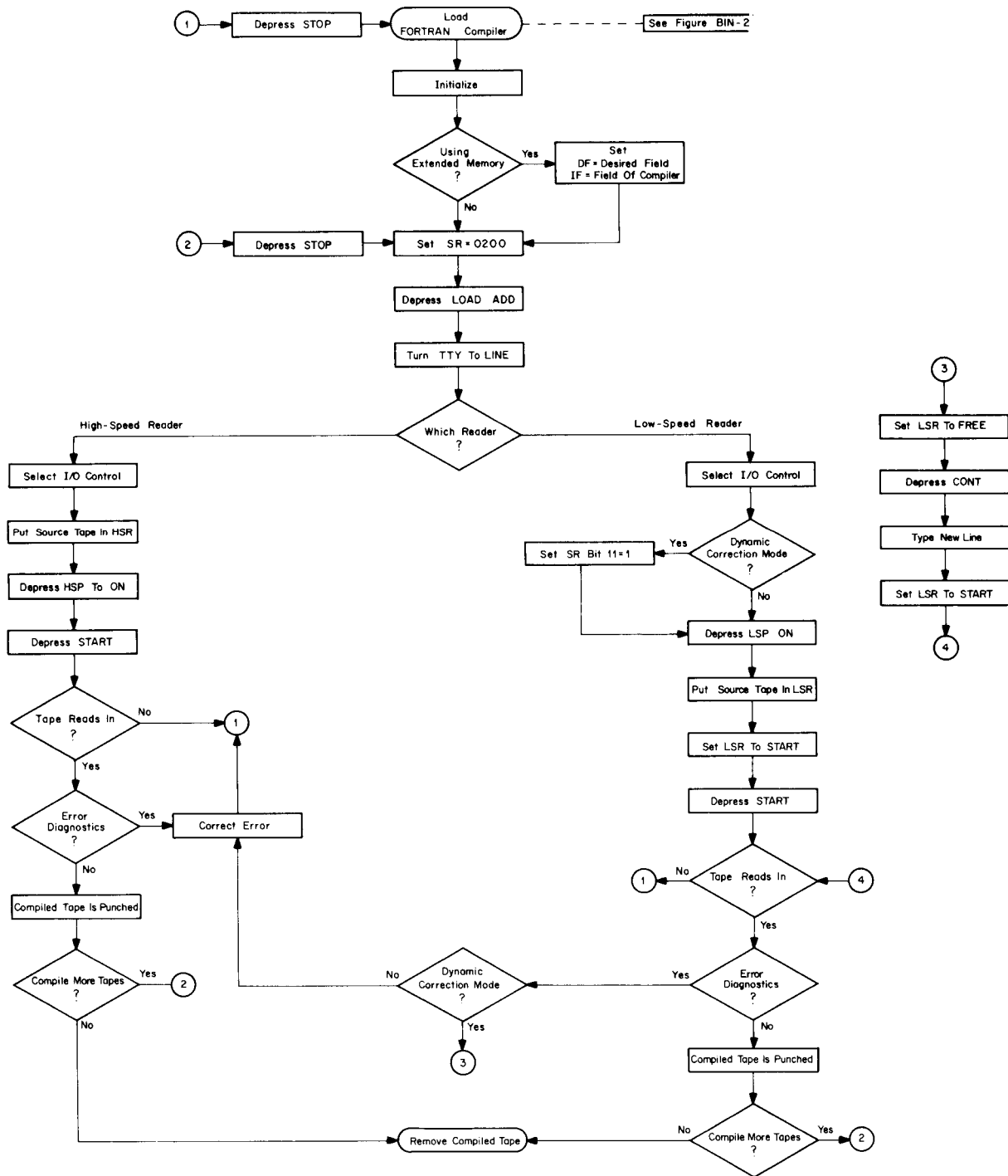


Figure FORTRAN(4)-1 Compiling a FORTRAN Program

SYMBOLPRINT

The Symbolprint program is used to print out a memory map of the compiled source program. The memory map is useful when debugging the program. Symbolprint is run immediately after compiling a source program and before compiling another or loading the Operating System.

NOTE

Symbolprint destroys the Compiler's DECTape I/O processors. Therefore, the Compiler must be reloaded to compile a source program containing DECTape I/O statements.

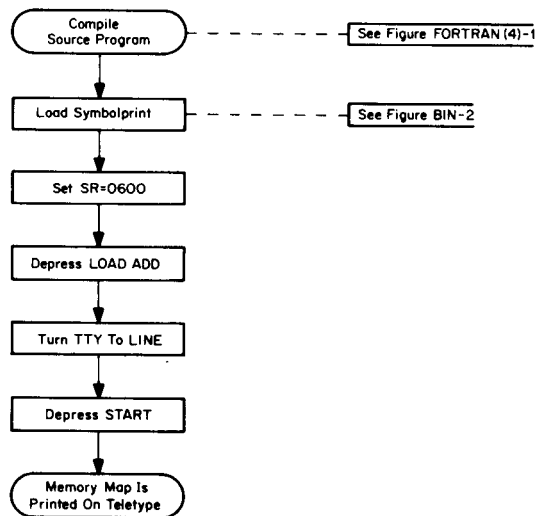


Figure FORTRAN(4)-2 Generating a Memory Map Using Symbolprint

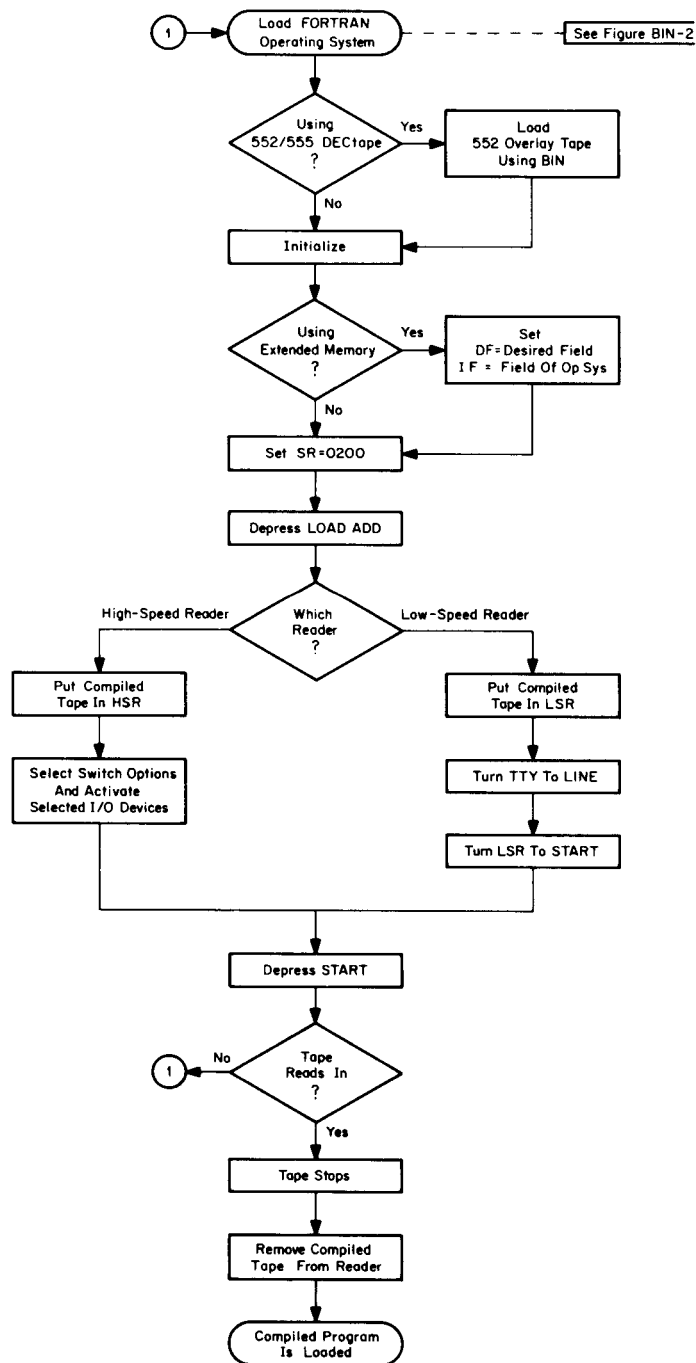
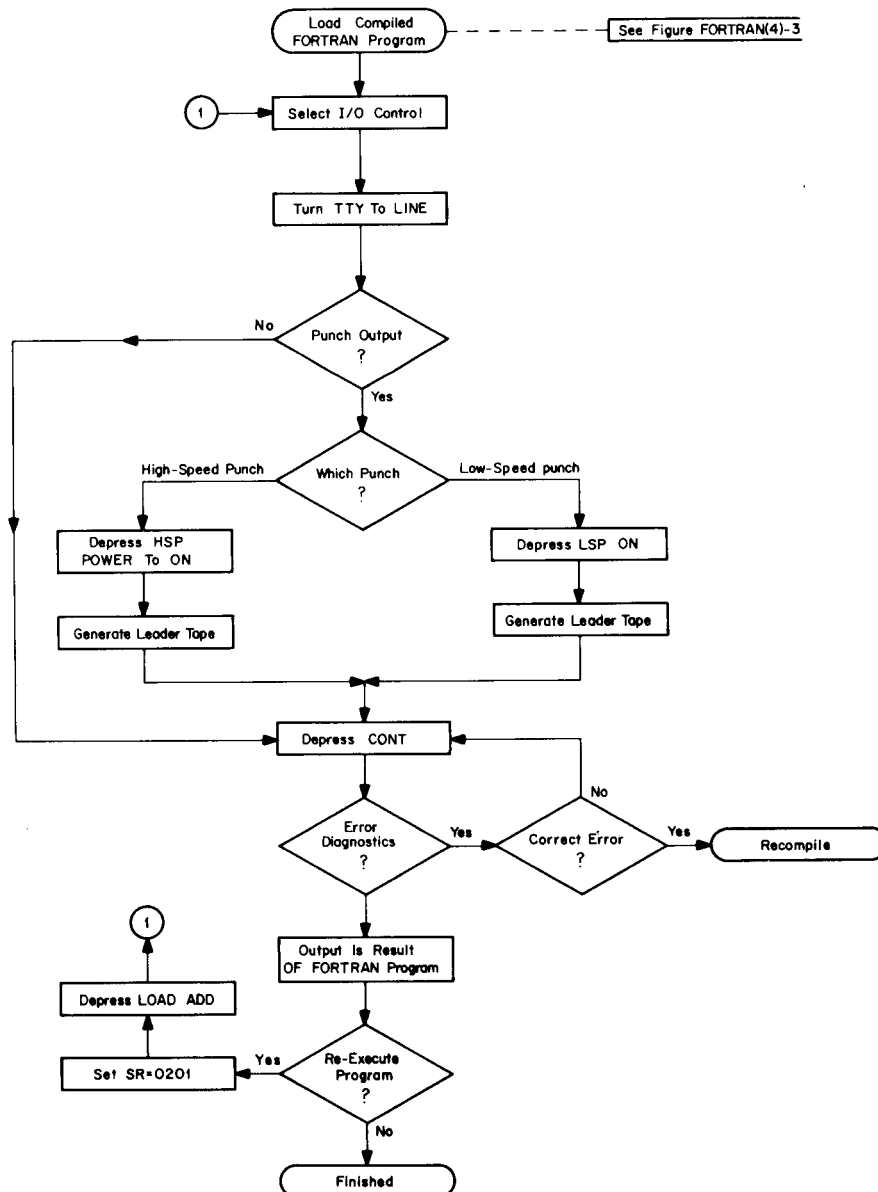


Figure FORTRAN(4)-3 Loading a Compiled FORTRAN Program



NOTE

If the FORTRAN program has been debugged, the internal stack overflow/underflow test can be removed to speed up program execution by setting location 0404=7000.

Figure FORTRAN(4)-4 Executing a Compiled FORTRAN Program

INPUT/OUTPUT CONTROL

The selection of I/O devices for both the Compiler and Operating System is controlled by setting the console switches as shown below.

<u>SR Bit</u>	<u>Set To</u>	<u>Results</u>
0	0	The program contains only paper tape I/O statements.
	1	The program contains DECTape I/O statements.
1	0	Compiler: Use low-speed reader for input of source program. Operating System: Use low-speed reader for input of object program and the keyboard for ACCEPT statements.
	1	Use the high-speed reader.
2	0	Compiler: Use Teletype and low-speed punch for compiler output (interpretive code) and diagnostics. Operating System: Use Teletype and low-speed punch for TYPE statements.
	1	Use the high-speed punch (diagnostics still appear on the printer).

I/O selections cannot be changed without reloading the compiler.

DECTAPE I/O STATEMENTS OPTION

The DECTape I/O statements are the READ and WRITE Statements. The I/O statements option must be set before compiling or running a program.

<u>SR Bit</u>	<u>Set To</u>	<u>Results</u>
0	0	Delete DECTape I/O processing routines.
	1	Use DECTape I/O processing routines.

DYNAMIC ERROR CORRECTION

When compiling in the dynamic correction mode (see Note, below) the user can correct a statement, which the compiler has determined contains a source-language error, by reentering the offending line from the keyboard. To implement the dynamic correction mode set SR bit 11 = 1 as shown in Figure FORTRAN(4)-1. If an error is detected, the diagnostic is typed out in the normal fashion and the computer halts. To correct the statement:

- a. Set LSR to FREE
- b. Depress CONT
- c. Type the new line in its entirety (excluding the statement number if any) followed by a carriage return-line feed.
- d. Set LSR to START and compilation will continue.

To leave the dynamic correction mode, restart the compiler at location 0200 with SR bit 11 = 0.

NOTE

This feature applies only to the low-speed paper tape reader.

DIAGNOSTICS

Compiler

Format: xxxx yy zz

where xxxx is statement number of last numbered statement, yy is numbered statement, yy is number of statements since the last numbered statement, and zz is error code (numbers in octal).

<u>Code</u>	<u>Explanation</u>
00	Fixed- and floating-point modes are mixed.
01	Two operators next to each other.
02	Compiler error; reload compiler.
03	Illegal comma in arithmetic statement.
04	Too many operators in a statement.
05	Function argument is in fixed mode.
06	Variable subscript in floating-point mode, or an operator is missing.
07	More than 64 ₁₀ variable names in program.
10	Program too large.
11	Unpaired parentheses.
12	Illegal character.
13	Error in statement format.
14	Program too large, or duplicate statement numbers.
15	Subscripted variable defined prior to DIMENSION statement or subscripted variable not in DIMENSION statement, or operator missing in fixed-mode expression.
16	Statement too long.
17	Floating-point operand should be fixed-point.
20	Referenced statement number not in program.
21	More than 40 ₁₀ numbered statements in program.
22	Too many incompleting operations in statement.
23	More than 20 ₁₀ statements referenced before being defined.
24	Illegal attempt to compile READ or WRITE statement.

Format: "TILT" nn
where nn is the error code.

Action Taken When
CONT is DepressedCodePossible Cause

11	Attempt to divide by zero.	Quotient set to + or - largest number representable in computer and execution continues.
12	Floating-point exponent on input greater than + or - 2047.	System executes next instruction.
13	Illegal operation code.	System executes next instruction.
14	Transfer to location 0 or 1.	No recovery possible; recompile.
15	Nonformat statement used as FORMAT statement.	Next instruction is executed.
16	Illegal FORMAT statement	Rest of statement is examined.
17	Attempt to fix large floating-point number.	Ignored.
20	Attempt to square root a negative number.	Square root of absolute value is taken.
21	Attempt to raise a negative number to a power.	Absolute value is raised to power specified.
22	Attempt to find logarithm of 0 or negative number.	Attempts to find logarithm of absolute value.
31	Select error: system halts with called unit in bits 0-2 of AC (0-3 with DECtape 552/555).	Recovered by correcting logical unit and depressing CONT.
32	Physical tape error.	System halts with error status in AC.
33	DECtape buffer exceeded.	Ignored.
34	DECtape control switch set incorrectly.	Ignored.
76	System stack overflow.	No recovery possible. Recompile.
77	System stack overflow.	Same as above.

PURPOSE

The FORTRAN(8K) Compiler and Operating System is used to compile a source program written in the FORTRAN(8K) language and to debug and execute the compiled program. FORTRAN(8K) consists of a two-pass Compiler, Linking Loader, Run-Time Monitor, Operating System, and a library of subprograms. (See DEC-08-A2A0-D for details.)

STORAGE
REQUIREMENTS

Compiler requires locations 1000-2000 (1001_8 locations)

Starting Address = 011000

Operating System requires locations 0006-7577 (7572_8 locations)

Starting Address = SA of Main Program (see Memory Map).

EQUIPMENT
REQUIREMENTS

PDP-8/I, 8/L, 8, 8/S, or 5 computer with 8K words of core memory and a high-speed reader and punch. The PDP-5 requires a PDP-8 extended memory control modification.

LOADING

FORTRAN(8K) is loaded into core memory using the BIN Loader. The user's FORTRAN source program is loaded, compiled and executed using FORTRAN(8K).

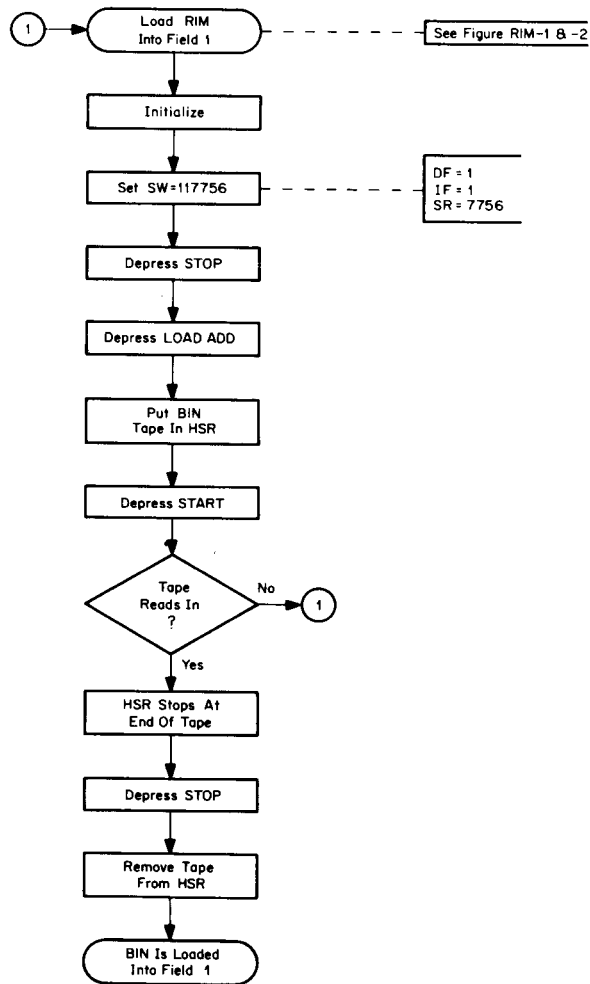


Figure FORTRAN(8K)-1 Loading BIN Into Data Field 1
Using RIM with HSR

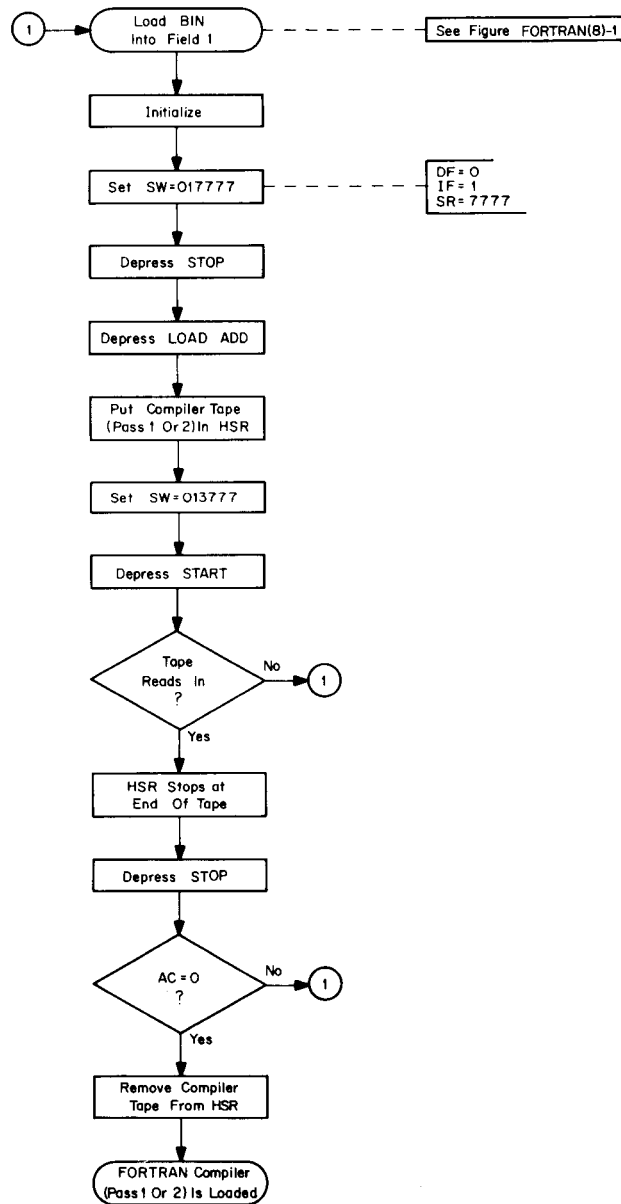


Figure FORTRAN(8K)-2 Loading FORTRAN Compiler (Pass 1 or 2)
Into Field 1 Using BIN with HSR

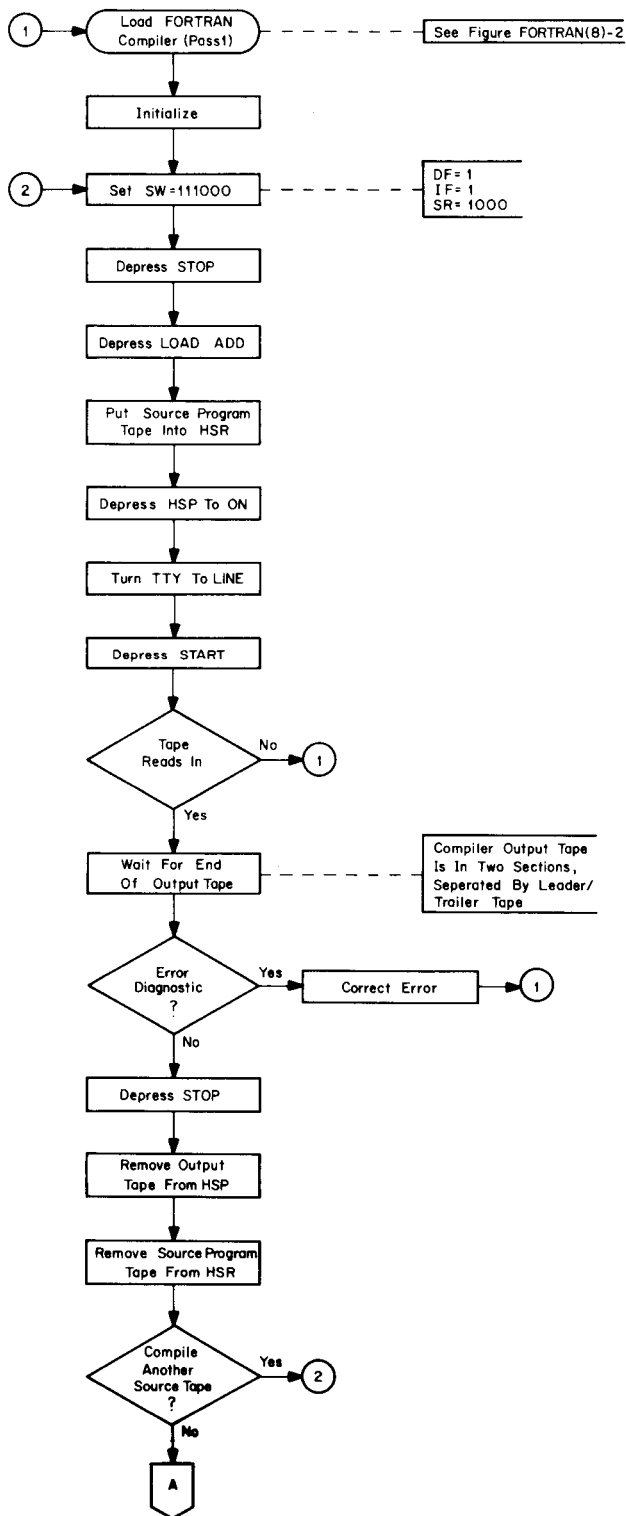


Figure FORTRAN(8K)-3 Executing Pass 1 of the FORTRAN Compiler

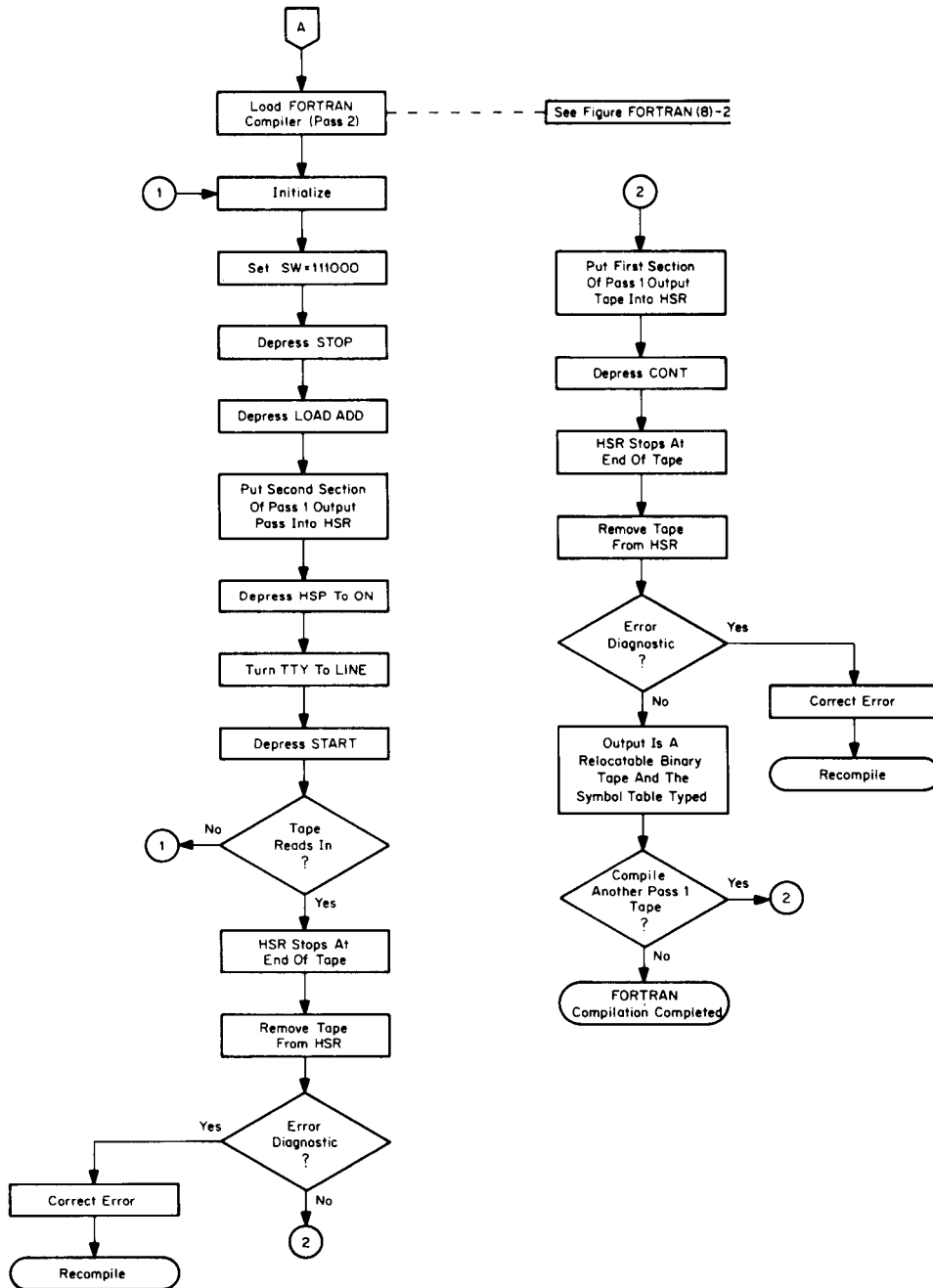


Figure FORTRAN(8K)-4 Executing Pass 2 of the FORTRAN Compiler

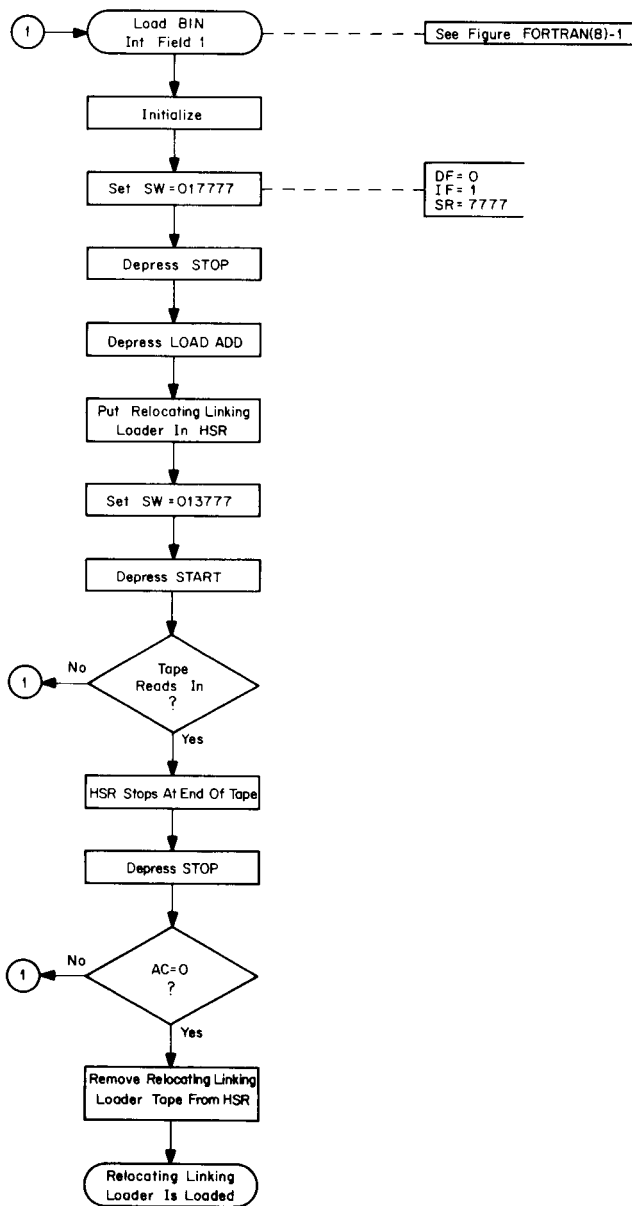


Figure FORTRAN(8K)-5 Loading Relocating Linking Loader
Using BIN with HSR

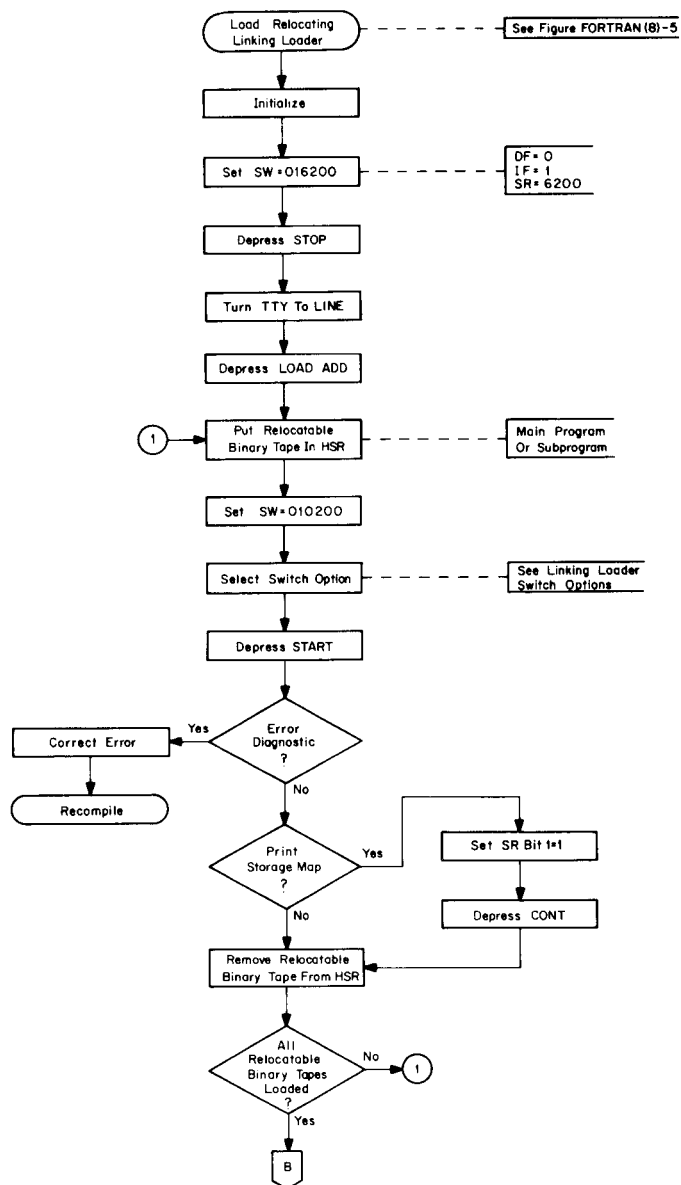


Figure FORTRAN(8K)-6 Loading Relocatable Binary Tapes
Using the Relocating Linking Loader

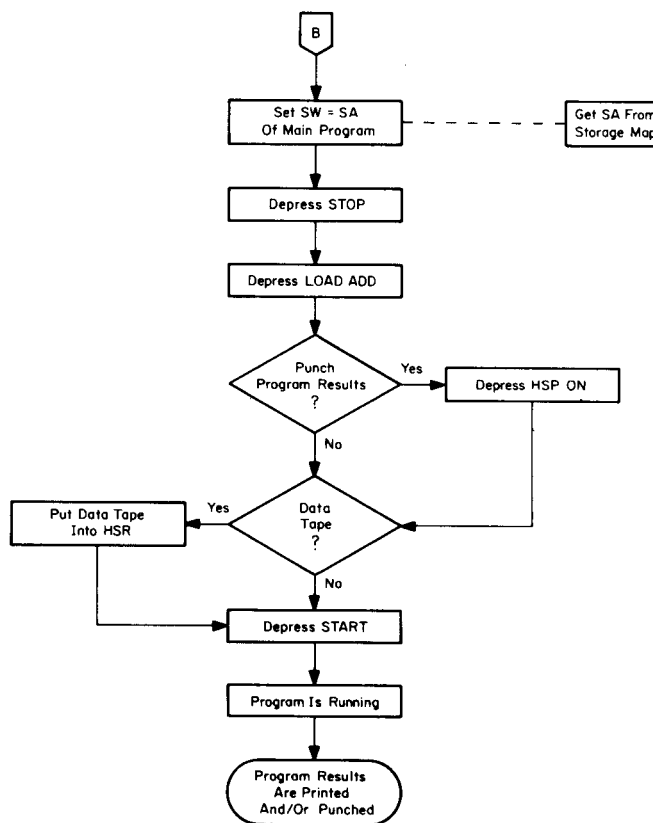


Figure FORTRAN(8K)-7 Executing Stored FORTRAN(8K) Program

Pass 2 Symbol Table

Immediately following the punching of the pass 2 relocatable binary tape, the entire symbol table is typed. Symbols are grouped in descending order as follows.

- a. External symbols (predefined symbols are suffixed with U2)
- b. Compiler symbols (preceded by a left bracket)
- c. User variables (preceded by a back-slash)
- d. Constants (preceded by a right bracket)
- e. Statement numbers (preceded by an up-arrow)
- f. Special compiler symbols (preceded by a left-arrow)

All undefined symbols are suffixed with U1, and are usually due to a reader error.

Linking Loader Restrictions

There are only two restrictions when loading with the Linking Loader:

- a. The first program or subprogram loaded must have the largest common storage declaration.
- b. A program or subprogram must not cross a memory field boundary.

Linking Loader Switch Options

The available switch options are:

Bit 0=1	Print the number of pages remaining in each field; do not print storage map.
Bit 0=0	Do not print.
Bit 1=1	Print storage map followed by number of remaining pages.
Bit 1=0	Do not print.
Bit 10=1	Load next subprogram into field 1.
Bit 10=0	Do not load next subprogram into field 1.
Bit 11=1	Load next subprogram into field 0.
Bit 11=0	Do not load next subprogram into field 0.

If neither bit 0 nor bit 1 is set to 1, another subprogram will be loaded upon depressing CONT.

If neither bit 10 nor bit 11 is set to 1, the next subprogram will be loaded in the same memory field as the preceding subprogram.

If both bits 10 and 11 are set to 1, the next subprogram will be loaded into field 1.

DIAGNOSTICS

Compiler

Format: E#XXXX YYYY+ZZZZ

where XXXX is an error code, YYYY is the statement number of the last numbered statement, and ZZZZ is the number of statements (including comments) since the last numbered statement.

DO?? at the end of the compiled tape means there is an ended DO loop or illegal DO loop nesting within the program.

Pass 1

<u>Error Code</u>	<u>Explanation</u>
0001	Illegal continuation line
0002	Line too long or improperly formed statement
0003-0009	Improperly formed arithmetic expression
0010-0011	Improper real constant
0012	Improper integer constant
0013-0014	Improper real constant
0015	Pass 1 symbol table exceeded
0016-0017	Improperly formed COMMON statement
0018-0025	Improperly formed DIMENSION statement
0026-0027	Improperly formed computed GO TO statement
0028	Improper index in computed GO TO statement
0029-0032	Improperly formed IF statement
0033-0035	Improperly formed DO statement
0036	Too many nested DO statements
0037	Improperly formed FORMAT statement
0038	Improperly formed arithmetic expression
0039	Arithmetic expression too complex
0040	Improperly formed arithmetic expression
0041	Mixed mode arithmetic expression
0042-0045	Improperly formed SUBROUTINE or FUNCTION statement
0046-0049	Improperly formed READ or WRITE statement
0050	Excessive subscripts
0051	Illegal equivalencing
0052-0056	Improperly formed EQUIVALENCE statement

Pass 2

<u>Error Code</u>	<u>Explanation</u>
0672	Hollerith statement counted incorrectly
3303 } 4524 }	Pass 2 symbol table exceeded
6636	Statement number duplicated

Any other pass 2 error can be corrected by repeating pass 1.

Linking Loader

Format: ERROR XXXX
where XXXX is the error code number.

<u>Error Code</u>	<u>Explanation</u>
0001	Loader has read more than 6410 subprogram names
0002	Current subprogram is trying to load above location 7600
0003	Current subprogram is trying to increase the size of common storage
0004	Checksum error on input tape
0005	Illegal relocation code
0006	An error is in pass 2

All errors are fatal except 0002, which can be corrected by resetting SR=correct option, putting leader code of tape over reader head, and depressing CONT.

Run-Time Monitor

Format: "XXXX" ERROR AT LOC NNNN
where XXXX is error code and NNNN is location of error.

<u>Error Code</u>	<u>Explanation</u>
"ATAN"	Result exceeds capacity of computer
"DIVZ"	Attempt to divide integer by 0
"EXP"	Result exceeds capacity of computer
"FMT1"	Multiple decimal points
"FMT2"	E or . in integer
"FMT3"	Illegal character in I, E, or F field
"FMT4"	Multiple minus signs
"FMT5"	Invalid FORMAT statement
"FPNT"	Improperly formed floating-point number
"SQRT"	Attempt to square root a negative number

To pinpoint the location of a run-time error:

- From the storage map, determine the next lowest numbered location (external symbol) which is the entry point of the program or subroutine containing the error.
- Subtract (in octal) the entry point location of the program or subroutine containing the error from the LOC of the error in the error message.
- From the pass 2 symbol table, determine the relative address of the external symbol found in step a and add that relative address to the result of step b.
- The sum of step c is the relative address of the error, which can then be compared with the relative addresses of the numbered statements in the program.

SECTION 3

DECTAPE

PURPOSE

The TC01 Bootstrap Loader is used to load the DECtape Library System programs into core memory. See DEC-08-LUAA-D and Section 4 of this manual for details.

STORAGE REQUIREMENTS

The TC01 Bootstrap Loader requires locations 7600-7623 (24₈ locations).
Starting Address=7600

REQUIRED EQUIPMENT

DECtape Control (TC01) and at least one DECtape Transport (TU55)

LOADING

The TC01 Bootstrap Loader may be toggled into core using the console switches or it may be read into core using the RIM Loader. The locations and corresponding instructions are listed below.

<u>Location</u>	<u>Instruction</u>
7600	6224
7601	6774
7602	1221
7603	4213
7604	1222
7605	3355
7606	1223
7607	4213
7610	0000
7611	0000
7612	0000
7613	0000
7614	6766
7615	3354
7616	6771
7617	5216
7620	5613
7621	0600
7622	7577
7623	0220

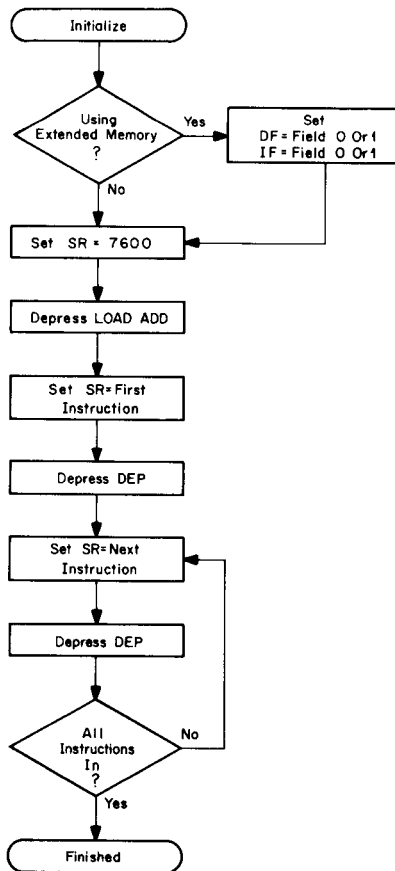


Figure DECTAPE-1 Toggling in TC01 Bootstrap Loader

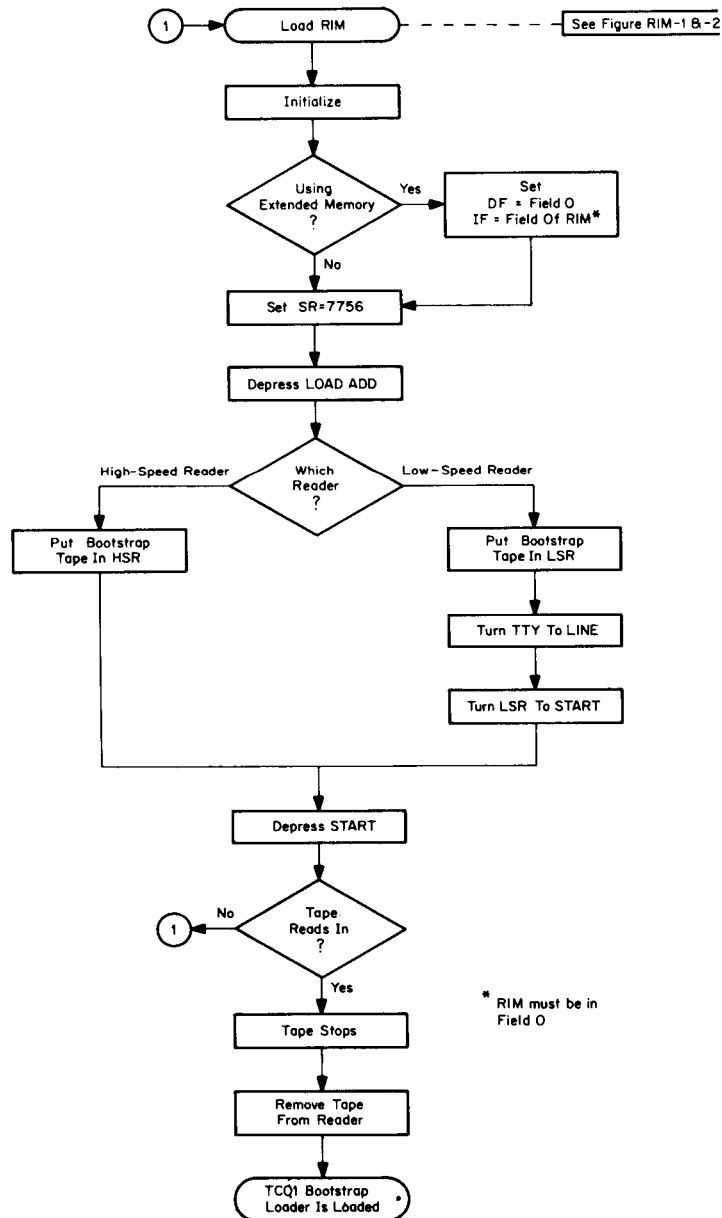


Figure DECTAPE-2 Loading the TC01 Bootstrap Loader Using RIM

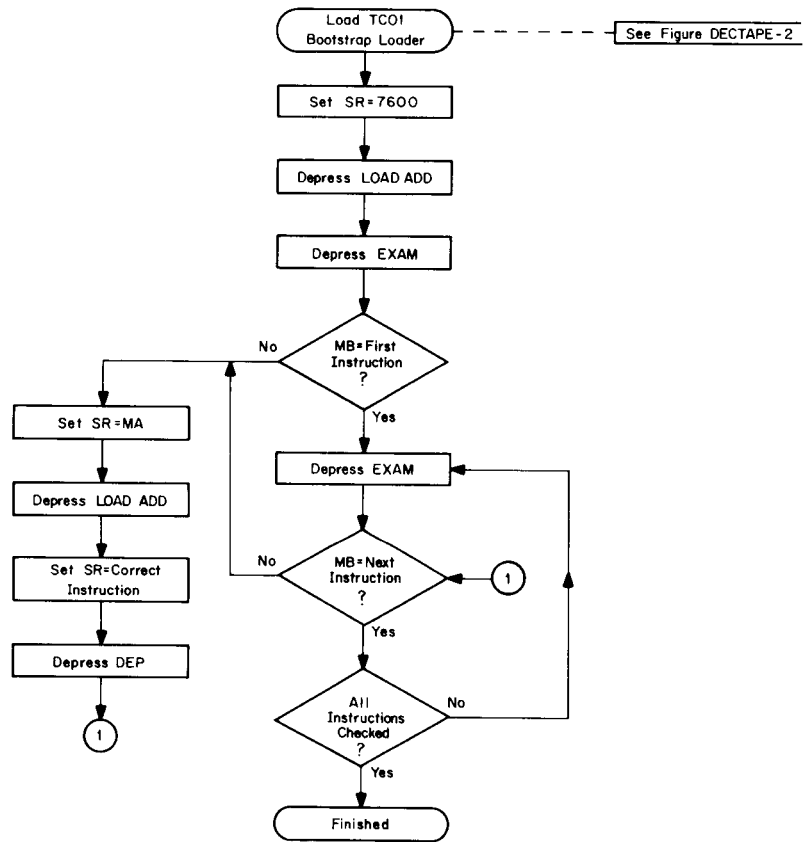


Figure DECTAPE-3 Checking TC01 Bootstrap Loader

PURPOSE

The DECTape Library System is a collection of five programs (INDEX, ESCAPE, UPDATE, DELETE, and GETSYS) stored on DECTape. They are used to load named files into core memory, define new named files, delete named files, and to create a new Library System. See DEC-08-SUB0-D for details.

STORAGE
REQUIREMENT

The five library programs will occupy the first 40₈ blocks of a certified DECTape.

EQUIPMENT
REQUIREMENT

The DECTape Library System requires a DECTape control (TC01) and at least one DECTape transport (TU55).

LOADING

The TC01 Bootstrap Loader is used to load the DECTape Library System from DECTape into core memory.

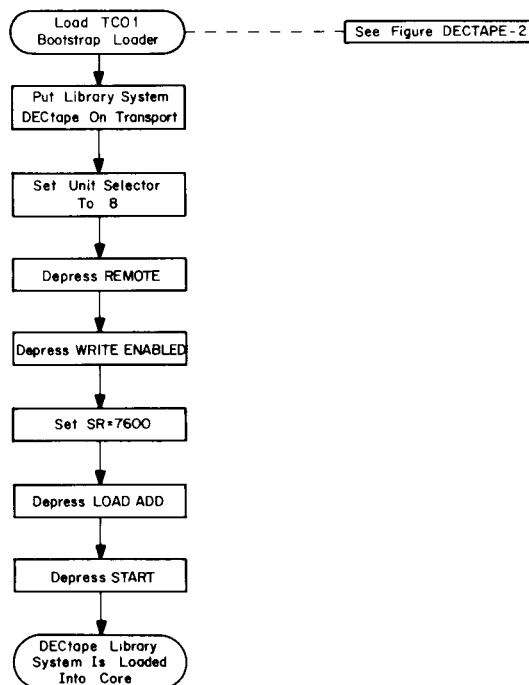


Figure DECTAPE-4 Loading DECTape Library System
Using TC01 Bootstrap Loader

LIBRARY SYSTEM

The Library System tape contains the five permanent programs explained below.

INDEX	causes the names of all files on the library tape to be typed.
ESCAPE	causes the Library System to exit core, and restores the RIM, BIN, and DECTape Bootstrap Loaders in core.
UPDATE	allows the user to add files to the library tape. When called, UPDATE types questions to be answered by the user (questions are underlined): <u>NAME OF PROGRAM: FRTRAN</u> user types a program name consisting of from one to six characters delimited by a carriage return. <u>SA (OCTAL) : 0200</u> user types an octal address delimited by a carriage return. <u>PAGE LOCATIONS: <0,2200><2400><4600,7577>;</u> user types the locations required by his program. A typing error causes UPDATE to retype the question.
DELETE	removes specified user program from the library tape. DELETE types a question to be answered by the user: <u>NAME OF FILE TO BE DELETED: FRTRAN</u> user types name of program to be deleted.
GETSYS	creates, on a specified tape unit, a new Library System tape consisting of the loaders and the system programs. When called, GETSYS types a question to be answered by the user: <u>SKELETON TAPE WILL BE CREATED ON UNIT# 5</u> user types a single digit from one to seven terminated by a carriage return.

SECTION 4
DISK MONITOR SYSTEM

PURPOSE

The Disk System Builder program is an easy-to-use dialogue technique used to build the customized Monitor suited to the particular machine configuration and to store the created Monitor on the system device. The Monitor is then used to create and save the System Program Library on the system device. See DEC-D8-SDAA-D for details.

STORAGE
REQUIREMENTS

See DEC-D8-SDAA-D for specific core requirements.

EQUIPMENT
REQUIREMENTS

A 4K PDP-8/I computer with 3-cycle data break, an ASR33 Teletype, a high-speed reader/punch, and a DF32 Disk. A TC01 DECtape Control with at least one TU55 DECtape Transport unit may also be used.

LOADING

BIN is used to load the Disk System Builder program into core.

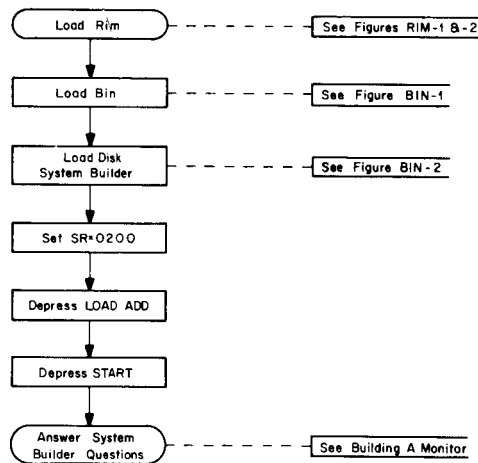


Figure DISK-1 Loading the Disk System Builder
Using the BIN Loader

BUILDING A MONITOR

When the System Builder is first loaded into core it will ask the following questions, which the user answers according to his machine configuration. (User response is underlined, and each response is terminated by depressing the RETURN key.)

*TYPE SIZE OF CORE (IN K)

*8

User enters core size of his computer

*HIGH SPEED PAPER TAPE?

*YES

User answers YES or NO

*PDP-8/S?

*NO

User answers YES or NO

*DISK?

*YES

User answers YES or NO

*TYPE NUMBER OF DISK UNITS?

*1

User types number of disk units in his machine configuration.

*TAPE?

*YES

User types YES if he has DECtape, NO if he does not

.

NOTE: If specified as present, the disk is automatically selected as the system device; if not, DEC-tape unit 8 is selected.

SYSTEM MODES

The system is always running in either Monitor mode or user mode.

Monitor Mode	is entered whenever the Monitor is started or when CTRL/C (↑ C) is typed; Monitor responds with a dot (.) typeout.
User Mode	is entered whenever the system is executing a system or user program; user mode is indicated by an asterisk (*) typeout.

PURPOSE

The Disk System Bootstrap loaders are used to load the Disk Monitor into core. The bootstrapping of Monitor into core is necessary only when the resident Monitor area (locations 7600-7777) has been cleared or its contents have been otherwise destroyed. System Builder leaves the resident portion of Monitor in core after building. There are two bootstrap routines, depending upon the type of system device. See DEC-D8-SDAA-D for details.

STORAGE REQUIREMENTS

Disk requires locations 200-204 and 7750-7751 (7 locations). Starting Address=0200.

DECtape requires location 200-222 (23₈ locations). Starting Address=0200.

REQUIRED EQUIPMENT

A 4K PDP-8/I computer with 3-cycle data break; and ASR33 Teletype; a high-speed reader/punch; a DF32 Disk and/or a TC01 DECtape Control with at least one TU55 DECtape Transport.

LOADING

Disk and DECtape Bootstrap loaders are loaded (toggled) into core memory using the console switches. The locations and corresponding instructions for both system devices are listed below.

Disk Bootstrap Loader

<u>Location</u>	<u>Instruction</u>	<u>Symbolic</u>
0200	6603	DMAR
0201	6622	DFSC
0202	5201	JMP .-1
0203	5604	JMP I .+1
0204	7600	7600
7750	7576	
7751	7576	

DECtape Bootstrap Loader

			*200
0200	7600	BEG,	7600
0201	1216		TAD MVB
0202	4210		JMS DO
0203	1217		TAD M201
0204	3620		DCA I CA
0205	1222		TAD RF
0206	4210		JMS DO
0207	5600		JMP I BEG
0210	0000	DO,	0000

<u>Location</u>	<u>Instruction</u>	<u>Symbolic</u>
0211	6766	DTXA DTCA
0212	3621	DCA I WC
0213	6771	DTSF
0214	5213	JMP .-1
0215	5610	JMP I DO
0216	0600	MVB, 0600
0217	7577	M201, -201
0220	7755	CA, 7755
0221	7754	WC, 7754
0222	0220	RF, 0220

After toggling in one of the above bootstrap routines, set the SR to 0200 and depress LOAD ADDRESS and START. Monitor should respond with a dot (.) after it has been brought into core.

PURPOSE

The Disk System Program Library is a collection of programs in an open ended Monitor librarian which allows for easy additions and deletions. The standard package of programs includes an Editor, transparent DDT, Assembler, FORTRAN, Peripheral Interchange Program (PIP), and a transparent Loader. All the above programs are device independent and may use the disk for source files, binary output, symbol table space, and overlay. The system is file structured and hardware independent via the System Builder program.

STORAGE
REQUIREMENTS

See DEC-D8-SDAA-D for specific storage requirements.

LOADING

The Disk/DECtape System Programs are loaded using the Monitor. See DEC-D8-SDAA-D for details.

DISK LIBRARY

The library system includes the following programs. See DEC-D8-SDAA-D for complete details.

DDT	is the standard DDT-8 but is overlaid to appear to be only two pages long, and it has three breakpoints.
EDITOR	is a device independent version of the Symbolic Editor, allowing sources to be edited using disk, high-speed reader, or Teletype in any combination.
PAL-D	is a device independent version of the MACRO-8 Assembler (without macros and floating-point pseudo-ops) which allows storage for over 1000 symbols.
FORTTRAN	is a device independent version of 4K FORTRAN, and is capable of load-and-go operation.
PIP	is a general utility program; it relieves the programmer of tape copying tasks. It includes directory list and delete functions.
LOADER	is a transparent, device independent version of the BIN Loader.
SAVE/CALL	are Monitor commands which allow the user to save and retrieve, on a page basis, segment of core.

SECTION 5
SYSTEM DEMONSTRATIONS

The demonstration programs convert the decimal numbers 20 through 30 into octal and type the octal numbers on the Teletype printer. These runnable programs demonstrate the ease with which DEC's system, utility, and service programs can be used. Each major step involved in writing, loading, assembling, compiling, and executing the programs is explained; for most operations, the reader is referred to the appropriate flowchart in Section 2.

The machine configuration being used is a 4K PDP-8 computer, an ASR33 Teletype, and a high-speed paper tape reader/punch.

A FORTRAN PROGRAM CALLING A PAL III SUBPROGRAM

This FORTRAN program calls a PAL III subprogram to convert the decimal numbers 20 through 30 into octal and to type the octal numbers on the Teletype printer.

All tape input is through the high-speed reader, and all tape output is from the high-speed punch. The Teletype keyboard is used to issue on-line commands and write the programs using the Editor program. The Teletype printer provides hard copy of all typed input, symbol tables, diagnostics, memory map, program listings, and program results.

To Use the System

Initialize (Section 1, Initializing the System)

Load RIM Loader (using high-speed reader version) (Figure RIM-1)

Load BIN Loader (Figure BIN-1)

To Generate Source Program Tapes

Load Editor (Figure BIN-2)

Start Editor at location 0200

Command Editor to append incoming text to text buffer; type source program.

```

A
C      * TYPE THE DECIMAL NUMBERS FROM 20 TO 30 IN OCTAL *
C
      DIMENSION TOCT(9),NUM(1)
      DO 50 I=20,30
      NUM=I
      PAUSE 3940
      TYPE 60
50;    CONTINUE
      STOP
60;    FORMAT(/)
      END

```

Reserve 1 location for number.
 Reserve 27 locations (3 x 9) for the subprogram TOCT
 Pause number = $3967_{10} - 27_{10} = 3940_{10}$
 ($3967_{10} = 7577_8$)

T	Output is on the high-speed punch (Figure EDIT-2)
P	Generate leader tape
F	Command Editor to punch entire text buffer; depress CONT
T	Insert FORM FEED character onto tape
	Generate trailer tape
	Remove punched tape from HSP and write some identification on its leader tape, e.g., FORTRAN Source, 6/19/68.
	Restart Editor at location 0176 to clear text buffer
	Command Editor to append incoming text to text buffer; type source program

A	
/ASSEMBLY PROGRAM TO TYPE AN OCTAL NUMBER	
	*7543
NUM,	0
TOCT,	0
	TAD NUM
	CLL RAL
	DCA TEM
	TAD M4
	DCA INDX
LOOP,	TAD TEM
	RTL
	RAL
	DCA TEM
	TAD TEM
	AND C7
	TAD C260
	TSF
	JMP *-1
	TLN
	CLA
	ISZ INDX
	JMP LOOP
	JMP I TOCT
/	
TEM,	0
INDX,	0
M4,	-4
C7,	7
C260,	260
\$	

T	Type CTRL/FORM to return to command mode
P	Output is on the high-speed punch (Figure EDIT-2)
F	Generate leader tape
T	Command Editor to punch entire text buffer; depress CONT
	Insert FORM FEED character onto tape
	Generate trailer tape
	Remove punched tape from HSP and write identifica- tion on leader tape, e.g., PAL III Source, 6/19/68.

To Generate PAL III
Object Program Tape

Load PAL III (Figure BIN-2)

Perform Pass 1 of assembly (Figure PAL-2)

Note error diagnostics, if any.

C260	7574
C7	7573
INDX	7571
LOOP	7552
M4	7572
NUM	7543
TEM	7570
TOCT	7544

Symbol table is typed on printer.

Perform Pass 2 of assembly (Figure PAL-2)

PAL III object tape is punched on the high-speed punch

Perform Pass 3 of assembly to get the program listing. (Figure PAL-2)

```

      /ASSEMBLY PROGRAM TO TYPE AN OCTAL NUMBER
      *7543
7543  0000  NUM,    0
7544  0000  TOCT,   0
7545  1343          TAD NUM
7546  7104          CLL RAL
7547  3370          DCA TEM
7550  1372          TAD M4
7551  3371          DCA INDX
7552  1370  LOOP,   TAD TEM
7553  7006          RTL
7554  7004          RAL
7555  3370          DCA TEM
7556  1370          TAD TEM
7557  0373          AND C7
7560  1374          TAD C260
7561  6041          TSF
7562  5361          JMP  .-1
7563  6046          TLS
7564  7200          CLA
7565  2371          ISZ INDX
7566  5352          JMP LOOP
7567  5744          JMP I TOCT
      /
7570  0000  TEM,    0
7571  0000  INDX,   0
7572  7774  M4,     -4
7573  0007  C7,      7
7574  0260  C260,   260

```

The program listing is
typed on the printer.

C260	7574
C7	7573
INDX	7571
LOOP	7552
M4	7572
NUM	7543
TEM	7570
TOCT	7544

The symbol table concludes
the program listing.

To Compile the FORTRAN Object Tape

I 7576
MUN 7575
 5206 7575

Load FORTRAN Compiler (Figure BIN-2)

Compile the FORTRAN object program
using the high-speed reader/punch
(Figure FORTRAN(4)-1)

Load Symbolprint (Figure BIN-2)

Execute Symbolprint to get memory map
(Figure FORTRAN(4)-2)

Lower and upper limits of the program

To Execute the Program

Load PAL III subprogram (Figure BIN-2)

Load FORTRAN Operating System
(Figure BIN-2)

Load compiled FORTRAN program
(Figure FORTRAN(4)-3)

Execute the stored FORTRAN and PAL III
programs (Figure FORTRAN(4)-4)

0024
0025
0026
0027
0030
0031
0032
0033
0034
0035
0036
!

The program results are typed on the
printer.

The ! (exclamation point) indicates that the
Operating System has come to the END of
the FORTRAN program.

A FOCAL PROGRAM CALLING A PAL III SUBPROGRAM

This is a demonstration of a FOCAL program calling an assembled PAL III subprogram to convert the decimal numbers 20 through 30 into octal and to type the octal numbers on the Teletype printer.

All tape input is through the low-speed reader, and all tape output is from the low-speed punch. The Teletype keyboard is used to write the PAL III subprogram, to issue on-line commands to the Editor program, and to communicate with FOCAL. As in the previous demonstration program, the Teletype printer provides hard copy of all input and output.

To Use the System

Initialize (Section 1, Initializing the System)

Load RIM Loader (using low-speed reader version) (Figure RIM-1)

Load BIN Loader (Figure BIN-1)

To Generate PAL III Source Tape

Load Editor (Figure BIN-2)

Start Editor at location 0200

Command Editor to append incoming text to text buffer.

Type the source program.

A

```
*411
XFNEW
*4550
XFNEW,      JMS I INTEGER    /GET INTEGER PART OF FLOATING AC
            TAD FLAC +1      /AND BRING INTO AC
            CLL RAL
            DCA TEM
            TAD M4           /INITIALIZE COUNTER
            DCA INDX
LOOP,      TAD TEM
            RTL
            RAL
            DCA TEM
            TAD TEM
            AND C7
            TAD C260
            JMS I OUTDEV     /CALL FOCAL'S TYPE ROUTINE
            ISZ INDX
            JMP LOOP
            JMP I EFUN3I     /RETURN TO MAIN PROGRAM.
/
TEM,        0
C7,         7
C260,       260
INDX,       0
M4,         -4
INTEGER=52
OUTDEV=147
EFUN3I=106
FLAC=44
*27
BOTTOM, 4550-1      /SHORTEN TEXT BUFFER TO ALLOW FOR FNEW.
$
```

4L Line 4 is listed
XFNEW, JMS I INTEGER /GET INTEGER PART OF FLOATING AC

4C The extra tab character is removed
XFNEW, JMS I INTEGER /GET INTEGER PART OF FLOATING AC

Type CTRL/FORM to return to Command Mode

Punch source tape on low-speed punch (Figure EDIT-2)

T Generate leader tape

P Command Editor to punch entire text buffer; depress CONT; source program is also typed out

```

*411
XFNEW
*4550
XFNEW, JMS I INTEGER /GET INTEGER PART OF FLOATING AC
      TAD FLAC +1 /AND BRING INTO AC
      CLL RAL
      DCA TEM
      TAD M4 /INITIALIZE COUNTER
      DCA INDX
LOOP, TAD TEM
      RTL
      RAL
      DCA TEM
      TAD TEM
      AND C7
      TAD C260
      JMS I OUTDEV /CALL FOCAL'S TYPE ROUTINE
      ISZ INDX
      JMP LOOP
      JMP I EFUN3I /RETURN TO MAIN PROGRAM.

```

```

/
TEM, 0
C7, 7
C260, 260
INDX, 0
M4, -4
INTEGER=52
OUTDEV=147
EFUN3I=106
FLAC=44
*27
BOTTOM, 4550-1 /SHORTEN TEXT BUFFER TO ALLOW FOR FNEW.
$

```

F Append a FORM FEED character to end of tape

T Generate trailer tape

Remove punched tape from LSP and write some identification on its leader tape, e.g., PAL III Source Tape, 6/19/68.

To Assemble and Generate
PAL III Object Tape

Load PAL III Assembler (Figure BIN-1)
Perform Pass 1 of assembly (Figure PAL-1)
Note error diagnostics, if any.

BOTTOM 0027
C260 4573
C7 4572
EFUN3I 0106
FLAC 0044
INDX 4574
INTEGE 0052
LOOP 4556
M4 4575
OUTDEV 0147
TEM 4571
XFNEW 4550

Symbol Table is typed on printer

Perform Pass 2 of assembly (Figure PAL-1)

Disregard meaningless characters typed
Note error diagnostics, if any.

Perform Pass 3 of assembly (Figure PAL-1)

BD%(E(\$*%99=<98899:;%'+.-0?<@%' +

0411	4550		*411 XFNEW *4550	
4550	4452	XFNEW,	JMS I INTEGER	/GET INTEGER PART OF FLOATING AC
4551	1045		TAD FLAC +1	/AND BRING INTO AC
4552	7104		CLL RAL	
4553	3371		DCA TEM	
4554	1375		TAD M4	/INITIALIZE COUNTER
4555	3374		DCA INDX	
4556	1371	LOOP,	TAD TEM	
4557	7006		RTL	
4560	7004		RAL	
4561	3371		DCA TEM	
4562	1371		TAD TEM	
4563	0372		AND C7	
4564	1373		TAD C260	
4565	4547		JMS I OUTDEV	/CALL FOCAL'S TYPE ROUTINE
4566	2374		ISZ INDX	
4567	5356		JMP LOOP	
4570	5506		JMP I EFUN3I	/RETURN TO MAIN PROGRAM.
		/		
4571	0000	TEM,	0	
4572	0007	C7,	7	
4573	0260	C260,	260	
4574	0000	INDX,	0	
4575	7774	M4,	-4	
		INTEGER=52		
		OUTDEV=147		
		EFUN3I=106		
		FLAC=44		
			*27	
0027	4547	BOTTOM,	4550-1	/SHORTEN TEXT BUFFER TO ALLOW FOR F

BOTTOM	0027
C260	4573
C7	4572
EFUN3I	0106
FLAC	0044
INDX	4574
INTEGE	0052
LOOP	4556
M4	4575
OUTDEV	0147
TEM	4571
XFNEW	4550

Symbol Table concludes the program
listing

To Execute the Programs

Load FOCAL (Figure BIN-2)

Start FOCAL at location 0200 and reply
to Initial Dialogue

CONGRATULATIONS!!

YOU HAVE SUCCESSFULLY LOADED 'FOCAL' ON A PDP-8 COMPUTER.

SHALL I RETAIN THE EXTENDED FUNCTIONS? :YES

DO YOU HAVE EXTENDED MEMORY? :NO

I SHALL USE 4K.

HAVE YOU LOADED THE DISC SYSTEM? :NO

Load PAL III Object Program tape
(Figure BIN-2)

*

?00.00

*

*1.10 C TYPE THE DECIMAL NUMBERS 20 THRU 30 IN OCTAL

*1.20 C

*1.30 FOR I=20,30; DO 2.0

*1.40 QUIT

*

*2.10 SET A=FNEW(I)

*2.20 TYPE !

*

*GO

0024

0025

0026

0027

0030

0031

0032

0033

0034

0035

0036

*

Restart FOCAL at location 0200 and type
the FOCAL program
(?00.00 denotes a manual restart)

Execute the FOCAL program and it will call
the PAL III subprogram to type the results
on the printer

A FOCAL PROGRAM

This program demonstrates the ease and convenience of FOCAL. In three lines of programming, FOCAL alone converts the decimal numbers 20 through 30 into octal and types them on the Teletype printer. The other line, line 1.1, is used to title the columns of typed numbers.

FOCAL is loaded into core memory using the high-speed reader. Communication with FOCAL is through the Teletype keyboard. The Teletype printer, as before, provides hard copy of all typed input and output.

To Use the System

Initialize (Section 1, Initializing the System)

Load HELP Loader (Figures HELP-1 and 2)

Load FOCAL (Figure BIN-2)

To Activate FOCAL for Use

Start FOCAL at location 0200 and reply to Initial Dialogue

CONGRATULATIONS!!

YOU HAVE SUCCESSFULLY LOADED 'FOCAL' ON A PDP-8 COMPUTER.

SHALL I RETAIN THE EXTENDED FUNCTIONS? :YES

DO YOU HAVE EXTENDED MEMORY? :NO

I SHALL USE 4K.

HAVE YOU LOADED THE DISC SYSTEM? :NO

Using FOCAL

Type the FOCAL program

```
*
*01.08 TYPE "NUMBER,DECIMAL          NUMBER,OCTAL",!
*01.10 FOR N=20,30; DO 2 ; TYPE %3,N,"          ",P,!
*
*02.10 S P=0; S M=N
*02.20 F J=0,4; S A=8↑(4-J); S D=FITR(M/A); S M=M-D*A; S P=P+D*10↑(4-J)
*
```

*GO

NUMBER,DECIMAL	NUMBER,OCTAL	Execute the FOCAL program
----------------	--------------	---------------------------

=+ 20	=+ 24	
=+ 21	=+ 25	
=+ 22	=+ 26	
=+ 23	=+ 27	
=+ 24	=+ 30	
=+ 25	=+ 31	
=+ 26	=+ 32	
=+ 27	=+ 33	
=+ 28	=+ 34	
=+ 29	=+ 35	
=+ 30	=+ 36	

*

APPENDICES

ASCII CHARACTER SET*

Character	8-Bit Octal	6-Bit Octal	Character	8-Bit Octal	6-Bit Octal
A	301	01	!	241	41
B	302	02	"	242	42
C	303	03	#	243	43
D	304	04	\$	244	44
E	305	05	%	245	45
F	306	06	&	246	46
G	307	07	'	247	47
H	310	10	(	250	50
I	311	11	)	251	51
J	312	12	*	252	52
K	313	13	+	253	53
L	314	14	,	254	54
M	315	15	-	255	55
N	316	16	.	256	56
O	317	17	/	257	57
P	320	20	:	272	72
Q	321	21	;	273	73
R	322	22	<	274	74
S	323	23	=	275	75
T	324	24	>	276	76
U	325	25	?	277	77
V	326	26	@	300	
W	327	27	[	333	33
X	330	30	\	334	34
Y	331	31	]	335	35
Z	332	32	↑	336	36
0	260	60	←	337	37
1	261	61	Leader/Trailer	200	
2	262	62	Line Feed	212	
3	263	63	Carriage Return	215	
4	264	64	Space	240	40
5	265	65	Rubout	377	
6	266	66	Blank	000	
7	267	67			
8	270	70			
9	271	71			

*An abbreviation for USA Standard Code for Information Interchange.

The following list of computer/programming terms is by no means complete. However, it does include many of the terms used in data processing.

Words underlined are defined elsewhere in this glossary.

Absolute Address	(1) An <u>address</u> that is permanently assigned by the machine designer to a storage location. (2) A pattern of <u>characters</u> that identifies a unique storage location without further modification.
Accumulator	A <u>register</u> in which the result of an operation is formed; Abbreviation: AC
Acronym	A word formed from the first letter or letters of the successive words of a multiple word term.
Accuracy	The degree of freedom from error, i.e., the degree of conformity to truth or to a rule.
Address	A <u>label</u> , name, or number which designates a <u>register</u> or a <u>location</u> where information is stored. That part of an instruction which specifies the location of an <u>operand</u> .
Address Register	A <u>register</u> in which an <u>address</u> is stored.
Algorithm	A prescribed set of well-defined rules or processes for the solution of a <u>problem</u> in a finite number of steps.
Alphabet	An ordered set of unique representations called characters, e.g., the 26 letters of the Roman alphabet.
Alphanumeric	Pertaining to a character set that contains both letters and numerals, and usually other characters.
Arithmetic Unit	The component of a computer where arithmetic and logical operations are performed.
ASCII	An abbreviation for USA Standard Code for Information Interchange.
Assemble	To <u>translate</u> from a symbolic (source) program to a <u>machine language</u> (object) program by substituting binary operation codes for symbolic operation codes and absolute or relocatable addresses for <u>symbolic addresses</u> .
Assembler	A <u>program</u> that assembles.
Auto-Indexing	When an absolute location 0010 through 0017 is addressed indirectly, the content of that location is incremented by one, rewritten in that same location, and then read as the <u>effective address</u> of the next instruction.
Auxiliary Operation	An operation performed by equipment not under direct control of the computer. Off-line operation.
Auxiliary Storage	Storage that supplements the primary storage.

Binary	(1) Pertaining to a characteristic or property involving a selection, choice, or condition in which there are two possibilities. (2) Pertaining to the numeration system with a <u>radix</u> of two.
Binary Digit	One of the symbols 1 or 0. A <u>digit</u> in the binary scale of notation; called a <u>bit</u> .
Bit	A <u>binary digit</u> .
Blank Character	A character used to produce a space on an output device.
Block	A set of things, such as words, characters, or digits, handled as a unit.
Bootstrap	A technique or device designed to bring itself into a desired state by means of its own action, e.g., a routine whose first few instructions are sufficient to bring the rest of itself into the computer from an input device.
Branch	A point in a <u>routine</u> where one of two or more choices is made under control of the routine, i.e., a conditional transfer (jump).
Buffer Storage	A part of core memory where information is stored temporarily during transfer; it may attempt to match the speeds of internal computation and the I/O device, thus permitting simultaneous computation and input/output.
Byte	A group of <u>binary digits</u> usually operated upon as a unit, e.g., 8-bit or 6-bit byte.
Call	To transfer control to a specified <u>routine</u> .
Calling Sequence	A specified set of instructions and data necessary to set up and <u>call</u> a given <u>routine</u> .
Carriage Return	The Teletype operation that causes the next character to be printed at the left margin.
Central Processing Unit	The unit of a computing system that includes the circuits controlling the interpretation and execution of instructions; the computer proper, excluding I/O and other peripheral devices.
Character	A single letter, numeral, or space mark used to represent information.
Clear	To erase the contents of a storage location by replacing the contents with blanks or zeros.
Closed Subroutine	A <u>subroutine</u> not stored in the main part of a program. Such a subroutine is entered by a jump operation and provision is made to return control to the main routine at the end of the subroutine.
Coding	To write instructions for a computer using symbols meaningful to the computer.
Command	A control signal, usually written as a <u>character</u> or group of characters, to direct action by a system program.
Compile	To produce a <u>machine language</u> routine from a routine written in <u>source language</u> by selecting appropriate subroutines from a subroutine library, as directed by the instructions or other symbols of the original routine, supplying the linkage which combines the subroutines into a workable routine and translating the subroutines and linkage into machine language.
Compiler	A <u>program</u> that compiles.
Complement	To form the negative of a binary word by replacing all 0 bits with 1 bits and vice versa.

Computer	A device capable of accepting information, processing it, and providing the results in a usable form.
Computer Program	A plan or <u>routine</u> for solving a problem on a computer.
Computer Word	A sequence of 12 <u>bits</u> treated as a unit and capable of being stored in one computer location.
Console	Usually the external front side of a device where controls and indicators are available for <u>manual operation</u> of the device.
Control Character	A character whose occurrence in a particular context initiates, modifies, or stops a control operation, e.g., a character to control carriage return.
Control Panel	The part of a device console that contains manual controls.
Convert	To change the representation of data from one form to another.
Copy	To reproduce data, leaving the original data unchanged.
Core Memory	The main <u>storage device</u> in the PDP-8 in which binary data is represented by the direction of magnetization in each unit of an array of magnetic material.
Cycle	To repeat a set of operations until a stated condition is met.
Cycle Time	An interval of time in which one set of events is completed.
Data	A general term used to denote any or all facts, numbers, letters, and symbols. It connotes basic elements of information which can be processed or produced by a computer.
Data Break	A facility which permits I/O transfers to occur simultaneously with program execution on a cycle-stealing basis.
Debug	To detect, locate, and correct mistakes in a program.
Decision	A determination of future action.
Delay	The amount of time by which an event is retarded.
Delimiter	A <u>character</u> that separates and organizes items of data.
Diagnostic	Pertaining to the detection and isolation of a malfunction or mistake.
Digit	A <u>character</u> used to represent one of the non-negative integers smaller than the <u>radix</u> , e.g., in binary notation, either 0 or 1.
Digital Computer	A device that operates on discrete data, performing sequences of arithmetic and logical operations on this data.
Direct Address	An <u>address</u> that specifies the <u>location</u> of an <u>operand</u> .
Display	A visual presentation of data.
Document	A medium on which information is recorded for human or machine use.
Double Precision	Pertaining to the use of two computer <u>words</u> to represent a number.
Downtime	The time interval during which a device is inoperative.
Dummy	An artificial address, instruction, or record of information inserted solely to fulfill prescribed conditions.
Dump	To copy the contents of all or part of core memory, usually onto an <u>external storage medium</u> .

Dynamic Dump	A <u>dump</u> that is performed during the execution of a program.
Edit	To rearrange information for machine input or output.
Effective Address	The <u>address</u> actually used in a particular execution of a computer instruction.
End-Around Carry	The action of adding the <u>most significant bit</u> of a binary number to the <u>least significant bit</u> .
Execute	To carry out an instruction or run a program on the computer.
Executive Routine	A <u>routine</u> that controls or monitors the execution of other routines.
External Storage	A facility or device, not an integral part of the computer, on which data usable by the computer is stored, such as paper tape, DECtape, or DECdisk.
File	A collection of related records treated as a unit.
Fixed Point	In a numeration system the position of the <u>radix</u> point is fixed with respect to one end of the numerals, according to some convention.
Flip-Flop	A basic computer circuit or device capable of assuming either one of two stable states at a given time.
Floating Point	A numeration system in which the position of the radix point is indicated by one part (the exponent part), the other part represents the significant digits (the fractional part).
Flowchart	A graphical representation of the sequence of instructions required to carry out a data processing operation.
Format	The arrangement of <u>data</u> .
Function	A specific purpose of an entity or its characteristic action.
Hardware	Physical equipment, e.g., mechanical, electrical, or electronic devices.
Head	A device that reads, records, or erases data on a <u>storage device</u> .
Heuristic	Pertaining to exploratory methods of problem solving.
I/O	<u>Input</u> or <u>output</u> or both.
Identifier	A symbol whose purpose is to identify, indicate, or name a body of data.
Indirect Address	An <u>address</u> in a computer <u>instruction</u> which indicates a <u>location</u> where the address of the referenced <u>operand</u> is to be found.
Initialize	To set counters, switches, and addresses to zero or other starting values at the beginning of, or at prescribed points in, a computer routine.
Input	The transferring of data from auxiliary or <u>external storage</u> into the <u>internal storage</u> of the computer.
Instruction	A set of bits (in an object program) or characters (in a source program) which as a unit cause the computer to perform an operation.
Internal Storage	The storage facilities forming an integral physical part of the computer and directly controlled by the computer. Also called main memory and core memory.
Interrupt	To stop a process in such a way that it can be resumed.
Jump	A departure from the normal sequence of executing instructions in a computer.

Label	An <u>identifier</u> .
Language	A set of representations, conventions, and rules used to convey information.
Leader	The blank section of tape at the beginning of the tape.
Least Significant Digit	The rightmost digit of a binary number.
Library	An organized collection of standard and proven <u>routines</u> and <u>subroutines</u> which can be incorporated in larger routines.
Library Routine	A proven <u>routine</u> that is maintained in a program library.
Load	To place data into <u>internal storage</u> .
Location	A place in <u>storage</u> or <u>memory</u> where a unit of <u>data</u> or an <u>instruction</u> may be stored.
Loop	A sequence of <u>instructions</u> that is executed repeatedly until a <u>terminal</u> condition prevails.
Machine Instruction	An <u>instruction</u> written in <u>machine language</u> .
Machine Language	A <u>language</u> designed for interpretation and use by the machine without translation.
Macro Instruction	An <u>instruction</u> in a <u>source language</u> that is equivalent to a specified sequence of <u>machine instructions</u> .
Manual Input	The entry of data by hand into a <u>device</u> at the time of processing.
Manual Operation	The processing of data in a system by direct manual techniques.
Memory	(1) The erasable <u>storage</u> in the computer. (2) Pertaining to a <u>device</u> in which data can be stored and from which it can be retrieved.
No Op	An <u>instruction</u> that specifically instructs the computer to do nothing, except to proceed to the next instruction in sequence.
Object Program	The <u>machine language</u> program which is the output after translation from the <u>source language</u> . The binary program which runs on the computer.
Octal	(1) Pertaining to a characteristic or property involving a selection, choice, or condition in which there are eight possibilities. (2) Pertaining to the numeration system with a <u>radix</u> of eight.
Off Line	Pertaining to equipment or devices not under direct control of the computer.
On Line	Pertaining to equipment or devices under direct control of the computer; also to programs operating directly and immediately to user commands, e.g., FOCAL and DDT.
Open Subroutine	A <u>subroutine</u> that must be relocated and inserted into a <u>routine</u> at each place it is used.
Operand	That which is effected, manipulated, or operated upon.
Origin	The <u>absolute address</u> of the beginning of a program.
Output	Information transferred from the <u>internal storage</u> of a computer to output devices or <u>external storage</u> .

Overflow	The generation of a quantity beyond the capacity of a <u>register</u> .
Page	In the PDP-8/1, a unit of 200 (octal) locations which may be addressed directly.
Patch	To modify a <u>routine</u> in a rough or expedient way.
Predefined Process	A named process consisting of one or more operations or program steps that are specified elsewhere in a routine.
Procedure	The course of action taken for the solution of a problem.
Processor	A <u>computer program</u> that includes the <u>compiling</u> , <u>assembling</u> , <u>translating</u> , and related functions for a specific programming language.
Program	The complete sequence of <u>instructions</u> and <u>routines</u> necessary to solve a problem.
Program Library	A collection of available <u>computer programs</u> and <u>routines</u> .
Programming Language	A <u>language</u> used to prepare <u>computer programs</u> .
Protected Location	A <u>storage location</u> reserved for special purposes in which data cannot be stored without undergoing a screening procedure to establish suitability for storage therein.
Punched Paper Tape	A paper tape on which a pattern of holes is used to represent <u>data</u> .
Pushdown List	A list that is constructed and maintained so that the next item to be retrieved is the most recently stored item in the list, i.e., last in, first out.
Radix	The quantity of characters for use in each of the digital positions of a numbering system.
Read	To transfer information from an input device to <u>internal storage</u> ; also refers to the internal acquisition of data from memory.
Real Time	Pertaining to computation performed while the related physical process is taking place so that results of the computation can be used in guiding the physical process.
Record	A collection of related items of <u>data</u> , treated as a unit.
Register	A device capable of storing a specified amount of data, such as one word.
Reset	To restore a <u>storage device</u> to a prescribed state.
Restart	To reestablish the execution of a <u>program</u> .
Routine	A set of <u>instructions</u> arranged in proper sequence to cause the computer to perform a desired task.
Run	A single, continuous performance of a <u>program</u> .
Scan	To examine sequentially part by part.
Search	To examine a set of items for those that have a desired property.
Set	To place a <u>storage device</u> into a specified state.
Single Step	Operation of the computer in which each <u>instruction</u> is performed in response to a single manual operation.

Skip	To ignore one or more <u>instructions</u> in a sequence of instructions.
Software	The collection of <u>programs</u> and <u>routines</u> associated with the computer.
Source Language	A symbolic <u>language</u> that is an <u>input</u> to a given translation process.
Source Program	A <u>program</u> written in a symbolic (<u>source</u>) <u>language</u> .
Statement	A meaningful expression or generalized <u>instruction</u> in a <u>source language</u> .
Step	One operation in a <u>routine</u> .
Storage Allocation	The assignment of blocks of data to specified <u>blocks of storage</u> .
Storage Capacity	The amount of <u>data</u> that can be contained in a <u>storage device</u> .
Storage Device	A <u>device</u> into which data can be entered, in which it can be held, and from which it can be retrieved.
Store	To enter <u>data</u> into a <u>storage device</u> .
String	A connected sequence of entities such as characters in a command string.
Subroutine	A <u>routine</u> that can be part of another routine.
Switch	A device or programming technique for making selections.
Symbolic Address	An <u>address</u> expressed in symbols convenient to the programmer. A <u>label</u> .
Symbolic Coding	Writing instructions using symbolic notation instead of actual machine instruction notation.
System	An assembly of <u>software</u> and <u>hardware</u> united to form an organized whole.
Tape Drive	A <u>device</u> that moves tape past a <u>head</u> .
Temporary Storage	<u>Storage locations</u> reserved for intermediate results.
Terminal	A point in a system at which data can either enter or leave.
Time Sharing	The interleaving of the time of a <u>device</u> .
Toggle	Pertaining to the operation of a <u>flip-flop</u> or switch.
Translate	To convert from one <u>language</u> to another.
Underflow	The condition that arises when a computation yields a result whose magnitude is smaller than the system is capable of representing.
Variable	A quantity that can assume any of a given set of values.
Word	A 12-bit unit of data in the PDP-8/I which may be stored in one addressable location.
Word Length	The number of <u>bits</u> in a <u>word</u> .
Write	To transfer information from internal storage to an output device or to auxiliary storage.

In order to run a program on the computer, instructions and data must first be fed into the computer from the input device.

The program and data could be typed into the computer on-line. However, computer time is valuable, and hunt-and-peck typing on-line can be an expensive process. For this reason, it may be desirable to prepare the program and data off-line, that is, to punch the program and data onto paper tape using a separate machine, one not actually connected to the computer.

The ASR33 Teletype can be used off-line to prepare source program tapes, to duplicate tapes, and to edit tapes previously punched in the ASCII format. (Tapes punched from the Teletype keyboard are in ASCII format.)

When the Teletype power control switch is turned to LOCAL, the unit becomes an off-line tape preparation facility. Procedures for using the Teletype off-line are listed below. The Teletype controls are described in Section 1, ASR33 Teletype, and are shown in Figure INTRO-2.

DUPLICATING TAPES

- a. Turn TTY to LOCAL.
- b. Set LSR to FREE.
- c. Put original tape into LSR.
- d. Depress LSP ON.
- e. Depress HERE IS key to generate leader tape.
- f. Set LSR to START. (New tape is punched and data is typed on printer.)
- g. After the original tape is read in, depress HERE IS key to generate trailer tape.
- h. Remove tapes from LSR and LSP.

PREPARING NEW PROGRAM TAPES

When preparing a program tape off-line, the user should observe the same conventions of his programming language as when preparing a program on-line using Editor. Following are the manual operating procedures for off-line tape preparation.

- a. Turn TTY to LOCAL.
- b. Depress LSP ON.

- c. Depress HERE IS key to generate leader tape.
- d. Type the source program, observing the conventions of the programming language being used.

NOTE

The RETURN and LINE FEED keys must be depressed at the end of each line.

Depressing the CTRL/TAB keys perforates the tab character onto the tape, and the typewheel moves only one position to the right. When the computer reads the punched tab character on output, it will cause the typewheel to tab (a tab is usually equal to 10 spaces).

- e. After the source program is punched, depress HERE IS to generate trailer tape.
- f. Remove the source program tape from LSP.

CORRECTING TYPING ERRORS

Typing errors can be corrected using the B. SP. button and the RUBOUT key. The B. SP. button backspaces the tape one column for each depression of the button, and the RUBOUT key perforates all eight channels of the tape (this perforation is ignored by the computer).

EDITING

Punched tapes can be edited off-line. However, the user must be able to read the perforations on the tape, otherwise, off-line editing is virtually impossible.

- a. Turn TTY to LOCAL.
- b. Set LSR to FREE.
- c. Put tape to be edited into LSR.
- d. Depress LSP ON.
- e. Depress HERE IS to generate leader tape.
- f. Set LSR to START.
- g. Observe the printer as the program is being typed, and
- h. Set LSR to STOP a few characters ahead of the text to be edited.
- i. Advance the tape one character at a time by toggling the LSR control from START to STOP.

For Minor Edit:

Advance tape past the text to be edited and use the B. SP. and RUBOUT keys to erase old text, then type and punch new text.

For Major Edit:

- (1) Set LSR to STOP one character ahead of the text to be edited.
 - (2) Type new text.
 - (3) Set LSR to FREE.
 - (4) Advance tape past edited area (reading the perforated tape).
 - (5) Set LSR to START.
- j. Repeat from step f until editing is completed.
- k. Set LSR to START.
- l. After new source program tape is punched, depress HERE IS to generate trailer tape.
- m. Remove old tape from LSR and discard.*
- n. Remove new tape from LSP and save.

*It's good programming practice to list the new tape before discarding the old, ensuring that the new tape is correct.

SUMMARY OF PDP-8/I SUBROUTINES

Name	DEC Number	Calling Sequence*	Memory Locations (Decimal)
		<u>Function Subroutines</u>	
1. Square Root Single Precision	FMAA	- /Square in AC JMS SQRT /Call - /Return with root	23
2. Signed Multiply Single Precision	FMBA	- /Multiplier in AC JMS MULT /Call ADDRESS /Address of multipli- /cand - /Return. High order product in AC; low order in MPI	44
3. Signed Divide Single Precision	FMCA	- /High dividend in AC JMS DIVIDE /Call LOWD /Low dividend DIVSOR /Divisor - /Return quotient in AC; remainder in HDIVND	62
4. Signed Multiply Double Precision	FMDA	- /AC ignored JMS DMUL /Call HORDMD /Address of high order multiplicand HORDMR /Address of high order multiplier - /Return high order prod- uct in AC. Remainder of product in B, C, D.	125
5. Signed Divide Double Precision	FMEA	- /Address of high order dividend in AC JMS DUBDIV /Call HORDDR /Address of high order divisor - /Return. High order quo- tient in AC; low order quotient in DIVND4. High and low remainder in DIVND 1 and DIVND2	105

*All of the calling sequences here assume that the data is in the correct format and that there are no overflow conditions to check upon completion. For details on the data and indicators for overflow conditions, the user is referred to the appropriate program write-up.

Name	DEC Number	Calling Sequence*	Memory Locations (Decimal)
<u>Function Subroutines (Cont)</u>			
6. Sine Routine Double Precision	FMFA	- /AC = 0000 JMS DSIN /Call ADDRESS /Address of high order word - /Return. AC = 0 1 = 0 Answer in ADDRESS and ADDRESS + 1	248 (+ double precision multiply)
7. Cosine Routine Double Precision	FMGA	- /AC = 0000 JMS DCOS /Call ADDRESS /Address of high order word - /Return. AC = 0 1 = 0 Answer in ADDRESS and ADDRESS + 1	64 (+ double precision sine and double precision multiply)
8. Four-Word Floating Point Package	FMHA	See Floating Point System Programming Manual, Digital-8-5-S	1041
9. Signed Multiply (EAE) Single Precision	8-21-F	- /Multiplier in AC JMS MULT /Call ADDRESS /Address of multiplicand - /Return. Most significant product in AC; least significant in MP1	
10. Signed Divide (EAE) Single Precision	8-22-F	- /High dividend in AC JMS SPDIV /Call LOWD /Low dividend DIVSOR /Divisor - /Return. Quotient in AC; remainder in DVD.	45
11. Signed Multiply (EAE) Double Precision	8-23-F	- /AC ignored JMS DMUL /Call HORDMD /Address of high order multiplicand HORDMR /Address of high order multiplier /Return high order product in AC; remainder in B, C, and D.	104

*All of the calling sequences here assume that the data is in the correct format and that there are no overflow conditions to check upon completion. For details on the data and indicators for overflow conditions, the user is referred to the appropriate program write-up.

Name	DEC Number	Calling Sequence*	Memory Locations (Decimal)
12. EAE Floating Point Package	8-25-F	<u>Function Subroutines (Cont)</u> See Floating Point System Programming Manual, Digital-8-5-S	See 8-5-S
<u>Punch Programs</u>		<u>Utility Programs</u>	
1. RIM Punch	PMPO	Binary tape (see write-up)	
2. BIN Punch	8-5-U	Binary tape (see write-up)	
ASR33			
75A			
<u>Processor Programs</u>			
1. Logical Subroutines	FMIA	- /One argument in AC	
Inclusive OR		JMS INCOR /Call	
		ADDRES /Address of second argument	12
		- /Return	
Exclusive OR		JMS EXCOR /One argument in AC	
		/Call	
		ADDRES /Address of second argument	14
2. Arithmetic Shift	FMJA	(General Calling Sequence)	
		- /Negative number of shifts in AC	
		JMS** /Call	
		ADDRES /Address to be shifted	
		- /Return with shifted number in AC	
Shift Left, Single Prec.		**SPSL	12
Shift Right, Single Prec.		**SPSR	15
Shift Left, Double Prec.		**DPSL (Least significant part in LSH)	24
Shift Right, Double Prec.		**DPSR (Least significant part in LSH)	
3. Logical Shift	FMKA		
Shift Right, Single Prec.		**LSRSP	12
Shift Right, Double Prec.		**LSRDP (Least significant part in LESTSG)	24
(Left Shift Logical; identical to left shift arithmetic)			

*All of the calling sequences here assume that the data is in the correct format and that there are no overflow conditions to check upon completion. For details on the data and indicators for overflow conditions, the user is referred to the appropriate program write-up.

Name	DEC Number	Calling Sequence*	Memory Locations (Decimal)
<u>Utility Programs (Cont)</u>			
4. EAE Instruction Set Simulator	8-17-U	(See write-up)	
<u>BCD - Binary Conversion</u>			
1. BCD-to-Binary Conversion	8-10-U	- /BCD number in AC JMS DCDBIN /Call	26
2. BCD-to-Binary Conversion	8-11-U		
3. Binary-to-BCD	8-14-U	- /Binary number in AC JMS BCD /Call	33
4. Binary-to-BCD Conversion (Used primarily for writing mag tape in BCD Format)	8-15-U	- /BCD number in AC - /AC contains binary number JMS BCD /Call - /Return. BCD number in ONE and TWO	53
<u>Teletype Message Sub-Routines</u>			
1. Alphanumeric Message	8-18-U	- /AC ignored JMS MESSAGE/Call - /First two characters of message .) .) /Remaining characters .) .) "XX" represents the last character XX00 /End of message code - /Return	51
2. Teletype Output Package	8-19-U	Return is to location following "call" AC = 0000	75
Type One Character		- (/AC 0-5 = 00 /AC 6-11 = trimmed code JMS TYPE /Call	
Type Two Characters		- /AC 0-5 = 1st character /AC 6-11 = 2nd character JMS TY2 /Call	
Type a Digit		- /AC 8-11 = digit JMS TDIG /Call	

*All of the calling sequences here assume that the data is in the correct format and that there are no overflow conditions to check upon completion. For details on the data and indicators for overflow conditions, the user is referred to the appropriate program write-up.

Name	DEC Number	Calling Sequence*	Memory Locations (Decimal)
<u>Utility Programs (Cont)</u>			
Type a Space		JMS TYSP /Call	
Type a CR and LF		JMS TYCR /Call	
Type a Tab		JMS TYTB /Call	
3. Character String Typeout	8-20-U	- /AC = initial address JMS TYPSTG /Call of string - /Return. AC clear	64
<u>Decimal Print Subroutines</u>			
1. Unsigned Decimal Print, Single Prec.	8-22-U	- /AC contains numbers JMS DECPRT /Call - /Return. AC clear	38
2. Signed Decimal Print, Single Prec.	8-23-U	- /AC contains number JMS SSPRNT /Call - /Return. AC clear	48
3. Unsigned Decimal Print, Double Prec.	8-24-U	- /AC ignored JMS UDPRNT /Call ADRESS /Address of high order word - /Return. AC clear	73
4. Signed Decimal Print, Double Prec.	8-25-U	- /AC ignored JMS SDPRNT /Call ADRESS /Address of high order word - /Return. AC clear	86
<u>Decimal Input Routines</u>			
1. Decimal to Binary Input. Signed or Unsigned, Single Prec.	8-28-U	- /AC ignored JMS SICONV /Call - /Return. AC contains number	74
2. Decimal to Binary Input, Signed or Unsigned, Double Prec.	8-29-U	- /AC ignored JMS DICONV /Call ADRESS /Address for high order word - /Return. AC clear	110
<u>Miscellaneous</u>			
1. Octal Memory Dump	8-6-U	None	77
2. DECtape Library System Loader	8-3-U	(See write-up)	17

*All of the calling sequences here assume that the data is in the correct format and that there are no overflow conditions to check upon completion. For details on the data and indicators for overflow conditions, the user is referred to the appropriate program write-up.

Name	DEC Number	Calling Sequence*	Memory Locations (Decimal)
		<u>Utility Programs (Cont)</u>	
3. Incremental Plotter Subroutines	8-12-U	(See write-up)	128
4. Symbolic Tape Format Generator	8-21-U	(See write-up)	-
5. DECtape Subroutine	SUB0-D	(See write-up)	256

*All of the calling sequences here assume that the data is in the correct format and that there are no overflow conditions to check upon completion. For details on the data and indicators for overflow conditions, the user is referred to the appropriate program write-up.

HOW TO OBTAIN SOFTWARE INFORMATION

Announcements for new and revised software, as well as programming notes, software problems, and documentation corrections are published monthly by Software Information Service in the following newsletters.

Digital Software News for the PDP-8 Family
Digital Software News for the PDP-9 Family

These newsletters contain information applicable to software available from Digital's Program Library (see title page for address). Software products and documents are usually shipped only after the Program Library receives a specific request from a user.

Digital Equipment Computer Users Society (DECUS) maintains a user library and publishes a catalog of programs as well as the DECUSCOPE magazine for its members and non-members who request it.

Please complete the card below to receive information on DECUS membership or to place your name on the newsletter mailing list.

Please send

☐ DECUS membership information,
or add my name to the
☐ DECUSCOPE non-membership list.

And, send me the Digital Software News for the

☐ PDP-8 ☐ PDP-9

NAME _____

COMPANY _____

ADDRESS _____

CITY _____ STATE _____ ZIP _____

DECUS
Digital Equipment Corporation
146 Main Street
Maynard, Mass. 01754



Postage will be paid by:

BUSINESS REPLY MAIL
NO POSTAGE STAMP NECESSARY IF MAILED IN THE UNITED STATES

FIRST CLASS
PERMIT NO. 33
MAYNARD, MASS.

..... Do Not Tear - Fold Here and Staple

..... Fold Here

READER'S COMMENTS

Digital Equipment Corporation maintains a continuous effort to improve the quality and usefulness of its publications. To do this effectively we need user feedback -- your critical evaluation of this manual.

Please comment on this manual's completeness, accuracy, organization, usability, and readability.

Did you find errors in this manual? _____

How can this manual be improved? _____

DEC also strives to keep its customers informed of current DEC software and publications. Thus, the following periodically distributed publications are available upon request. Please check the appropriate box(s) for a current issue of the publication(s) desired.

- ☐ PDP-8/I Software Manual Update, a quarterly collection of revisions to current software manuals.
- ☐ PDP-8/I User's Bookshelf, a bibliography of current software manuals.
- ☐ Program Library Price List, a list of currently available Family of 8 software programs and manuals.

Please describe your position. _____

Name _____ Organization _____

Street _____ Department _____

City _____ State _____ Zip or Country _____

Postage will be paid by:

digital

Digital Equipment Corporation
Software Quality Control
Building 12
146 Main Street
Maynard, Mass. 01754

BUSINESS REPLY MAIL
NO POSTAGE STAMP NECESSARY IF MAILED IN THE UNITED STATES

FIRST CLASS
PERMIT NO. 33
MAYNARD, MASS.

Do Not Tear - Fold Here and Staple

Fold Here